



Курс-квест TypeScript Middle+

Модуль 1 - Пролог Зачем TypeScript Middle-разработчику

Урок 1 - Что ожидается от Middle-уровня

Шаг 01:

Миссия 1

Ты только устроился в новую команду, и тимлид спрашивает: «Что должен уметь разработчик уровня Middle по TypeScript?»

Подсказка: Подумай: от Middle ждут уверенного владения инструментом, а не поверхностных знаний.

Ответ:

- Понимать и использовать сложные типы (Generics, Utility Types)
- Знать только базовые типы number, string, boolean
- Всегда писать код без типизации
- Игнорировать ошибки компилятора

Шаг 02:

Миссия 2

Вечером за пиццей коллеги обсуждают: «А что самое главное для Middle-разработчика — скорость или качество?»

Подсказка: В реальных проектах чаще всего ценится поддерживаемость и надёжность.

Ответ:

- Всегда писать максимально быстро, даже жертвуя качеством

- **Ставить на первое место читаемость и надёжность кода**
- Ориентироваться только на личное удобство
- Избегать тестов ради экономии времени

Шаг 03:

Миссия 3

Утром на планёрке тебя просят объяснить разницу между Junior и Middle. Какой ответ верный?

Подсказка: Junior чаще всего нуждается в подсказках, а Middle уже может работать более автономно.

Ответ:

- **Middle умеет самостоятельно принимать технические решения и объяснять их команде**
- Junior всегда лучше Middle, потому что у него больше энергии
- Middle работает только под контролем старших коллег
- Разницы между Junior и Middle нет

Шаг 04:

Миссия 4

Ты обсуждаешь новый проект с коллегой. Он предлагает писать весь код на JavaScript без типов. Какое решение выберет Middle-разработчик?

Подсказка: Подумай, зачем используется TypeScript.

Ответ:

- **Выбрать TypeScript для предотвращения ошибок на этапе компиляции**
- Согласиться и писать без типов
- Использовать только комментарии для описания типов
- Оставить решение на потом

Шаг 05:

Миссия 5

На собеседовании тебя спрашивают: «Что важнее для Middle — знать синтаксис или архитектуру?»

Подсказка: Синтаксис — это база, но уровень Middle оценивают иначе.

Ответ:

- Только синтаксис TypeScript
- **Понимание архитектурных подходов и умение применять их в TS**
- Умение быстро находить чужой код в интернете
- Запоминание всех возможных ошибок наизусть

Шаг 06:

Миссия 6

Ты получил задачу с legacy-кодом, где всё написано без типов. Какой шаг сделает Middle-разработчик?

Подсказка: Важно не переписать всё сразу, а двигаться постепенно.

Ответ:

- **Постепенно добавлять типы в проблемные места, сохраняя рабочий функционал**

- Полностью переписать проект с нуля
- Игнорировать ошибки и оставить всё как есть
- Удалить legacy-код без замены

🔮 Шаг 07:

Миссия 7

Коллега просит объяснить, зачем нужны Generics. Какой ответ даст Middle?

Подсказка: Generics — это про гибкость и переиспользование.

Ответ:

- **Чтобы писать универсальный код, работающий с разными типами**
- Чтобы заменить все типы на any
- Чтобы сократить количество строк кода любой ценой
- Чтобы скрыть ошибки от компилятора

🔮 Шаг 08:

Миссия 8

Сопоставь обязанности Middle-разработчика с их описанием.

Подсказка: Подумай: Middle уже не новичок, но ещё не архитектор.

Ответ:

- **Код-ревью** — Анализ кода коллег с рекомендациями по улучшению
- **Архитектура модуля** — Выбор структуры для части приложения
- **Работа с тестами** — Написание unit-тестов для проверки функционала
- **Поддержка новичков** — Помощь Junior-разработчикам в понимании кода

🔮 Шаг 09:

Миссия 9

Тимлид попросил объяснить, что значит «писать поддерживаемый код». Сопоставь признаки с объяснениями.

Подсказка: Поддерживаемый код легко читать и изменять.

Ответ:

- **Понятные имена** — Названия функций и переменных отражают их суть
- **Минимум дублирования** — Код повторяется как можно реже
- **Модульность** — Код разделён на независимые части
- **Документация** — Есть краткие комментарии там, где это необходимо

🔮 Шаг 10:

Миссия 10

Сопоставь инструмент TypeScript с его назначением.

Подсказка: Вспомни, зачем нужны интерфейсы, типы и utility types.

Ответ:

- **Interface** — Описание структуры объекта
- **Type alias** — Создание псевдонима для типа
- **Generics** — Написание универсального кода
- **Partial** — Делает все свойства типа необязательными

🔗 Шаг 11:

Миссия 11

Сопоставь уровень разработчика с типичными характеристиками.

Подсказка: Обрати внимание на степень самостоятельности.

Ответ:

- **Junior** — Нуждается в постоянных подсказках
- **Middle** — Может самостоятельно решать задачи среднего уровня
- **Senior** — Принимает ключевые архитектурные решения
- **Tech Lead** — Руководит процессом разработки команды

🔗 Шаг 12:

Миссия 12

Сопоставь проблемы командной разработки с их последствиями.

Подсказка: Подумай, что чаще всего мешает команде работать эффективно.

Ответ:

- **Нет код-ревью** — Ошибки попадают в продакшен
- **Игнорирование типов** — Неожиданные ошибки в рантайме
- **Плохая документация** — Сложности при подключении новых разработчиков
- **Отсутствие тестов** — Трудно вносить изменения без страха сломать код

🔗 Шаг 13:

Миссия 13

Расставь шаги роста разработчика от Junior до Middle в правильном порядке.

Подсказка: Сначала — базовые знания, затем — уверенность в задачах, а потом — архитектура.

Ответ:

- Освоение базовых типов и синтаксиса
- Решение простых задач под присмотром
- Автономное выполнение задач среднего уровня
- Понимание архитектурных решений

🔗 Шаг 14:

Миссия 14

Расположи шаги грамотного код-ревью в правильном порядке.

Подсказка: Сначала нужно понять задачу, потом проверить код, а уже после — обсудить улучшения.

Ответ:

- Изучение контекста задачи
- Проверка читаемости и структуры
- Поиск возможных багов
- Предложение улучшений

🔗 Шаг 15:

Миссия 15

Расставь этапы работы над задачей Middle-разработчика в правильной последовательности.

Подсказка: Подумай: сначала требования, потом код, тесты и только потом деплой.

Ответ:

- Анализ требований
- Написание кода
- Написание тестов
- Деплой

🔮 **Шаг 16:**

Миссия 16

Расположи действия при работе с legacy-кодом в правильном порядке.

Подсказка: Важно сначала понять код, потом покрыть тестами, а уже потом менять.

Ответ:

- Изучение текущей логики
- Написание тестов для проверки поведения
- Внесение улучшений
- Проверка, что всё работает как раньше

📖 **Урок 2 - Проблемы JS и как их решает TS**

🔮 **Шаг 01:**

Миссия 1

Ты работаешь над старым проектом на JavaScript и замечаешь, что функция возвращает разные типы данных в зависимости от входа. Что здесь может стать проблемой?

Подсказка: Подумай, что произойдет, если один раз вернётся строка, а другой — число.

Ответ:

- **Неопределённость типов, которая может привести к ошибкам в рантайме**
- Более быстрая работа кода
- Автоматическое исправление ошибок интерпретатором
- Оптимизация памяти

🔮 **Шаг 02:**

Миссия 2

На демо клиент жалуется: кнопка иногда работает, иногда нет. Ты находишь, что ошибка связана с тем, что переменная была undefined. Как бы помог TypeScript?

Подсказка: Вспомни про строгую проверку типов и null-safety.

Ответ:

- **TypeScript предупредил бы о возможном значении undefined на этапе компиляции**
- TypeScript автоматически исправил бы баг
- TypeScript сделал бы кнопку всегда кликабельной
- TypeScript отключил бы проверки полностью

🔮 **Шаг 03:**

Миссия 3

Твой коллега утверждает: «JavaScript достаточно, чтобы писать большие проекты». В чём главный риск?

Подсказка: Вспомни, как сложно сопровождать код без статической типизации.

Ответ:

- Приложение не сможет работать в браузере
- Нельзя будет использовать модули
- **Рост числа ошибок в рантайме и трудности поддержки**
- Снижение скорости компиляции

Шаг 04:

Миссия 4

На ревью ты видишь функцию JS, которая принимает объект user, но никто не знает, какие у него свойства. Чем поможет TS?

Подсказка: Вспомни про интерфейсы и type alias.

Ответ:

- **Явное описание структуры объекта через интерфейс или тип**
- Случайный выбор свойств на этапе исполнения
- Автоматическое добавление всех возможных свойств браузером
- Игнорирование проверки свойств

Шаг 05:

Миссия 5

Твоя команда обсуждает: «Почему ошибки в JS иногда видны только на продакшене?» Что бы ответил Middle-разработчик?

Подсказка: Подумай, на какой стадии можно поймать ошибки в TS.

Ответ:

- **Потому что JS проверяет ошибки только во время исполнения, а TS ловит их при компиляции**
- Потому что JS всегда игнорирует ошибки
- Потому что браузеры сами исправляют код
- Потому что JS требует ручной компиляции

Шаг 06:

Миссия 6

Ты пишешь функцию в JS и случайно передаёшь строку вместо числа. Чем поможет TS?

Подсказка: Подумай о проверке типов аргументов функций.

Ответ:

- **Сообщит об ошибке типов при компиляции**
- Исправит строку на число автоматически
- Игнорирует ошибку и продолжит
- Сделает функцию работать только со строками

Шаг 07:

Миссия 7

В JS легко случайно использовать глобальную переменную. Как это решает TS?

Подсказка: Вспомни про строгий режим и проверку областей видимости.

Ответ:

- **Предупреждает об использовании необъявленных переменных**
- Автоматически добавляет все переменные в глобальный объект
- Игнорирует глобальные переменные
- Отключает блочную область видимости

🔮 **Шаг 08:**

Миссия 8

Сопоставь проблему JavaScript с тем, как её решает TypeScript.

Подсказка: Подумай о статической типизации, интерфейсах и строгих проверках.

Ответ:

- **Неопределённые свойства объекта** — Интерфейсы описывают обязательные поля
- **Ошибки типов на продакшене** — Компилятор ловит ошибки до запуска
- **Сложно понять, какие значения принимает функция** — Generics и строгая типизация параметров
- **Проблемы с null и undefined** — Включение strictNullChecks

🔮 **Шаг 09:**

Миссия 9

Сопоставь возможности TypeScript с тем, какие проблемы JS они решают.

Подсказка: Обрати внимание на расширения языка.

Ответ:

- **Type alias** — Делает код читаемее, сокращает дублирование
- **Enums** — Избавляют от магических строк
- **Union Types** — Позволяют явно описать несколько возможных вариантов
- **Readonly модификатор** — Запрещает изменять данные, которые должны быть константными

🔮 **Шаг 10:**

Миссия 10

Ты объясняешь новичку, зачем нужны интерфейсы в TS. Сопоставь термин с определением.

Подсказка: Интерфейсы — это про описание структуры.

Ответ:

- **Interface** — Задаёт форму объекта
- **Optional property** — Позволяет делать поля необязательными
- **Readonly property** — Запрещает изменять значение свойства
- **Extends** — Наследует структуру одного интерфейса от другого

🔮 **Шаг 11:**

Миссия 11

Сопоставь, что чаще всего вызывает баги в JS, и как это предотвращает TS.

Подсказка: Подумай про строгую типизацию и контракты функций.

Ответ:

- **Динамическая типизация** — TS добавляет статическую типизацию

- **Ошибки в сигнатуре функций** — TS проверяет параметры и возвращаемое значение
- **Использование `any` повсюду** — TS предлагает строгий режим и linting
- **Случайное изменение объекта** — TS позволяет использовать `readonly`

🔮 Шаг 12:

Миссия 12

Сопоставь особенности JS и решение TS.

Подсказка: Вспомни, чем именно статическая типизация полезна.

Ответ:

- **Отсутствие проверки типов до запуска** — TS проверяет код при компиляции
- **Трудно работать с большими командами** — TS облегчает коммуникацию через типы
- **Неочевидные ошибки при изменении кода** — TS гарантирует корректность контрактов
- **Сложность в автодополнении** — TS улучшает поддержку IDE

🔮 Шаг 13:

Миссия 13

Расставь шаги работы компилятора TS в правильном порядке.

Подсказка: Подумай, что идёт раньше: проверка или генерация JS.

Ответ:

- Анализ кода и проверка типов
- Сообщение об ошибках при несоответствиях
- Трансляция TS в JS
- Генерация итоговых файлов

🔮 Шаг 14:

Миссия 14

Ты объясняешь стажёру, как JS-проблемы решаются с помощью TS. Расставь шаги по важности.

Подсказка: Сначала база — типы, затем структура, после — `null-checks`.

Ответ:

- Добавление статической типизации
- Введение интерфейсов и типов
- Включение строгих проверок (`strict mode`)
- Оптимизация архитектуры

🔮 Шаг 15:

Миссия 15

Расположи шаги работы Middle-разработчика при миграции проекта с JS на TS.

Подсказка: Сначала нужно подготовить проект, потом вводить типы постепенно.

Ответ:

- Настройка `tsconfig.json`
- Добавление базовых типов
- Переписывание ключевых модулей
- Введение строгого режима

Шаг 16:

Миссия 16

Тимлид просит тебя объяснить коллегам, как TS предотвращает баги. Расставь шаги в порядке. Подсказка: Вспомни, как работает статическая типизация.

Ответ:

- Проверка кода до выполнения
- Выявление ошибок типов
- Исправление проблем до релиза
- Снижение числа багов в продакшене

Урок 3 - Обзор возможностей TypeScript 5+

Шаг 01:

Миссия 1

Алексей пишет код на старом проекте и не понимает, зачем обновлять компилятор. В TS 5.0 появились новые возможности. Что из перечисленного — новинка?

Подсказка: Вспомните про поддержку модификатора `satisfies`.

Ответ:

- **Модификатор `satisfies` для проверки типов**
- Поддержка классов
- Дженерики
- Интерфейсы

Шаг 02:

Миссия 2

Марина хочет гарантировать, что объект соответствует интерфейсу, но без расширения. Какая новая возможность TS 5 помогает ей?

Подсказка: Это ключевое слово проверяет совместимость, а не изменяет объект.

Ответ:

- **Оператор `satisfies`**
- Оператор `extends`
- Оператор `implements`
- Оператор `as`

Шаг 03:

Миссия 3

Игорь заметил, что в TS 5+ улучшили поддержку декораторов. Для чего это сделано?

Подсказка: Декораторы нужны для аннотаций и метапрограммирования.

Ответ:

- **Добавление метаданных к классам**
- **Изменение поведения методов**
- Замена дженериков
- Удаление интерфейсов

Шаг 04:

Миссия 4

Настя увидела ошибку типов и узнала, что TS 5+ улучшил поддержку конфигурации в tsconfig.json . Что изменилось?

Подсказка: Думайте о возможностях расширения конфигов.

Ответ:

- **Поддержка нескольких extends в конфиге**
- Возможность писать конфиг только в YAML
- Запрет на использование strict
- Автоматическое создание кода

Шаг 05:

Миссия 5

Даниил работает с огромным проектом. TS 5+ ускорил компиляцию. Какая из этих оптимизаций появилась?

Подсказка: Обратите внимание на кеширование и инкрементальные сборки.

Ответ:

- **Оптимизация инкрементальной сборки**
- Удаление системы типов
- Запрет JSX
- Полный отказ от Node.js

Шаг 06:

Миссия 6

Лена слышала про улучшения работы с Enum в TS 5+. Что добавили?

Подсказка: Речь о строгих проверках.

Ответ:

- **Усиленная проверка совместимости enum**
- Автоматическая генерация SQL
- Замена enum на any
- Удаление enum из языка

Шаг 07:

Миссия 7

Антон работает с JSX и React. Какое улучшение в TS 5+ его порадует?

Подсказка: Вспомните про автоматическое определение JSX-файлов.

Ответ:

- **Улучшенная поддержка JSX и автоматическое определение jsxImportSource**
- Отмена поддержки JSX
- Полный переход на Vue
- Удаление типов из React

Шаг 08:

Миссия 8

Сопоставьте нововведение TS 5+ с его описанием.

Подсказка: Каждая пара связывает конкретную фичу с её объяснением.

Ответ:

- **Оператор `satisfies`** — Гарантирует, что объект соответствует интерфейсу, без изменения типа.
- **Новые декораторы** — Позволяют аннотировать классы и методы для метапрограммирования.
- **Улучшенный JSX** — Облегчает работу с React и автоматизирует настройки.
- **Расширенный `tsconfig`** — Поддержка нескольких `extends` для конфигов.

Шаг 09:

Миссия 9

Разработчик хочет быстро вспомнить, какие улучшения связаны с компиляцией, а какие — с синтаксисом. Сопоставьте категории и примеры.

Подсказка: Одни фичи касаются производительности, другие — языка.

Ответ:

- **Оптимизация сборки** — Инкрементальная компиляция стала быстрее.
- **Синтаксические улучшения** — Добавлен оператор `satisfies`.
- **Работа с `enum`** — Более строгая проверка совместимости.
- **JSX** — Автоматическое определение `jsxImportSource`.

Шаг 10:

Миссия 10

Сопоставьте версию TypeScript с ключевым изменением.

Подсказка: Сосредоточьтесь только на TS 5.0+.

Ответ:

- **TS 5.0** — Появился оператор `satisfies`.
- **TS 5.1** — Улучшена работа с JSX.
- **TS 5.2** — Усиленная проверка `enum`.
- **TS 5.3** — Оптимизация инкрементальной сборки.

Шаг 11:

Миссия 11

Ты попал на собеседование. Сопоставь вопрос интервьюера и правильный ответ.

Подсказка: Вопросы касаются конкретных нововведений TS 5.

Ответ:

- **Что делает оператор `satisfies`?** — Гарантирует соответствие объекта типу без изменения его структуры.
- **Для чего нужны новые декораторы?** — Для аннотаций и модификации поведения классов и методов.
- **Что улучшено в `tsconfig`?** — Теперь можно использовать несколько `extends`.
- **Что изменилось в `enum`?** — Появились более строгие проверки.

Шаг 12:

Миссия 12

В команде спор: кто помнит, какие новинки влияют на читаемость кода, а какие — на стабильность сборки? Сопоставь.

Подсказка: Делим на визуальные улучшения и технические.

Ответ:

- **Читаемость кода** — Новые декораторы, улучшенный JSX.
- **Стабильность сборки** — Оптимизация инкрементальной компиляции, расширенный tsconfig.

Шаг 13:

Миссия 13

Расположи шаги использования оператора `satisfies` при работе с объектом.

Подсказка: Вспомни: сначала объект, потом проверка, затем использование.

Ответ:

- Объявить объект
- Применить оператор `satisfies` к интерфейсу
- Использовать объект в коде с уверенностью в его типе

Шаг 14:

Миссия 14

Расположи по порядку, как работает новый декоратор в TS 5+.

Подсказка: Декоратор сначала применяется, затем вносит изменения, и только потом метод используется.

Ответ:

- Определить декоратор
- Применить к классу или методу
- Выполнить метод с учетом изменений

Шаг 15:

Миссия 15

Расположи действия разработчика при настройке `tsconfig` с несколькими `extends`.

Подсказка: Важно понимать, как сливаются конфиги.

Ответ:

- Создать базовый конфиг
- Добавить второй конфиг для конкретного окружения
- Указать `extends` в основном файле
- Запустить сборку

Шаг 16:

Миссия 16

Порядок работы компилятора TS 5+ при оптимизированной инкрементальной сборке:

Подсказка: Думай о кешировании и повторной сборке.

Ответ:

- Сохранение кеша после первой сборки
- Проверка изменений в файлах

- Компиляция только изменённых частей
- Обновление кеша

Модуль 2 - Сильная типизация глубокие основы

Урок 1 - Прimitives, объекты, массивы

Шаг 01:

Миссия 1

Даша пишет скрипт на TypeScript и случайно присваивает строку в переменную, где ожидается число. Какой примитивный тип предотвратит такую ошибку?

Подсказка: Вспомни базовые типы: string, number, boolean.

Ответ:

- **number**
- string
- boolean
- any

Шаг 02:

Миссия 2

Игорь пишет функцию, которая возвращает true/false в зависимости от наличия пользователя. Какой тип возвращаемого значения ему стоит явно указать?

Подсказка: Логическое выражение всегда сводится к одному из двух значений.

Ответ:

- **boolean**
- number
- string
- null

Шаг 03:

Миссия 3

Света хранит имя пользователя в переменной. Какой примитивный тип ей лучше выбрать?

Подсказка: Текстовые данные — это всегда...

Ответ:

- **string**
- number
- boolean
- object

Шаг 04:

Миссия 4

Никита использует массив чисел для хранения очков игроков. Какой тип массива он должен указать?

Подсказка: Вспомни синтаксис записи массива в TypeScript.

Ответ:

- **number[]**
- Array
- string[]
- object[]

🔮 **Шаг 05:**

Миссия 5

Оля описывает объект пользователя с полями name (строка) и age (число). Какой синтаксис верный?

Подсказка: Вспомни объявление объекта через фигурные скобки с ключами и типами.

Ответ:

- **{ name: string; age: number }**
- { name: number; age: string }
- { name: boolean; age: string }
- { user: string }

🔮 **Шаг 06:**

Миссия 6

Артем хочет, чтобы переменная могла хранить только одно из значений: 'success', 'error' или 'loading'. Какой тип ему нужен?

Подсказка: Это разновидность строкового типа, ограничивающая выбор.

Ответ:

- **string literal union**
- boolean
- number
- any

🔮 **Шаг 07:**

Миссия 7

Катя делает объект настроек, где одно из свойств может быть либо строкой, либо числом. Какой тип лучше использовать?

Подсказка: Подумай о возможности объединения типов через специальный оператор.

Ответ:

- **string | number**
- object
- any
- never

🔮 **Шаг 08:**

Миссия 8

Сопоставь примитивные типы с их описанием.

Подсказка: Обрати внимание: число и строка — это разные вселенные.

Ответ:

- **number** — Представляет как целые, так и дробные числа.
- **string** — Последовательность символов, заключённая в кавычки.
- **boolean** — Может быть true или false.
- **null** — Явно заданное отсутствие значения.

🔮 Шаг 09:

Миссия 9

Сопоставь массивы с их правильным описанием.

Подсказка: Вспомни разницу между string[] и number[].

Ответ:

- **string[]** — Массив строк.
- **number[]** — Массив чисел.
- **boolean[]** — Массив значений true/false.
- **object[]** — Массив объектов.

🔮 Шаг 10:

Миссия 10

Сопоставь объекты с их описанием.

Подсказка: Каждый объект — набор ключей с типами.

Ответ:

- **{ name: string }** — Объект с полем name, которое строка.
- **{ age: number }** — Объект с полем age, которое число.
- **{ active: boolean }** — Объект с полем active, которое true/false.
- **{ }** — Пустой объект без полей.

🔮 Шаг 11:

Миссия 11

Сопоставь сложные типы с их определением.

Подсказка: Тут пригодятся знания про union и literal types.

Ответ:

- **string | number** — Тип, который может быть строкой или числом.
- **'yes' | 'no'** — Строковый литеральный тип.
- **any** — Может быть чем угодно, без ограничений.
- **never** — Тип, который невозможен к достижению.

🔮 Шаг 12:

Миссия 12

Сопоставь ключевые слова с их назначением в TypeScript.

Подсказка: Вспомни, чем null отличается от undefined.

Ответ:

- **null** — Явное отсутствие значения.
- **undefined** — Значение по умолчанию для необъявленных переменных.
- **object** — Набор свойств с типами.

- **array** — Упорядоченная коллекция элементов одного типа.

Шаг 13:

Миссия 13

Расположи шаги создания массива в правильном порядке.

Подсказка: Сначала объявляется переменная, потом указывается тип, затем значения.

Ответ:

- Объявить переменную
- Указать тип массива
- Присвоить значения
- Использовать в коде

Шаг 14:

Миссия 14

Расположи шаги работы с объектом.

Подсказка: Сначала определи структуру, потом создай объект.

Ответ:

- Определить интерфейс/тип
- Создать объект
- Заполнить поля
- Использовать объект в коде

Шаг 15:

Миссия 15

Расположи типы по степени строгости от самого строгого к самому слабому.

Подсказка: Подумай, какой тип максимально ограничивает значения.

Ответ:

- never
- boolean
- string | number
- any

Шаг 16:

Миссия 16

Расположи действия при объявлении union-типа.

Подсказка: Сначала назови переменную, потом укажи типы через |, и только после этого присвой значения.

Ответ:

- Назвать переменную
- Указать тип через |
- Присвоить допустимое значение
- Использовать в коде

🔧 Шаг 17:

Миссия 17

Расположи шаги при описании объекта пользователя в правильном порядке.

Подсказка: Не забудь: интерфейс или type обычно идут до создания переменной.

Ответ:

- Объявить интерфейс User
- Создать переменную user
- Заполнить поля name и age
- Использовать объект в логике программы

📖 Урок 2 - Кортежи и кортежи с именованными полями

🔧 Шаг 01:

Миссия 1

Михаил работает с API, которое возвращает пару: имя пользователя и его возраст. Какой тип лучше всего использовать для представления этой пары?

Подсказка: Важно, чтобы первый элемент всегда был строкой, а второй — числом.

Ответ:

- **type UserData = [string, number];**
- type UserData = (string | number)[];
- type UserData = object;
- type UserData = { name: string; age: number };

🔧 Шаг 02:

Миссия 2

Ольга использует кортеж для представления RGB-цвета. Какой тип лучше всего подходит?

Подсказка: Каждый цветовой канал — это число.

Ответ:

- **type RGB = [number, number, number];**
- type RGB = number[];
- type RGB = [string, string, string];
- type RGB = [number, number?];

🔧 Шаг 03:

Миссия 3

Алексей решил использовать кортеж с именованными полями: type Point = [x: number, y: number] . Что это даёт?

Подсказка: Именованные поля не влияют на типизацию напрямую.

Ответ:

- **Улучшает читаемость кода, но тип остаётся [number, number]**
- Меняет тип на объект с полями x и y
- Запрещает использовать индексацию через числа
- Добавляет возможность менять порядок элементов

🔮 Шаг 04:

Миссия 4

Никита хочет использовать кортеж для описания HTTP-ответа: статус (число), сообщение (строка), данные (объект, необязательный). Какой вариант корректен?

Подсказка: Помни о необязательных элементах.

Ответ:

- **type HttpResponse = [number, string, object?];**
- type HttpResponse = [string, number, object?];
- type HttpResponse = [number, string, object]; // всегда 3 элемента
- type HttpResponse = (number | string | object)[];

🔮 Шаг 05:

Миссия 5

Сопоставьте типы кортежей с их назначением.

Подсказка: Подумайте о порядке и содержании элементов.

Ответ:

- **[string, number]** — Имя пользователя и его возраст
- **[number, number, number]** — RGB-цвет
- **[boolean, string]** — Флаг успеха и сообщение
- **[string, number?]** — Название и необязательный код

🔮 Шаг 06:

Миссия 6

Сопоставьте описание с правильным кортежем.

Подсказка: Обратите внимание на необязательные элементы.

Ответ:

- **Точка в 2D-пространстве** — [number, number]
- **HTTP-ответ с необязательными данными** — [number, string, object?]
- **Флаг + сообщение** — [boolean, string]
- **Название и код (необязательный)** — [string, number?]

🔮 Шаг 07:

Миссия 7

Сопоставьте кортежи с ситуациями из жизни разработчиков.

Подсказка: Представьте реальные примеры.

Ответ:

- **[string, number]** — Имя файла и его размер
- **[number, string]** — Код ошибки и сообщение
- **[boolean, object]** — Флаг успеха и данные
- **[string, number, boolean?]** — Название, версия, флаг стабильности

🔮 Шаг 08:

Миссия 8

Сопоставьте кортежи с их назначением в проектах.

Подсказка: Думайте о реальных данных, которые часто встречаются.

Ответ:

- **[string, Date]** — Имя задачи и дата дедлайна
- **[number, number, number]** — Координаты в 3D
- **[boolean, string]** — Результат проверки и описание
- **[string, string, number?]** — ФИО сотрудника и необязательный возраст

Шаг 09:

Миссия 9

Расположите шаги создания кортежа в правильном порядке.

Подсказка: Сначала объявление типа, потом использование.

Ответ:

- Объявить тип кортежа
- Присвоить значения
- Использовать элементы по индексам
- Добавить необязательный элемент (при необходимости)

Шаг 10:

Миссия 10

Расположите шаги работы с кортежем [string, number] в порядке.

Подсказка: Сначала объявляем, потом присваиваем, затем используем.

Ответ:

- Объявление кортежа
- Присваивание значения
- Доступ к элементам
- Передача в функцию

Шаг 11:

Миссия 11

Упорядочите этапы добавления именованных полей в кортеж.

Подсказка: Сначала создаётся базовый кортеж, затем добавляются имена.

Ответ:

- Создать кортеж [number, number]
- Добавить имена: [x: number, y: number]
- Присвоить значения
- Использовать для улучшения читаемости

Шаг 12:

Миссия 12

Упорядочите шаги использования кортежа в API-ответе.

Подсказка: Сначала определяем структуру, потом возвращаем данные.

Ответ:

- Определить тип кортежа
- Использовать его в описании ответа

- Вернуть данные в соответствии с кортежем
- Деструктурировать значения

Урок 3 - Enum vs Literal Types

Шаг 01:

Миссия 1

Катя создает список валют для интернет-магазина. У неё только три валюты: USD, EUR, RUB. Что проще и безопаснее применить?

Подсказка: Подумайте: если список короткий и фиксированный, то enum может быть избыточным.

Ответ:

- Enum
- **String literal union**
- Number literal union
- Любая строка

Шаг 02:

Миссия 2

Сергей работает над системой логирования, где уровней больше 10 (debug, info, warn, error и т.д.). Что лучше использовать?

Подсказка: Подумайте: когда значений много, лучше выбрать инструмент, где удобно управлять и расширять набор.

Ответ:

- **Enum**
- Literal types
- Простые строки
- Массив строк

Шаг 03:

Миссия 3

Иван пишет библиотеку графиков. У него десятки режимов отображения. Что даст больше удобства при масштабировании?

Подсказка: Когда список значений со временем растёт, лучше иметь централизованный контроль.

Ответ:

- **Enum**
- Literal union
- Любая строка
- Number array

Шаг 04:

Миссия 4

У Олега есть фишка: выбор дня недели. Он думает — enum или literal union? (понедельник, вторник... воскресенье)

Подсказка: Значений семь, но список фиксирован и редко меняется.

Ответ:

- Enum
- **Literal union**
- Любая строка
- Boolean

 Шаг 05:

Миссия 5

Миша создает библиотеку для управления статусами заказов (Pending, Shipped, Delivered, Canceled). Какой подход будет более универсален?

Подсказка: Учтите: бизнес-логика может расширяться новыми статусами.

Ответ:

- **Enum**
- Literal union
- Массив строк
- Boolean

 Шаг 06:

Миссия 6

Расположите шаги выбора между enum и literal union.

Подсказка: Сначала проанализируйте список значений, потом подумайте о расширяемости.

Ответ:

- Определить размер и изменчивость списка
- Рассмотреть частоту расширений
- Выбрать literal union при фиксированных значениях
- Выбрать enum при больших и изменяемых списках

 Шаг 07:

Миссия 7

Расположите преимущества literal union в порядке от простого к сложному.

Подсказка: Начните с базовой компактности, закончите безопасностью типов.

Ответ:

- Короткая запись
- Легкость чтения
- Ограничение значений
- Повышенная безопасность типов

 Шаг 08:

Миссия 8

Расположите преимущества enum от базовых к более продвинутым.

Подсказка: Начните с читаемости, закончите поддержкой больших систем.

Ответ:

- Читаемость и автодополнение
- Централизованность значений

- Простое добавление новых элементов
- Поддержка больших проектов

🔧 Шаг 09:

Миссия 9

Расположите шаги анализа проекта перед выбором enum или literal union.

Подсказка: Подумайте: сначала анализ требований, затем оценка списка, потом выбор типа.

Ответ:

- Проанализировать требования проекта
- Определить количество возможных значений
- Понять, насколько часто список будет меняться
- Сделать выбор между enum и literal union

📖 Урок 4 - Union, Intersection, Discriminated Unions

🔧 Шаг 01:

Миссия 1

Анна пишет чат-бота и использует тип `string | number` для ID пользователей. Что означает такой тип?

Подсказка: Представь, что у разных пользователей могут быть либо числовые ID, либо строковые UUID.

Ответ:

- **Переменная может содержать либо строку, либо число**
- Переменная должна содержать строку и число одновременно
- Тип всегда преобразуется в строку
- Это сокращение для типа `any`

🔧 Шаг 02:

Миссия 2

Денис проектирует API и использует тип `{name: string} & {age: number}`. Что он получает?

Подсказка: Intersection объединяет все обязательные свойства в один тип.

Ответ:

- **Объект, который должен содержать и name, и age**
- Объект, который может содержать либо name, либо age
- Объект без обязательных свойств
- Это то же самое, что `union`

🔧 Шаг 03:

Миссия 3

Ира реализует систему уведомлений. Тип сообщения описан так: `{type: 'email', address: string} | {type: 'sms', phone: string}`. Какой это паттерн?

Подсказка: Этот подход помогает отличать варианты объекта по специальному полю.

Ответ:

- **Discriminated Union**
- Intersection Type

- Union без дискриминатора
- Literal Type

🔗 Шаг 04:

Миссия 4

Разработчик хочет описать переменную, которая может хранить только 'on' или 'off'. Какой тип выбрать?

Подсказка: Иногда union используется с литеральными значениями.

Ответ:

- 'on' | 'off'
- boolean
- string
- any

🔗 Шаг 05:

Миссия 5

Петр пишет функцию, которая принимает параметр string | null . Для чего здесь используется union?

Подсказка: Иногда union помогает явно указать на возможность отсутствия значения.

Ответ:

- Чтобы параметр мог быть либо строкой, либо null
- Чтобы параметр всегда приводился к строке
- Чтобы параметр был только null
- Чтобы избежать проверки типов

🔗 Шаг 06:

Миссия 6

Сопоставь ключевые понятия с их определениями:

Подсказка: Union — это «или», Intersection — это «и», а Discriminated Union — способ их умного объединения.

Ответ:

- **Union** — Тип, который допускает одно из нескольких значений
- **Intersection** — Тип, который объединяет свойства всех включённых типов
- **Discriminated Union** — Union с дополнительным дискриминирующим полем
- **Literal Union** — Union, состоящий из конкретных значений

🔗 Шаг 07:

Миссия 7

Свяжи примеры кода с типом, который они иллюстрируют:

Подсказка: Взгляни на структуру: фигурные скобки и дискриминатор намекают на Discriminated Union.

Ответ:

- **type A = string | number** — Union
- **type B = {a: number} & {b: string}** — Intersection
- **type C = {kind: 'x', value: number} | {kind: 'y', value: string}** — Discriminated Union

- **type D** = 'left' | 'right' — Literal Union

🔮 Шаг 08:

Миссия 8

Соедини разработчиков с типами, которые они используют:

Подсказка: Кто-то любит чёткие варианты значений, кто-то объединяет всё сразу.

Ответ:

- **Анна** — Union для ID: string | number
- **Денис** — Intersection для профиля: {name} & {age}
- **Ира** — Discriminated Union для уведомлений
- **Петр** — Literal Union для флагов: 'on' | 'off'

🔮 Шаг 09:

Миссия 9

Сопоставь типы с их поведением в коде:

Подсказка: Подумай, какие операции требуют проверки ключевого поля.

Ответ:

- **Union** — Требуется проверка типа перед использованием
- **Intersection** — Объединяет все свойства, нельзя пропустить ни одно
- **Discriminated Union** — Легко проверять по полю kind/type
- **Literal Union** — Ограничивает переменную конкретными строковыми значениями

🔮 Шаг 10:

Миссия 10

Сопоставь преимущества с соответствующими конструкциями:

Подсказка: Intersection усиливает строгую структуру, Union даёт гибкость.

Ответ:

- **Union** — Позволяет работать с несколькими альтернативами
- **Intersection** — Создаёт строгий комбинированный контракт
- **Discriminated Union** — Удобные проверки switch-case
- **Literal Union** — Лёгкая валидация строковых констант

🔮 Шаг 11:

Миссия 11

Сопоставь ситуации с выбором правильного типа:

Подсказка: Если у тебя фиксированный набор строк — это Literal Union.

Ответ:

- **Статусы заказа: 'new' | 'shipped' | 'delivered'** — Literal Union
- **Пользовательский ID: string или number** — Union
- **Объект с данными профиля и настройками** — Intersection
- **Разные форматы уведомлений с полем type** — Discriminated Union

🔮 Шаг 12:

Миссия 12

Расположи шаги работы с Discriminated Union:

Подсказка: Сначала смотришь на поле-дискриминатор, потом обрабатываешь соответствующие данные.

Ответ:

- Проверить значение дискриминатора
- Выбрать нужную ветку кода
- Обработать свойства конкретного варианта
- Завершить выполнение функции

Шаг 13:

Миссия 13

Расположи шаги проектирования Union:

Подсказка: Сначала определяешь возможные варианты, потом используешь проверки в коде.

Ответ:

- Определить набор возможных типов
- Объявить тип как объединение
- Использовать проверки `typeof/instanceof`
- Реализовать бизнес-логику

Шаг 14:

Миссия 14

Расположи шаги проектирования Intersection:

Подсказка: Сначала создаёшь базовые типы, потом объединяешь их и работаешь с результатом.

Ответ:

- Определить несколько базовых типов
- Объявить новый тип с помощью `&`
- Создать объект, удовлетворяющий обоим контрактам
- Использовать объект в коде

Шаг 15:

Миссия 15

Расположи шаги использования Literal Union:

Подсказка: Вспомни про фиксированный список строковых значений.

Ответ:

- Определить набор строковых литералов
- Создать `union` из этих литералов
- Использовать в параметрах функции
- Ограничить ввод только этими значениями

Шаг 16:

Миссия 16

Расставь шаги от идеи до кода для Discriminated Union:

Подсказка: Не забудь про ключевое поле-дискриминатор.

Ответ:

- Определить варианты сущности
- Выбрать общее поле-дискриминатор
- Объявить union с этим полем
- Использовать switch для обработки

 Шаг 17:

Миссия 17

Расположи шаги тестирования union-типов:

Подсказка: Проверка каждого варианта — ключ к надёжности.

Ответ:

- Создать тестовые данные для каждого варианта
- Вызвать функцию с этими данными
- Проверить корректность обработки
- Зафиксировать результат в тестах

 Урок 5 - Nullable и Non-Null Assertion

 Шаг 01:

Миссия 1

Аня получает данные с сервера, и переменная `userName: string | null`. Что означает такой тип?

Подсказка: Обрати внимание, что иногда сервер может вернуть пустое значение.

Ответ:

- **Переменная может быть строкой или null**
- Переменная всегда содержит строку
- Переменная не может быть null
- Тип автоматически преобразуется в string

 Шаг 02:

Миссия 2

Борис хочет использовать Non-Null Assertion для переменной `inputValue: string | null`. Что делает оператор `!`?

Подсказка: Он как бы говорит компилятору: «Я уверен, что здесь не будет null».

Ответ:

- **Гарантирует, что значение не null или undefined в этой точке**
- Автоматически присваивает значение по умолчанию
- Проверяет значение на null во время выполнения
- Превращает тип в any

 Шаг 03:

Миссия 3

Валерия работает с формой, где поле `age: number | null`. Как безопасно получить значение, чтобы не вызвать ошибку?

Подсказка: Проверь, не равно ли значение null перед использованием.

Ответ:

- **Сделать проверку: `if (age !== null) {...}`**
- Присвоить число без проверки
- Использовать `undefined` вместо `null`
- Привести к `string`

🔗 Шаг 04:

Миссия 4

Иван использует `Non-Null Assertion` в коде `element!.value`. Что произойдет, если элемент на самом деле равен `null`?

Подсказка: Подумай про реальное выполнение кода в браузере.

Ответ:

- **Во время выполнения будет ошибка**
- Компилятор автоматически присвоит значение
- Значение будет `undefined`
- Код безопасно пропустит `null`

🔗 Шаг 05:

Миссия 5

Светлана получает объект `profile: {name: string | null, age: number | null}`. Какой подход безопаснее для доступа к свойствам?

Подсказка: Подумай про проверку значений перед использованием.

Ответ:

- **Проверять каждое свойство на `null` перед использованием**
- Использовать `!` для всех свойств без проверок
- Присвоить любое значение по умолчанию
- Игнорировать `null`, TypeScript сам разберется

🔗 Шаг 06:

Миссия 6

Сопоставь концепции `Nullable` и `Non-Null Assertion` с их определениями:

Подсказка: `Nullable` указывает на возможность отсутствия значения, `Non-Null Assertion` — уверенность, что значение присутствует.

Ответ:

- **`Nullable`** — Тип переменной может быть `null` или `undefined`
- **`Non-Null Assertion`** — Оператор `!` утверждает, что значение не `null` и не `undefined`

🔗 Шаг 07:

Миссия 7

Сопоставь преимущества с соответствующими подходами:

Подсказка: `Non-Null Assertion` ускоряет код, `Nullable` повышает безопасность.

Ответ:

- **`Nullable`** — Позволяет безопасно работать с отсутствующими значениями
- **`Non-Null Assertion`** — Обходит проверку типов, когда мы уверены в наличии значения

Шаг 08:

Миссия 8

Сопоставь разработчиков с подходами, которые они используют:

Подсказка: Кто проверяет null, а кто уверен в наличии значения.

Ответ:

- **Аня** — Nullable для userName
- **Борис** — Non-Null Assertion для inputValue
- **Валерия** — Nullable для age
- **Иван** — Non-Null Assertion для element

Шаг 09:

Миссия 9

Расположи шаги безопасного использования Nullable:

Подсказка: Сначала проверяем значение, потом используем.

Ответ:

- Проверить переменную на null
- Если не null, использовать её значение
- Иначе обработать случай отсутствия значения
- Продолжить работу с безопасным значением

Шаг 10:

Миссия 10

Расположи шаги использования Non-Null Assertion:

Подсказка: Убедись, что значение существует, прежде чем использовать !.

Ответ:

- Определить переменную, которая может быть null
- Убедиться, что значение существует
- Применить оператор !
- Использовать значение без ошибок типов

Шаг 11:

Миссия 11

Расположи шаги при обработке формы с Nullable полями:

Подсказка: Обработка должна быть безопасной даже при отсутствии значений.

Ответ:

- Проверить каждое поле на null
- Использовать значение, если не null
- Заполнить дефолтные значения при необходимости
- Отправить форму

Шаг 12:

Миссия 12

Расположи шаги при работе с Non-Null Assertion для DOM элементов:

Подсказка: Убедись, что элемент есть, прежде чем добавлять !.

Ответ:

- Получить элемент через `querySelector/getElementById`
- Проверить, что элемент существует
- Применить !
- Использовать свойства элемента

Урок 6 - Утилиты **Partial, Required, Readonly, Pick, Omit**

Шаг 01:

Миссия 1

Сопоставь утилиты TypeScript с их описанием:

Подсказка: Подумай о том, какие утилиты делают поля опциональными, обязательными или защищают от изменения.

Ответ:

- **Partial** — Делает все свойства объекта необязательными
- **Required** — Делает все свойства объекта обязательными
- **Readonly** — Запрещает изменение свойств объекта после создания
- **Pick** — Создает новый тип, выбирая только указанные свойства
- **Omit** — Создает новый тип, исключая указанные свойства

Шаг 02:

Миссия 2

Сопоставь разработчиков с утилитами, которые они применяют:

Подсказка: Подумай, кто делает свойства обязательными, кто — только чтение.

Ответ:

- **Аня делает все поля формы необязательными** — Partial
- **Борис делает настройки обязательными** — Required
- **Валерия защищает объект от изменений** — Readonly
- **Иван выбирает только name и email** — Pick
- **Светлана исключает password** — Omit

Шаг 03:

Миссия 3

Сопоставь типовые ситуации с подходящей утилитой:

Подсказка: Обрати внимание, где нужны опциональные свойства, где — защита от изменений.

Ответ:

- **Создание объекта конфигурации с опциональными полями** — Partial
- **Создание объекта настроек, где все поля обязательны** — Required
- **Объект с данными, которые нельзя менять после инициализации** — Readonly
- **Выбор только некоторых свойств из типа User** — Pick
- **Удаление ненужного поля password из типа** — Omit

Шаг 04:

Миссия 4

Сопоставь разработчиков с их утилитами:

Подсказка: Кто делает поля обязательными, кто защищает объект от изменений.

Ответ:

- **Анна создаёт объект профиля постепенно** — Partial
- **Борис делает все настройки обязательными** — Required
- **Валерия запрещает изменение данных после создания** — Readonly
- **Иван выбирает только необходимые поля** — Pick
- **Светлана исключает конфиденциальное поле** — Omit

Шаг 05:

Миссия 5

Расположи шаги создания объекта с Partial:

Подсказка: Сначала можно оставить поля пустыми, затем заполнить их по мере необходимости.

Ответ:

- Создать объект с Partial
- Заполнить нужные поля
- Передать объект в функцию
- Использовать объект в логике программы

Шаг 06:

Миссия 6

Расположи шаги защиты объекта с Readonly:

Подсказка: Сначала создаём объект, затем применяем Readonly.

Ответ:

- Создать объект
- Применить Readonly
- Попытаться изменить поле (ошибка TypeScript)
- Использовать объект безопасно

Шаг 07:

Миссия 7

Расположи шаги использования Pick:

Подсказка: Сначала выбираем свойства, затем формируем новый тип, потом используем.

Ответ:

- Выбрать нужные поля из типа
- Создать новый тип с Pick
- Создать объект нового типа
- Использовать объект в программе

Шаг 08:

Миссия 8

Расположи шаги использования Omit:

Подсказка: Сначала исключаем ненужные поля, потом создаём объект нового типа.

Ответ:

- Определить поля для исключения
- Создать новый тип с Omit
- Создать объект нового типа
- Использовать объект в программе

 Шаг 09:

Миссия 9

Сопоставь преимущества утилит с их назначением:

Подсказка: Подумай, какая утилита делает свойства опциональными, какая защищает объект.

Ответ:

- **Partial** — Позволяет создавать объекты постепенно, заполняя только нужные поля
- **Required** — Гарантирует, что все поля обязательны
- **Readonly** — Защищает объект от изменений после создания
- **Pick** — Позволяет выбрать только нужные поля
- **Omit** — Исключает ненужные поля из типа

 Шаг 10:

Миссия 10

Кто-то хочет комбинировать утилиты, чтобы создать объект с обязательными полями, но с ограничением изменения отдельных свойств. Какой вариант подойдет?

Подсказка: Можно использовать комбинацию Required и Readonly.

Ответ:

- **Required>**
- Partial
- Pick
- Omit

 Шаг 11:

Миссия 11

Разработчик хочет создать новый тип, который включает только email и username, но все поля должны быть обязательными. Как это сделать?

Подсказка: Сначала используем Pick, затем Required.

Ответ:

- **Required>**
- Partial>
- Readonly>
- Omit

 Шаг 12:

Миссия 12

Расположи шаги комбинирования утилит для создания безопасного и обязательного объекта:

Подсказка: Сначала выбираем нужные поля, потом делаем их обязательными и защищаем от изменений.

Ответ:

- Выбрать нужные поля с Pick
- Сделать поля обязательными с Required
- Запретить изменения с Readonly
- Создать объект и использовать в коде

Урок 7 - Квест Написать типизированный конвертер JSON → UI

Шаг 01:

Миссия 1

Вы создаёте первый типизированный UI-конвертер. Какой тип лучше всего использовать для представления необязательных полей JSON?

Подсказка: Подумайте, как TypeScript может отличать отсутствие поля от его значения.

Ответ:

- **nullable (field?: string)**
- number
- boolean
- any

Шаг 02:

Миссия 2

JSON содержит поле "type", которое может быть 'text' или 'number'. Какой тип TypeScript лучше использовать для безопасной типизации?

Подсказка: Подумайте о union type для ограниченного набора строк.

Ответ:

- **'text' | 'number'**
- string
- any
- number

Шаг 03:

Миссия 3

Для отображения JSON-поля с опциональными значениями в UI нужно проверить значение перед рендером. Какой оператор TypeScript поможет безопасно?

Подсказка: Рассмотрите оператор, который предотвращает ошибку при null или undefined.

Ответ:

- **Optional chaining (?.)**
- Non-null assertion (!)
- Type assertion as string
- Nullish coalescing (??)

Шаг 04:

Миссия 4

Сопоставьте поля JSON с подходящими TypeScript типами:

Подсказка: Подумайте о правильной типизации каждого поля для UI.

Ответ:

- **fieldName: string** — Отображается как текстовое поле в UI
- **value: number | null** — Числовое поле, которое может быть пустым
- **isActive: boolean** — Переключатель в UI
- **type: 'text' | 'number'** — Тип элемента, определяет компонент для рендеринга

🔮 Шаг 05:

Миссия 5

Сопоставьте конвертеры JSON с их назначением:

Подсказка: Один конвертер для текстовых полей, другой для числовых и булевых.

Ответ:

- **TextConverter** — Преобразует строки JSON в UI-текстовое поле
- **NumberConverter** — Преобразует числовые значения JSON в input type number
- **BooleanConverter** — Отображает checkbox или toggle
- **GenericConverter** — Обработывает поля с union типом

🔮 Шаг 06:

Миссия 6

Какой подход поможет безопасно отобразить массив JSON-объектов в UI?

Подсказка: Используйте цикл с типизацией элементов.

Ответ:

- **array.map(item =>)**
- **array.forEach(item =>)**
- **JSON.stringify(array)**
- **for (let i in array) render(i)**

🔮 Шаг 07:

Миссия 7

Для обработки поля с union типом 'text' | 'number' безопаснее всего использовать:

Подсказка: Нужно различать возможные типы на этапе рендеринга.

Ответ:

- **Type guard (typeof field === 'string')**
- **any**
- **Non-null assertion (!)**
- **as string**

🔮 Шаг 08:

Миссия 8

Сопоставьте ошибки при конвертации JSON с их причиной:

Подсказка: Некоторые ошибки происходят из-за неправильной типизации.

Ответ:

- **Cannot read property 'value' of undefined** — Попытка доступа к необязательному полю без проверки
- **Type 'string' is not assignable to type 'number'** — Неправильная типизация поля union

- **JSX element type 'undefined' is not a constructor** — Компонент не импортирован или неправильно определён
- **Object is possibly 'null'** — Не использован optional chaining или проверка на null

🔧 Шаг 09:

Миссия 9

Расположите шаги безопасной конвертации JSON в UI:

Подсказка: Сначала проверяем данные, потом типизируем, затем рендерим.

Ответ:

- Получить JSON
- Проверить на null/undefined
- Типизировать с TypeScript
- Рендерить в UI

🔧 Шаг 10:

Миссия 10

Расположите этапы работы с массивом union-полей JSON:

Подсказка: Сначала проверяем элементы, потом конвертируем, потом рендерим.

Ответ:

- Итерировать массив
- Type guard для каждого элемента
- Вызывать конвертер для элемента
- Рендерить компонент

🔧 Шаг 11:

Миссия 11

Сопоставьте UI-элементы с типами JSON полей:

Подсказка: Текстовое поле для string, checkbox для boolean и т.д.

Ответ:

- **string** — Input type text
- **number** — Input type number
- **boolean** — Checkbox
- **'text' | 'number'** — Выбор компонента в зависимости от type

🔧 Шаг 12:

Миссия 12

Как безопасно добавить новое поле в типизированный конвертер без ошибок TypeScript?

Подсказка: Используйте union и опциональные поля.

Ответ:

- **Добавить поле с типом string | undefined**
- Добавить поле как any
- Игнорировать типы
- Удалить старые поля

Шаг 13:

Миссия 13

Какой способ предотвратит ошибку при рендеринге null поля JSON в UI?

Подсказка: Optional chaining и проверка на null помогут избежать ошибки.

Ответ:

- **field?.value ?? ''**
- field.value
- field!
- as string

Шаг 14:

Миссия 14

Сопоставьте ошибки рендеринга JSON с их устранением:

Подсказка: Обратите внимание на типы и наличие данных.

Ответ:

- **Cannot read property 'name' of undefined** — Использовать optional chaining или проверку на null
- **Type 'string | number' is not assignable to type 'string'** — Применить type guard или union тип
- **JSX element type is not assignable** — Проверить импорт компонента

Модуль 3 - Функции и сложные типы

Урок 1 - Типы функций и сигнатуры

Шаг 01:

Миссия 1

Какой синтаксис TypeScript лучше всего использовать для объявления простой функции, принимающей число и возвращающей строку?

Подсказка: Подумайте о явной сигнатуре параметров и возвращаемого типа.

Ответ:

- **function fn(x: number): string**
- function fn(x): string
- function fn(x: number)
- fn(x: number) => string

Шаг 02:

Миссия 2

Что является корректной сигнатурой функции, которая принимает строку и необязательное число и возвращает boolean?

Подсказка: Не забудьте про optional параметры.

Ответ:

- **function check(text: string, n?: number): boolean**
- function check(text?: string, n: number): boolean
- function check(text: string, n: number): boolean

- `function check(text: string): boolean | number`

🔗 Шаг 03:

Миссия 3

Какой вариант описывает функцию обратного вызова (callback) для массива чисел?

Подсказка: Callback принимает элемент массива и возвращает `boolean`.

Ответ:

- **`(num: number) => boolean`**
- `(num: string) => boolean`
- `function(num: number)`
- `num => string`

🔗 Шаг 04:

Миссия 4

Сопоставьте типы функций с их описанием:

Подсказка: Подумайте, что именно возвращает функция и какие параметры принимает.

Ответ:

- **`() => void`** — Функция без параметров, ничего не возвращает
- **`(a: number) => string`** — Функция с числовым параметром, возвращает строку
- **`(x: string, y: string) => boolean`** — Функция с двумя строковыми параметрами, возвращает булево значение
- **`(...args: number[]) => number`** — Функция принимает любое количество чисел и возвращает число

🔗 Шаг 05:

Миссия 5

Сопоставьте виды функций с примерами их использования:

Подсказка: Некоторые функции используются как обработчики событий, а другие — для вычислений.

Ответ:

- **Callback функция** — Используется в `map`, `filter`, `forEach`
- **Чистая функция** — Возвращает результат без побочных эффектов
- **Асинхронная функция** — Возвращает `Promise`
- **Функция с перегрузкой** — Разные сигнатуры для одного имени функции

🔗 Шаг 06:

Миссия 6

Как правильно определить функцию, которая возвращает другой тип функции?

Подсказка: Функция высшего порядка возвращает функцию.

Ответ:

- **`function fn(): (x: number) => string`**
- `function fn(): string`
- `function fn(x: number): string`
- `() => number`

🔗 Шаг 07:

Миссия 7

Какая сигнатура описывает функцию с необязательным параметром и возвратом числа?

Подсказка: Не забудьте про optional параметр (?) в TypeScript).

Ответ:

- **(x?: number) => number**
- (x: number?) => number
- (x?: number) => void
- x => number

🔗 Шаг 08:

Миссия 8

Какой вариант описывает корректную типизацию функции, возвращающей callback?

Подсказка: Callback должен иметь явно заданную сигнатуру.

Ответ:

- **function createCallback(): (x: string) => void**
- function createCallback(): void
- () => string
- (x: string) => string

🔗 Шаг 09:

Миссия 9

Сопоставьте ошибки сигнатур с их причиной:

Подсказка: Ошибки часто связаны с несовпадением типов параметров и возвращаемых значений.

Ответ:

- **Parameter 'x' implicitly has an 'any' type** — Не указан тип параметра функции
- **Type 'string' is not assignable to type 'number'** — Неверный тип возвращаемого значения
- **Expected 2 arguments, but got 1** — Неверное количество параметров при вызове функции
- **Cannot assign to 'constVar' inside function** — Попытка изменить константу

🔗 Шаг 10:

Миссия 10

Какой вариант объявления функции позволит указать тип возвращаемого промиса?

Подсказка: Асинхронные функции всегда возвращают Promise с указанным типом.

Ответ:

- **async function fetchData(): Promise**
- function fetchData(): string
- async function fetchData(): string
- function fetchData(): Promise

🔗 Шаг 11:

Миссия 11

Для функции с rest-параметрами, принимающей числа и возвращающей их сумму, какая сигнатура корректна?

Подсказка: Используйте ...args и указание типа элементов массива.

Ответ:

- **function sum(...nums: number[]): number**
- function sum(nums: number[]): number
- function sum(...nums: any[]): any
- function sum(...nums: string[]): number

🔗 Шаг 12:

Миссия 12

Сопоставьте типы функций с реальными жизненными ситуациями:

Подсказка: Подумайте, где callback, а где чистая функция пригодится.

Ответ:

- **Callback функция** — Обработчик клика на кнопке
- **Чистая функция** — Вычисление площади квадрата
- **Асинхронная функция** — Запрос данных с API
- **Функция с перегрузкой** — Функция, которая складывает числа или объединяет строки

🔗 Шаг 13:

Миссия 13

Расположите шаги создания функции с типами и callback:

Подсказка: Сначала определяем сигнатуру, затем параметры, потом реализацию и вызов callback.

Ответ:

- Определить сигнатуру функции
- Добавить параметры и их типы
- Реализовать тело функции
- Вызвать callback с аргументами

🔗 Шаг 14:

Миссия 14

Расположите шаги работы с асинхронной функцией:

Подсказка: Сначала вызываем асупс функцию, потом обрабатываем результат, затем используем в UI.

Ответ:

- Вызвать асинхронную функцию
- Дождаться Promise с await или then
- Обработать результат
- Отобразить результат в UI

🔮 Шаг 15:

Миссия 15

Какая сигнатура функции позволяет комбинировать перегрузки, async и callback?

Подсказка: Можно комбинировать несколько сигнатур и возвращать Promise с callback внутри.

Ответ:

- **async function fn(a: number): Promise; async function fn(a: string): Promise; async function fn(a: any): Promise { return callback(a); }**
- function fn(a: number | string): string
- async function fn(): void
- function fn(a: any) => Promise

📖 Урок 2 - OptionalRest-параметры

🔮 Шаг 01:

Миссия 1

Как правильно объявить функцию с опциональным параметром age?

Подсказка: В TypeScript optional-параметры указываются знаком ?.

Ответ:

- **function greet(name: string, age?: number)**
- function greet(name: string, optional age: number)
- function greet(name: string, age)
- function greet(name: string, age!: number)

🔮 Шаг 02:

Миссия 2

Для функции с rest-параметрами, принимающей числа и возвращающей их сумму, какая сигнатура корректна?

Подсказка: Используйте ...args и укажите тип элементов массива.

Ответ:

- **function sum(...nums: number[]): number**
- function sum(nums: number[]): number
- function sum(...nums: string[]): number
- function sum(...nums: any[]): any

🔮 Шаг 03:

Миссия 3

Как вызвать функцию greet с опциональным параметром, если возраст неизвестен?

Подсказка: Optional-параметр можно просто не передавать.

Ответ:

- **greet('Alice')**
- greet('Alice', undefined)
- greet('Alice', null)
- greet('Alice', 0)

🔗 Шаг 04:

Миссия 4

Сопоставьте ключевые понятия Optional и Rest-параметров с примерами:

Подсказка: Подумайте, когда параметр может быть пропущен, а когда принимается произвольное число аргументов.

Ответ:

- **Optional-параметр** — `age?: number` в `function greet(name: string, age?: number)`
- **Rest-параметр** — `...nums: number[]` в `function sum(...nums: number[])`

🔗 Шаг 05:

Миссия 5

Можно ли использовать Optional-параметр перед обязательным параметром?

Подсказка: В TypeScript порядок параметров важен для корректного вызова функции.

Ответ:

- Да, можно
- **Нет, обязательные параметры должны идти первыми**

🔗 Шаг 06:

Миссия 6

Сопоставьте возможные ошибки при использовании Optional/Rest-параметров с их причиной:

Подсказка: Ошибки часто связаны с порядком параметров или неправильной типизацией.

Ответ:

- **Ошибка: ожидается 2 аргумента, получен 1** — Optional-параметр поставлен перед обязательным
- **Ошибка: Type 'string' не назначаем Type 'number'** — Неверный тип аргумента в rest-параметрах

🔗 Шаг 07:

Миссия 7

Какой способ объединяет Optional и Rest-параметры в одной функции?

Подсказка: Optional-параметр идет первым, rest-параметры — последними.

Ответ:

- **`function buildMessage(prefix?: string, ...words: string[])`**
- `function buildMessage(...words: string[], prefix?: string)`
- `function buildMessage(prefix: string, ...words?: string[])`
- `function buildMessage(...words?: string[], prefix: string)`

🔗 Шаг 08:

Миссия 8

Сопоставьте функции с правильными описаниями использования Optional/Rest-параметров:

Подсказка: Подумайте, когда нужно принимать произвольное количество аргументов, а когда параметр может быть пропущен.

Ответ:

- **`function greet(name: string, age?: number)`** — Приветствие с опциональным возрастом

- **function sum(...nums: number[])** — Сумма произвольного количества чисел

🔗 Шаг 09:

Миссия 9

Можно ли передавать `undefined` в Optional-параметр?

Подсказка: Optional-параметр по сути может быть пропущен или явно указан как `undefined`.

Ответ:

- **Да, можно передавать `undefined`**
- Нет, нельзя

🔗 Шаг 10:

Миссия 10

Как правильно вызвать функцию `sum(...nums: number[])` с массивом чисел `arr`?

Подсказка: Используйте `spread` оператор для передачи массива как аргументов.

Ответ:

- **`sum(...arr)`**
- `sum(arr)`
- `sum(arr...)`
- `sum(...arr[])`

🔗 Шаг 11:

Миссия 11

Сопоставьте функции с ожидаемым поведением при использовании Optional/Rest-параметров:

Подсказка: Подумайте, какие значения могут быть пропущены, а какие переданы как массив аргументов.

Ответ:

- **`greet('Alice')`** — Optional-параметр `age` отсутствует
- **`greet('Alice', 25)`** — Optional-параметр `age` передан
- **`sum(1,2,3)`** — Rest-параметры `nums` включают 1,2,3
- **`sum(...arr)`** — Rest-параметры `nums` равны элементам массива `arr`

🔗 Шаг 12:

Миссия 12

Можно ли комбинировать несколько Optional-параметров в одной функции?

Подсказка: Optional-параметры могут идти подряд, главное — после обязательных.

Ответ:

- **Да, можно несколько optional-параметров подряд**
- Нет, только один optional-параметр

📖 Урок 3 - Overloads (перегрузки функций)

🔗 Шаг 01:

Миссия 1

Что такое перегрузка функции (function overload) в TypeScript?

Подсказка: Подумайте о возможности использовать одну функцию с разными типами аргументов.

Ответ:

- **Определение нескольких сигнатур функции с разными типами аргументов**
- Создание функции с неопределенным типом `any`
- Использование одной функции без аргументов
- Создание нескольких функций с одинаковым именем

Шаг 02:

Миссия 2

Как правильно объявить перегруженную функцию для `greet(name: string)` и `greet(name: string, age: number)`?

Подсказка: Сначала идут сигнатуры, потом реализация функции.

Ответ:

- **`function greet(name: string): void; function greet(name: string, age: number): void; function greet(name: string, age?: number) { ... }`**
- `function greet(name: string, age?: number): void;`
- `function greet(name: string | number) { ... }`
- `function greet(name: string, age: number = 0) { ... }`

Шаг 03:

Миссия 3

Можно ли перегрузить функцию, изменяя только возвращаемый тип?

Подсказка: TypeScript смотрит на аргументы, а не только на тип возвращаемого значения.

Ответ:

- Да, можно
- **Нет, аргументы должны отличаться**

Шаг 04:

Миссия 4

Сопоставьте типичные сценарии использования перегрузки функции с описанием:

Подсказка: Подумайте, когда одна и та же функция может принимать разное количество аргументов.

Ответ:

- **Функция `greet(name: string)`** — Приветствие без возраста
- **Функция `greet(name: string, age: number)`** — Приветствие с возрастом
- **Функция `format(value: string)`** — Форматирование строки
- **Функция `format(value: number, decimals: number)`** — Форматирование числа с количеством знаков

Шаг 05:

Миссия 5

Что произойдет, если вызвать перегруженную функцию с аргументами, не соответствующими ни одной сигнатуре?

Подсказка: TypeScript проверяет вызовы на соответствие сигнатурам.

Ответ:

- **Ошибка компиляции**
- Функция выполнится, вернув undefined

🔗 Шаг 06:

Миссия 6

Сопоставьте элементы перегрузки функции с их назначением:

Подсказка: Каждая сигнатура описывает разные варианты вызова.

Ответ:

- **function calculate(a: number, b: number): number;** — Суммирование чисел
- **function calculate(a: string, b: string): string;** — Конкатенация строк
- **function calculate(a: any, b: any) { ... }** — Реализация перегруженной функции

🔗 Шаг 07:

Миссия 7

Где должна находиться реализация перегруженной функции?

Подсказка: Она должна следовать после всех сигнатур.

Ответ:

- **После всех сигнатур**
- До сигнатур

🔗 Шаг 08:

Миссия 8

Что будет, если вызвать перегруженную функцию с правильными аргументами, но не указав реализацию?

Подсказка: Подумайте, нужна ли функция без тела для компиляции TypeScript.

Ответ:

- **Ошибка компиляции**
- Функция выполнится с undefined

🔗 Шаг 09:

Миссия 9

Сопоставьте сигнатуры функции с их примером использования:

Подсказка: Подумайте, какие аргументы соответствуют конкретной сигнатуре.

Ответ:

- **function print(value: string): void;** — Вывод строки в консоль
- **function print(value: number): void;** — Вывод числа в консоль
- **function print(value: boolean): void;** — Вывод логического значения

🔗 Шаг 10:

Миссия 10

Можно ли в перегрузке функции использовать необязательные параметры (optional)?

Подсказка: Рассмотрите синтаксис с `?` и его сочетание с перегрузками.

Ответ:

- **Да, можно использовать optional-параметры в реализации**
- Нет, только обязательные параметры

 **Шаг 11:**

Миссия 11

Сопоставьте преимущества перегрузки функций с их описанием:

Подсказка: Подумайте, зачем программисты используют перегрузки.

Ответ:

- **Повышение читаемости кода** — Разные вызовы одной функции легче понять
- **Соккрытие логики реализации** — Пользователь видит только сигнатуры
- **Снижение ошибок типов** — TypeScript проверяет аргументы вызова

 **Шаг 12:**

Миссия 12

Расположите шаги создания перегруженной функции в правильном порядке:

1. Написать все сигнатуры функции
2. Реализовать функцию с телом
3. Проверить вызовы функции на соответствие сигнатурам
4. Использовать функцию в коде

Подсказка: Сначала TypeScript должен видеть сигнатуры, затем реализацию, потом вызовы.

Ответ:

- Написать все сигнатуры функции
- Реализовать функцию с телом
- Проверить вызовы функции на соответствие сигнатурам
- Использовать функцию в коде

Урок 4 - Callback-и и колбэк с типами

 **Шаг 01:**

Миссия 1

Аня пишет функцию, которая принимает колбэк и число, а затем применяет колбэк к числу. Что правильнее всего указывать как тип колбэка в TypeScript?

Подсказка: Подумайте, что принимает и что возвращает функция.

Ответ:

- **(x: number) => number**
- string
- boolean

 **Шаг 02:**

Миссия 2

Игорь хочет написать функцию, которая получает массив чисел и колбэк, который проверяет каждое число. Как правильно указать тип колбэка?

Подсказка: Колбэк должен возвращать логическое значение.

Ответ:

- **(n: number) => boolean**
- (n: number) => string
- (n: string) => number

Шаг 03:

Миссия 3

Дмитрий создает функцию `logMessage`, которая принимает колбэк для обработки сообщения перед выводом. Какой тип правильнее указать?

Подсказка: Сообщение — это строка, колбэк возвращает `void`.

Ответ:

- **(msg: string) => void**
- (msg: number) => string
- () => string

Шаг 04:

Миссия 4

Саша разрабатывает функцию, которая фильтрует список студентов по условию, заданному колбэком. Как правильно описать тип колбэка?

Подсказка: Каждый студент — объект с полями, проверка возвращает `true/false`.

Ответ:

- **(student: {name: string, age: number}) => boolean**
- (student: string) => string
- (student: number) => void

Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы колбэков с их примерами использования.

Понятия:

Ответ:

- Колбэк для вычислений
- Колбэк для проверки условия
- Колбэк для обработки события

Определения:

1. Функция, которая возвращает новое значение после вычисления
2. Функция, возвращающая `true/false` для фильтрации данных
3. Функция, вызываемая при наступлении события

Подсказка: Вспомните типичные сценарии: вычисление, фильтрация, событие.

- **Колбэк для вычислений** — Функция, которая возвращает новое значение после вычисления
- **Колбэк для проверки условия** — Функция, возвращающая `true/false` для фильтрации данных
- **Колбэк для обработки события** — Функция, вызываемая при наступлении события

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте колбэки с их возвращаемыми типами.

Понятия:

Ответ:

- Сложный математический колбэк
- Проверка наличия элемента в массиве
- Колбэк для логирования сообщений

Определения:

1. number
2. boolean
3. void

Подсказка: Подумайте, что возвращает каждая функция по сути.

- **Сложный математический колбэк** — number
- **Проверка наличия элемента в массиве** — boolean
- **Колбэк для логирования сообщений** — void

🔗 Шаг 07:

Миссия 7

Формулировка: Сопоставьте разработчиков с типами колбэков, которые они чаще всего используют.

Понятия:

Ответ:

- **Аня** — фильтрует массивы чисел
- **Игорь** — выводит сообщения в консоль
- **Дмитрий** — вычисляет новые значения для UI

Определения:

1. (n: number) => boolean
2. (msg: string) => void
3. (x: number) => number

Подсказка: Вспомните их ежедневные задачи и что возвращает колбэк.

- **Аня** — фильтрует массивы чисел — (n: number) => boolean
- **Игорь** — выводит сообщения в консоль — (msg: string) => void
- **Дмитрий** — вычисляет новые значения для UI — (x: number) => number

🔗 Шаг 08:

Миссия 8

Формулировка: Сопоставьте функции высшего порядка с типами колбэков, которые они принимают.

Понятия:

Ответ:

- map

- filter
- forEach

Определения:

1. Колбэк возвращает новое значение
2. Колбэк возвращает true/false
3. Колбэк выполняет действие, не возвращая значение

Подсказка: Подумайте о результатах работы этих методов массива.

- **map** — Колбэк возвращает новое значение
- **filter** — Колбэк возвращает true/false
- **forEach** — Колбэк выполняет действие, не возвращая значение

Шаг 09:

Миссия 9

Формулировка: Сопоставьте тип колбэка с ожидаемым результатом для функции reduce.

Понятия:

Ответ:

- Колбэк суммирования чисел
- Колбэк построения строки
- Колбэк подсчета элементов

Определения:

1. number
2. string
3. Record

Подсказка: Reduce аккумулирует результат — подумайте, какой тип мы хотим получить.

- **Колбэк суммирования чисел** — number
- **Колбэк построения строки** — string
- **Колбэк подсчета элементов** — Record

Шаг 10:

Миссия 10

Вопрос: Упорядочьте шаги создания функции с колбэком правильно:

1. Определяем, что делает функция
2. Определяем типы параметров и колбэка
3. Реализуем тело функции
4. Тестируем функцию с разными колбэками

Подсказка: Сначала продумываем функциональность, затем типы, потом код, и только потом проверяем результат.

Ответ:

- Определяем, что делает функция
- Определяем типы параметров и колбэка
- Реализуем тело функции
- Тестируем функцию с разными колбэками

Шаг 11:

Миссия 11

Вопрос: Упорядочьте этапы использования колбэка для фильтрации массива студентов:

1. Создание массива студентов
2. Определение функции фильтрации с колбэком
3. Применение фильтра к массиву
4. Вывод результатов в консоль

Подсказка: Сначала формируем данные, потом фильтруем, и только потом показываем результат.

Ответ:

- Создание массива студентов
- Определение функции фильтрации с колбэком
- Применение фильтра к массиву
- Вывод результатов в консоль

Шаг 12:

Миссия 12

Вопрос: Расположите шаги работы метода `map` с колбэком:

1. Определяем исходный массив
2. Определяем колбэк для преобразования элементов
3. Применяем `map` с колбэком
4. Получаем новый массив с преобразованными значениями

Подсказка: `Map` создаёт новый массив после применения функции к каждому элементу.

Ответ:

- Определяем исходный массив
- Определяем колбэк для преобразования элементов
- Применяем `map` с колбэком
- Получаем новый массив с преобразованными значениями

Шаг 13:

Миссия 13

Вопрос: Упорядочьте процесс использования `reduce` с колбэком для подсчета элементов массива:

1. Создаем массив с данными
2. Определяем начальное значение аккумулятора
3. Определяем колбэк для `reduce`
4. Применяем `reduce` и получаем результат

Подсказка: `Reduce` требует аккумулялирующего значения и функции обработки.

Ответ:

- Создаем массив с данными
- Определяем начальное значение аккумулятора
- Определяем колбэк для `reduce`
- Применяем `reduce` и получаем результат

Урок 5 - Типизация замыканий

Шаг 01:

Миссия 1

Аня создает функцию-счетчик, которая возвращает другую функцию для увеличения числа. Какой тип правильно указать для возвращаемой функции?

Подсказка: Замыкание должно возвращать число при каждом вызове.

Ответ:

- **() => number**
- () => string
- (x: number) => void

Шаг 02:

Миссия 2

Игорь пишет функцию makeMultiplier, которая принимает число и возвращает функцию для умножения на это число. Какой тип указать для внутренней функции?

Подсказка: Внутренняя функция должна принимать число и возвращать число.

Ответ:

- **(n: number) => number**
- () => number
- (n: string) => string

Шаг 03:

Миссия 3

Дмитрий создает замыкание, чтобы хранить приватное значение внутри функции. Какой тип нужно использовать для переменной внутри замыкания?

Подсказка: Приватная переменная должна сохранять число, строку или объект?

Ответ:

- **number**
- boolean
- undefined

Шаг 04:

Миссия 4

Саша хочет сделать функцию-счетчик с замыканием, которая принимает стартовое число. Какой тип должна принимать входная переменная?

Подсказка: Стартовое значение должно быть числом для математических операций.

Ответ:

- **number**
- string
- boolean

Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы замыканий с их примерами использования.

Понятия:

Ответ:

- Счетчик
- Функция умножения
- Приватное хранилище

Определения:

1. Функция, которая увеличивает число при каждом вызове
2. Функция, которая возвращает результат умножения на фиксированное число
3. Переменная внутри функции, недоступная снаружи

Подсказка: Подумайте о том, что сохраняет замыкание и что возвращает функция.

- **Счетчик** — Функция, которая увеличивает число при каждом вызове
- **Функция умножения** — Функция, которая возвращает результат умножения на фиксированное число
- **Приватное хранилище** — Переменная внутри функции, недоступная снаружи

Шаг 06:

Миссия 6

Формулировка: Сопоставьте разработчиков с типами замыканий, которые они создают.

Понятия:

Ответ:

- Аня
- Игорь
- Дмитрий

Определения:

1. Счетчик чисел
2. Функция умножения
3. Приватная переменная для хранения состояния

Подсказка: Вспомните, кто что делает в своей повседневной работе.

- **Аня** — Счетчик чисел
- **Игорь** — Функция умножения
- **Дмитрий** — Приватная переменная для хранения состояния

Шаг 07:

Миссия 7

Формулировка: Сопоставьте функции с типами возвращаемых замыканий.

Понятия:

Ответ:

- makeCounter()
- makeMultiplier(2)
- createStorage()

Определения:

1. `() => number`
2. `(n: number) => number`
3. `{get: () => number, set: (v: number) => void}`

Подсказка: Подумайте о том, что возвращает каждая функция.

- **makeCounter()** — `() => number`
- **makeMultiplier(2)** — `(n: number) => number`
- **createStorage()** — `{get: () => number, set: (v: number) => void}`

Шаг 08:

Миссия 8

Формулировка: Сопоставьте методы работы с замыканием с их функциями.

Понятия:

Ответ:

- Increment
- Multiply
- Set/Get

Определения:

1. Увеличивает текущее значение
2. Возвращает произведение числа на фиксированное значение
3. Сохраняет и возвращает приватное значение

Подсказка: Подумайте, что делает каждая функция с внутренним состоянием.

- **Increment** — Увеличивает текущее значение
- **Multiply** — Возвращает произведение числа на фиксированное значение
- **Set/Get** — Сохраняет и возвращает приватное значение

Шаг 09:

Миссия 9

Формулировка: Сопоставьте типы замыканий с их предназначением.

Понятия:

Ответ:

- Приватная переменная
- Функция с аргументом
- Функция без аргумента

Определения:

1. Сохраняет состояние, недоступное снаружи
2. Используется для вычислений с внешними параметрами
3. Используется для накопления значений внутри замыкания

Подсказка: Замыкание позволяет хранить данные и использовать их в функции.

- **Приватная переменная** — Сохраняет состояние, недоступное снаружи
- **Функция с аргументом** — Используется для вычислений с внешними параметрами
- **Функция без аргумента** — Используется для накопления значений внутри замыкания

Шаг 10:

Миссия 10

Вопрос: Расположите шаги создания функции-счетчика с замыканием:

1. Определяем стартовое значение
2. Создаем внутреннюю функцию для увеличения значения
3. Возвращаем внутреннюю функцию
4. Вызываем замыкание несколько раз, чтобы проверить результат

Подсказка: Сначала определяем начальные данные, потом создаем логику, возвращаем функцию и проверяем её работу.

Ответ:

- Определяем стартовое значение
- Создаем внутреннюю функцию для увеличения значения
- Возвращаем внутреннюю функцию
- Вызываем замыкание несколько раз, чтобы проверить результат

Шаг 11:

Миссия 11

Вопрос: Упорядочьте этапы работы функции makeMultiplier:

1. Функция принимает число-множитель
2. Возвращает замыкание, которое умножает входное число на множитель
3. Вызываем замыкание с аргументом
4. Получаем результат умножения

Подсказка: Сначала создаем множитель, потом замыкание, потом используем его.

Ответ:

- Функция принимает число-множитель
- Возвращает замыкание, которое умножает входное число на множитель
- Вызываем замыкание с аргументом
- Получаем результат умножения

Шаг 12:

Миссия 12

Вопрос: Упорядочьте процесс создания приватного хранилища с замыканием:

1. Создаем объект состояния внутри функции
2. Определяем методы get и set
3. Возвращаем объект с методами
4. Используем методы для чтения и изменения значения

Подсказка: Замыкание скрывает объект состояния и позволяет работать только через методы.

Ответ:

- Создаем объект состояния внутри функции
- Определяем методы get и set
- Возвращаем объект с методами
- Используем методы для чтения и изменения значения

Шаг 13:

Миссия 13

Вопрос: Расположите этапы использования счетчика-сумматора с замыканием:

1. Создаем функцию makeSumCounter
2. Определяем начальное значение суммы
3. Возвращаем функцию для добавления к сумме
4. Вызываем замыкание несколько раз и получаем текущую сумму

Подсказка: Сначала определяем данные, затем логику, возвращаем функцию, используем её.

Ответ:

- Создаем функцию makeSumCounter
- Определяем начальное значение суммы
- Возвращаем функцию для добавления к сумме
- Вызываем замыкание несколько раз и получаем текущую сумму

Урок 6 - Функции как параметры, типы возвращаемых значений

Шаг 01:

Миссия 1

Аня, разработчица, создает функцию для фильтрации списка пользователей. Какой подход правильный для передачи функции как параметра?

Подсказка: В TypeScript функции можно передавать как аргументы, чтобы менять поведение без дублирования кода.

Ответ:

- **Передать функцию в качестве аргумента другой функции**
- Вставить тело функции напрямую внутрь другой функции
- Использовать глобальную переменную вместо передачи функции
- Вызывать функцию через eval

Шаг 02:

Миссия 2

Игорь хочет определить тип возвращаемого значения функции. Как правильно указать тип в TypeScript?

Подсказка: Тип возвращаемого значения помогает IDE и компилятору проверять корректность работы функции.

Ответ:

- **Указать тип после круглых скобок: `function fn(): string {}`**
- Указать тип перед именем функции: `string function fn() {}`
- Не указывать тип, TypeScript сам угадает

Шаг 03:

Миссия 3

Лена передает функцию сравнения в метод `sort()`. Какой принцип TypeScript поможет ей безопасно типизировать эту функцию?

Подсказка: Функция сравнения должна принимать два параметра одного типа и возвращать число.

Ответ:

- **Указать тип параметров и возвращаемого значения: (a: number, b: number) => number**
- Не указывать типы, использовать any
- Типизировать как (a: string, b: string) => string

Шаг 04:

Миссия 4

 Сопоставьте функции с их типами возвращаемых значений.

Функции:

Ответ:

- getName
- calculateSum
- isActive
- fetchData

Типы:

1. boolean
2. Promise
3. string
4. number

Подсказка:

Подумайте, какие данные возвращает каждая функция: текст, число, логическое значение или асинхронный результат.

- **getName** — string
- **calculateSum** — number
- **isActive** — boolean
- **fetchData** — Promise

Шаг 05:

Миссия 5

 Сопоставьте параметры функций с их типами.

Параметры:

Ответ:

- age
- name
- isEnabled
- data

Типы:

1. string
2. number
3. boolean
4. any[]

Подсказка:

Подумайте, какие данные обычно соответствуют имени, возрасту, логическому флагу и массиву.

- **age** — number
- **name** — string
- **isEnabled** — boolean
- **data** — any[]

Шаг 06:

Миссия 6

 Сопоставьте функции с описанием их задач.

Функции:

Ответ:

- filterActiveUsers
- sortByDate
- calculateTotal
- logMessage

Описание:

1. Фильтрует массив, оставляя только активных пользователей
2. Сортирует элементы по дате
3. Считает сумму числовых значений
4. Выводит сообщение в консоль

Подсказка:

Обратите внимание на глаголы: filter, sort, calculate, log — они подсказывают назначение функций.

- **filterActiveUsers** — Фильтрует массив, оставляя только активных пользователей
- **sortByDate** — Сортирует элементы по дате
- **calculateTotal** — Считает сумму числовых значений
- **logMessage** — Выводит сообщение в консоль

Шаг 07:

Миссия 7

Саша создает функцию, которая возвращает другую функцию. Как называется такой подход?

Подсказка: Это позволяет создавать динамическое поведение и замыкания.

Ответ:

- **Функция высшего порядка**
- Асинхронная функция
- Генератор

Шаг 08:

Миссия 8

Какой синтаксис в TypeScript безопасен для передачи функции обратного вызова, которая возвращает boolean?

Подсказка: Укажите типы для параметров и возвращаемого значения, чтобы избежать ошибок компиляции.

Ответ:

- **(item: string) => boolean**
- (item) => Boolean
- item: any => boolean

Шаг 09:

Миссия 9

 Сопоставьте типы функций с примерами использования.

Типы:

Ответ:

- Callback
- Async function
- Higher-order function
- Arrow function

Примеры:

1. Функция, переданная как аргумент и вызываемая позже
2. Функция, возвращающая другую функцию
3. Функция с `async/await` для работы с промисами
4. Краткий синтаксис функции с `=>`

Подсказка:

Обратите внимание на характер функции: возвращает ли она другую функцию, работает с промисами или просто синтаксис стрелки.

- **Callback** — Функция, переданная как аргумент и вызываемая позже
- **Higher-order function** — Функция, возвращающая другую функцию
- **Async function** — Функция с `async/await` для работы с промисами
- **Arrow function** — Краткий синтаксис функции с `=>`

Шаг 10:

Миссия 10

 Сопоставьте ошибки типизации с их описанием.

Ошибки:

Ответ:

- Тип `any`
- Неправильный тип параметра
- Отсутствие возвращаемого типа
- Несовместимые типы в условии

Описание:

1. Использование `any` может скрыть реальные ошибки
2. Функция получает аргумент не того типа
3. Функция не указала явно возвращаемый тип
4. Сравнение несовместимых типов, например `string` и `number`

Подсказка:

Подумайте, какие ошибки могут проявиться только при проверке типов TypeScript.

- **Тип any** — Использование any может скрыть реальные ошибки
- **Неправильный тип параметра** — Функция получает аргумент не того типа
- **Отсутствие возвращаемого типа** — Функция не указала явно возвращаемый тип
- **Несовместимые типы в условии** — Сравнение несовместимых типов, например string и number

🔮 Шаг 11:

Миссия 11

Расположите шаги создания функции высшего порядка в правильном порядке:

1. Определение функции, которая принимает другую функцию
2. Передача функции как аргумента
3. Вызов возвращаемой функции внутри основной
4. Тестирование результата работы функции

🔮 Подсказка:

Сначала создаем функцию, потом передаем аргументы, вызываем и проверяем.

Ответ:

- Определение функции, которая принимает другую функцию
- Передача функции как аргумента
- Вызов возвращаемой функции внутри основной
- Тестирование результата работы функции

🔮 Шаг 12:

Миссия 12

Расположите этапы работы с callback-функциями:

1. Определение функции, которая вызывает callback
2. Передача callback-функции как аргумента
3. Вызов callback внутри функции
4. Обработка результата callback

🔮 Подсказка:

Подумайте: сначала функция должна быть определена, затем мы передаем callback и вызываем его.

Ответ:

- Определение функции, которая вызывает callback
- Передача callback-функции как аргумента
- Вызов callback внутри функции
- Обработка результата callback

🔮 Шаг 13:

Миссия 13

Расположите этапы безопасной типизации функций в TypeScript:

1. Определение параметров с типами
2. Определение возвращаемого типа
3. Реализация функции

4. Тестирование с разными входными данными

Подсказка:

Сначала задаем типы, потом пишем логику и проверяем корректность работы.

Ответ:

- Определение параметров с типами
- Определение возвращаемого типа
- Реализация функции
- Тестирование с разными входными данными

Урок 7 - Квест Типизированный обработчик событий и цепочка колбэков

Шаг 01:

Миссия 1

Аня создает обработчик клика для кнопки. Какой подход в TypeScript безопасен для типизации события?

Подсказка: События имеют встроенные типы, которые помогают избежать ошибок.

Ответ:

- **(event: MouseEvent) => void**
- (event) => void
- event: any => void

Шаг 02:

Миссия 2

Игорь пишет цепочку колбэков: первый колбэк должен вернуть число, второй строку. Как правильно типизировать функции?

Подсказка: Тип возвращаемого значения каждой функции помогает безопасно передавать результат следующей.

Ответ:

- **Указать возвращаемый тип каждой функции явно**
- Использовать any для всех функций
- Не указывать тип, TypeScript сам определит

Шаг 03:

Миссия 3

Лена хочет написать универсальный обработчик событий для нескольких кнопок. Какой подход поможет ей избежать дублирования кода?

Подсказка: Используйте функцию высшего порядка, чтобы создавать обработчики динамически.

Ответ:

- **Создать функцию, которая возвращает обработчик события**
- Дублировать код для каждой кнопки
- Присвоить всем кнопкам один и тот же обработчик без параметров

Шаг 04:

Миссия 4

 Сопоставьте параметры колбэков с их назначением.

Параметры:

Ответ:

- event
- value
- callback
- index

Назначение:

1. Ссылка на событие, которое произошло
2. Текущее значение поля ввода
3. Функция, которую нужно вызвать после завершения
4. Позиция элемента в массиве

 **Подсказка:**

Подумайте, что передают функции-обработчики для работы с DOM и массивами.

- **event** — Ссылка на событие, которое произошло
- **value** — Текущее значение поля ввода
- **callback** — Функция, которую нужно вызвать после завершения
- **index** — Позиция элемента в массиве

 **Шаг 05:**

Миссия 5

 Сопоставьте типы возвращаемых значений колбэков с примерами использования.

Типы:

Ответ:

- void
- number
- boolean
- Promise

Примеры:

1. Функция для логирования клика
2. Функция вычисления суммы
3. Функция проверки условий
4. Асинхронная функция получения данных

 **Подсказка:**

Обратите внимание: что функция возвращает — результат вычислений, true/false или ничего?

- **void** — Функция для логирования клика
- **number** — Функция вычисления суммы
- **boolean** — Функция проверки условий
- **Promise** — Асинхронная функция получения данных

 **Шаг 06:**

Миссия 6

Саша создает функцию, которая принимает колбэк и возвращает новый колбэк. Как называется такой подход?

Подсказка: Этот паттерн помогает создавать цепочки вызовов и композицию функций.

Ответ:

- **Функция высшего порядка**
- Асинхронная функция
- Генератор

 **Шаг 07:**

Миссия 7

Какой синтаксис TypeScript безопасен для передачи колбэка, который возвращает boolean?

Подсказка: Укажите типы параметров и возвращаемого значения.

Ответ:

- **(item: string) => boolean**
- (item) => Boolean
- item: any => boolean

 **Шаг 08:**

Миссия 8

 Сопоставьте типы колбэков с их примерами:

Типы:

Ответ:

- Callback
- Async callback
- Higher-order callback
- Arrow callback

Примеры:

1. Функция, переданная как аргумент и вызываемая позже
2. Асинхронная функция с await
3. Функция, возвращающая другую функцию
4. Краткий синтаксис =>

 **Подсказка:**

Обратите внимание на характер функции: возвращает ли она другую функцию, работает с промисами или просто стрелка.

- **Callback** — Функция, переданная как аргумент и вызываемая позже
- **Async callback** — Асинхронная функция с await
- **Higher-order callback** — Функция, возвращающая другую функцию
- **Arrow callback** — Краткий синтаксис =>

 **Шаг 09:**

Миссия 9

 Сопоставьте ошибки типизации колбэков с описанием:

Ошибки:

Ответ:

- Тип any

- Неправильный тип параметра
- Отсутствие возвращаемого типа
- Несовместимые типы в цепочке

Описание:

1. Использование `any` скрывает реальные ошибки
2. Функция получает аргумент не того типа
3. Не указан явно тип возвращаемого значения
4. Передаваемые значения разных типов не совместимы

🔗 Подсказка:

Подумайте, какие ошибки могут проявиться только при проверке типов TypeScript.

- **Тип `any`** — Использование `any` скрывает реальные ошибки
- **Неправильный тип параметра** — Функция получает аргумент не того типа
- **Отсутствие возвращаемого типа** — Не указан явно тип возвращаемого значения
- **Несовместимые типы в цепочке** — Передаваемые значения разных типов не совместимы

🔗 Шаг 10:

Миссия 10

Расположите шаги создания цепочки колбэков:

1. Определить функцию, принимающую колбэк
2. Создать колбэк с правильной типизацией
3. Вызвать функцию с колбэком
4. Обработать результат

🔗 Подсказка:

Сначала определяем функцию, потом колбэк и вызываем цепочку.

Ответ:

- Определить функцию, принимающую колбэк
- Создать колбэк с правильной типизацией
- Вызвать функцию с колбэком
- Обработать результат

🔗 Шаг 11:

Миссия 11

Расположите этапы безопасной типизации колбэков в TypeScript:

1. Указать типы параметров
2. Указать тип возвращаемого значения
3. Реализовать логику функции
4. Протестировать с разными входными значениями

🔗 Подсказка:

Сначала типы, потом логика и проверка.

Ответ:

- Указать типы параметров
- Указать тип возвращаемого значения

- Реализовать логику функции
- Протестировать с разными входными значениями

Шаг 12:

Миссия 12

Расположите шаги создания универсального обработчика событий:

1. Создать функцию высшего порядка
2. Типизировать входные события
3. Вернуть колбэк для конкретного элемента
4. Применить к элементам DOM

Подсказка:

Сначала пишем шаблон функции, потом типизация и применение.

Ответ:

- Создать функцию высшего порядка
- Типизировать входные события
- Вернуть колбэк для конкретного элемента
- Применить к элементам DOM

Модуль 4 - Generics как суперсила

Урок 1 - Основы обобщений

Шаг 01:

Миссия 1

Аня хочет создать функцию, которая может работать с любым типом данных. Какой инструмент TypeScript поможет ей?

Подсказка: Подумайте о параметрах, которые делают функцию универсальной.

Ответ:

- **Использовать generic-параметр**
- Присвоить тип any
- Создать несколько перегрузок функции

Шаг 02:

Миссия 2

Игорь создает функцию addItem, которая добавляет элемент в массив. Как правильно типизировать массив с generics?

Подсказка: Тип массива должен быть согласован с типом добавляемого элемента.

Ответ:

- **function addItem(arr: T[], item: T): T[]**
- function addItem(arr: any[], item: any): any[]
- function addItem(arr: number[], item: string): number[]

Шаг 03:

Миссия 3

Лена хочет написать универсальную функцию `swap` для двух значений. Какой подход использовать?

Подсказка: Функция должна работать с любыми типами без дублирования кода.

Ответ:

- **Использовать два generic-параметра**
- Использовать `any` для обоих параметров
- Создать две отдельные функции для разных типов

🔮 Шаг 04:

Миссия 4

📖 Сопоставьте термин `generics` с описанием:

Ответ:

- `Generic`
 - `Type parameter`
 - `Constraint`
 - `Default type`
1. Позволяет функции или классу быть универсальным
 2. Переменная типа, передаваемая в `generic`
 3. Ограничение на тип параметра
 4. Тип по умолчанию, если параметр не указан

🔮 **Подсказка:**

Подумайте, какая роль каждого элемента в системе `generics`.

- **Generic** — Позволяет функции или классу быть универсальным
- **Type parameter** — Переменная типа, передаваемая в `generic`
- **Constraint** — Ограничение на тип параметра
- **Default type** — Тип по умолчанию, если параметр не указан

🔮 Шаг 05:

Миссия 5

📖 Сопоставьте примеры функций с подходящими `generic`-типами:

Ответ:

- `identity`
 - `getFirst`
 - `mergeObjects`
 - `wrapInArray`
1. `(value: T) => T`
 2. `(arr: T[]) => T`
 3. `(obj1: T, obj2: U) => T & U`
 4. `(value: T) => T[]`

🔮 **Подсказка:**

Подумайте, что возвращает каждая функция и как `generic` помогает типизировать результат.

- **identity** — `(value: T) => T`
- **getFirst** — `(arr: T[]) => T`

- **mergeObjects** — (obj1: T, obj2: U) => T & U
- **wrapInArray** — (value: T) => T[]

🔗 Шаг 06:

Миссия 6

Саша хочет создать функцию, которая фильтрует элементы массива по любому типу. Какой синтаксис использовать?

Подсказка: Обратите внимание на generic для элементов массива.

Ответ:

- **function filterArray(arr: T[], predicate: (item: T) => boolean): T[]**
- function filterArray(arr: any[], predicate: (item: any) => boolean): any[]
- function filterArray(arr: string[], predicate: (item: number) => boolean): string[]

🔗 Шаг 07:

Миссия 7

Игорь хочет ограничить тип generic числовыми типами. Как это сделать?

Подсказка: TypeScript позволяет накладывать ограничения через extends.

Ответ:

- **function calculate(value: T): T**
- function calculate(value: T): number
- function calculate(value: number): number

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте преимущества использования generics:

Ответ:

- Повторное использование кода
 - Безопасная типизация
 - Универсальные функции и классы
 - Сокращение дублирования
1. Функция работает с любыми типами без переписывания
 2. TypeScript проверяет типы на этапе компиляции
 3. Можно создавать функции и классы для разных типов
 4. Меньше повторяющегося кода

🔗 Подсказка:

Подумайте, как generics помогают писать универсальный и безопасный код.

- **Повторное использование кода** — Функция работает с любыми типами без переписывания
- **Безопасная типизация** — TypeScript проверяет типы на этапе компиляции
- **Универсальные функции и классы** — Можно создавать функции и классы для разных типов
- **Сокращение дублирования** — Меньше повторяющегося кода

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте виды generic-ограничений с их примером:

Ответ:

- Типовой параметр с ограничением
- Multiple constraints
- Default generic
- Extending интерфейс

- 1.
- 2.
- 3.
- 4.

💡 **Подсказка:**

Обратите внимание на ключевые слова extends и комбинации типов.

- **Типовой параметр с ограничением** —
- **Multiple constraints** —
- **Default generic** —
- **Extending интерфейс** —

🔗 Шаг 10:

Миссия 10

Лена пишет функцию mergeArrays, которая объединяет массивы любых типов. Какой синтаксис использовать?

Подсказка: Используйте несколько generic-параметров для разных массивов.

Ответ:

- **function mergeArrays(a: T[], b: U[]): (T|U)[]**
- function mergeArrays(a: any[], b: any[]): any[]
- function mergeArrays(a: T[], b: T[]): T[]

🔗 Шаг 11:

Миссия 11

Саша хочет сделать generic-класс Storage для хранения элементов разных типов. Как объявить класс?

Подсказка: Generic-параметр добавляется сразу после имени класса.

Ответ:

- **class Storage { items: T[] = []; }**
- class Storage { items: any[] = []; }
- class Storage { items: T[] = []; }

🔗 Шаг 12:

Миссия 12

📖 Сопоставьте методы generic-класса Storage с их назначением:

Ответ:

- addItem
- getItem
- removeItem
- getAll

1. Добавляет элемент в хранилище
2. Возвращает элемент по индексу
3. Удаляет элемент по индексу
4. Возвращает все элементы

💡 Подсказка:

Подумайте, как класс управляет массивом generic-типов.

- **addItem** — Добавляет элемент в хранилище
- **getItem** — Возвращает элемент по индексу
- **removeItem** — Удаляет элемент по индексу
- **getAll** — Возвращает все элементы

📖 Урок 2 - Ограничения (extends)

💡 Шаг 01:

Миссия 1

Аня разрабатывает функцию, которая работает с объектами пользователей. Она хочет, чтобы функция принимала только объекты, у которых есть поле name . Как правильно ограничить тип через generics?

Подсказка: Подумайте, как гарантировать наличие ключевого свойства у любого передаваемого объекта.

Ответ:

- **function printName(obj: T) {}**
- function printName(obj: T) {}
- function printName(obj: any) {}
- function printName(obj: T) {}

💡 Шаг 02:

Миссия 2

Иван создает generic-функцию, которая должна принимать массив объектов с полем id . Как ограничить generic тип?

Подсказка: Используйте подход, который позволяет работать с объектами, имеющими конкретное свойство.

Ответ:

- **function processItems(items: T[]) {}**
- function processItems(items: T[]) {}
- function processItems(items: any[]) {}
- function processItems(items: T[]) {}

💡 Шаг 03:

Миссия 3

Сергей хочет, чтобы его функция generic принимала объекты с любыми строковыми полями, но как минимум с полем email . Как это записать?

Подсказка: Используйте extends для объекта с обязательным ключом.

Ответ:

- **function sendEmail(user: T) {}**
- function sendEmail(user: T) {}
- function sendEmail(user: any) {}
- function sendEmail(user: T) {}

Шаг 04:

Миссия 4

 Сопоставьте примеры generic-функций с правильными ограничениями

Подсказка: Внимательно прочтите, какое свойство обязательно должно быть в объекте.

Функции:

Ответ:

- Функция для пользователей с именем
- Функция для объектов с id
- Функция для объектов с email
- Функция для любых объектов без ограничений

Типы ограничений:

1. T extends { name: string }
2. T extends { id: number }
3. T extends { email: string }
4. T

- **Функция для пользователей с именем** — T extends { name: string }
- **Функция для объектов с id** — T extends { id: number }
- **Функция для объектов с email** — T extends { email: string }
- **Функция для любых объектов без ограничений** — T

Шаг 05:

Миссия 5

 Сопоставьте персонажей с их задачами при использовании generic-ограничений

Подсказка: Вспомните, какие свойства должны быть у объектов, с которыми они работают.

Персонажи:

Ответ:

- Аня
- Иван
- Сергей
- Ольга

Задачи:

1. Функция для пользователей с именем
2. Функция для объектов с id
3. Функция для объектов с email
4. Функция для любых объектов

- **Аня** — Функция для пользователей с именем
- **Иван** — Функция для объектов с id
- **Сергей** — Функция для объектов с email
- **Ольга** — Функция для любых объектов

Шаг 06:

Миссия 6

 Сопоставьте тип ограничения с его объяснением:

Подсказка: Подумайте, что гарантирует extends.

Типы:

Ответ:

- `T extends { name: string }`
- `T extends { id: number }`
- `T`
- `T extends { email: string }`

Объяснения:

1. Гарантирует, что объект имеет поле name
2. Гарантирует, что объект имеет поле id
3. Нет ограничений на объект
4. Гарантирует, что объект имеет поле email

- **`T extends { name: string }`** — Гарантирует, что объект имеет поле name
- **`T extends { id: number }`** — Гарантирует, что объект имеет поле id
- **`T`** — Нет ограничений на объект
- **`T extends { email: string }`** — Гарантирует, что объект имеет поле email

Шаг 07:

Миссия 7

 Расположите шаги при создании generic-функции с ограничением:

1. Определяем обязательные поля объекта
2. Объявляем generic-параметр с extends
3. Создаем функцию с типом T
4. Проверяем работу функции на объектах с нужными свойствами

Подсказка: Сначала нужно понять требования к объекту, потом объявить тип, затем функцию и проверить её работу.

Ответ:

- Определяем обязательные поля объекта
- Объявляем generic-параметр с extends
- Создаем функцию с типом T
- Проверяем работу функции на объектах с нужными свойствами

Шаг 08:

Миссия 8

 Расположите шаги от простой generic-функции к функции с ограничением:

1. Создаем функцию с generic-параметром T
2. Добавляем extends для обязательных свойств
3. Используем свойства объекта внутри функции
4. Тестируем функцию на объектах с разными свойствами

Подсказка: Сначала generic, потом ограничение, потом использование и проверка.

Ответ:

- Создаем функцию с generic-параметром T
- Добавляем extends для обязательных свойств
- Используем свойства объекта внутри функции
- Тестируем функцию на объектах с разными свойствами

Шаг 09:

Миссия 9

Мария создает generic-функцию для фильтрации объектов с полем role . Как ограничить generic?

Подсказка: Нужно убедиться, что каждый объект имеет поле role .

Ответ:

- **function filterByRole(items: T[]) {}**
- function filterByRole(items: T[]) {}
- function filterByRole(items: any[]) {}
- function filterByRole(items: T[]) {}

Шаг 10:

Миссия 10

 Сопоставьте задачи программистов с правильными generic-ограничениями

Подсказка: Обратите внимание, какое поле обязательно должно присутствовать у объектов.

Задачи:

Ответ:

- Обработка пользователей с именем
- Фильтрация объектов с role
- Сортировка объектов с id
- Любая функция без ограничений

Ограничения:

1. T extends { name: string }
2. T extends { role: string }
3. T extends { id: number }
4. T

- **Обработка пользователей с именем** — T extends { name: string }
- **Фильтрация объектов с role** — T extends { role: string }
- **Сортировка объектов с id** — T extends { id: number }
- **Любая функция без ограничений** — T

Шаг 11:

Миссия 11

 Расположите шаги при добавлении нового ограничения к generic-функции:

1. Определяем обязательные свойства объекта
2. Добавляем extends к generic-параметру
3. Используем свойства объекта в функции
4. Проверяем корректность работы функции на примерах

Подсказка: Помните: сначала требования, потом тип, потом использование и тестирование.

Ответ:

- Определяем обязательные свойства объекта
- Добавляем extends к generic-параметру
- Используем свойства объекта в функции
- Проверяем корректность работы функции на примерах

Шаг 12:

Миссия 12

Павел хочет создать функцию, которая объединяет два массива объектов с обязательным полем id . Как правильно задать generic?

Подсказка: Важно, чтобы объекты имели общее свойство для объединения.

Ответ:

- **function mergeArrays(a: T[], b: T[]) {}**
- function mergeArrays(a: T[], b: T[]) {}
- function mergeArrays(a: any[], b: any[]) {}
- function mergeArrays(a: T[], b: T[]) {}

Шаг 13:

Миссия 13

 Сопоставьте тип ограничения с примером использования:

Подсказка: Вспомните, какое свойство должно быть у объекта для безопасного использования функции.

Типы:

Ответ:

- T extends { name: string }
- T extends { id: number }
- T extends { role: string }
- T

Примеры:

1. Вывод имени пользователя
2. Объединение массивов по id
3. Фильтрация объектов по роли
4. Функция для любых данных

- **T extends { name: string }** — Вывод имени пользователя
- **T extends { id: number }** — Объединение массивов по id

- **T extends { role: string }** — Фильтрация объектов по роли
- **T** — Функция для любых данных

🔗 Шаг 14:

Миссия 14

📖 Расположите шаги создания универсальной generic-функции с ограничением по id:

1. Определяем, какое свойство будет обязательным
2. Создаем generic-параметр с extends
3. Пишем функцию с использованием этого свойства
4. Тестируем функцию на нескольких объектах

Подсказка: Сначала определяем требования, потом тип, потом функция и проверка.

Ответ:

- Определяем, какое свойство будет обязательным
- Создаем generic-параметр с extends
- Пишем функцию с использованием этого свойства
- Тестируем функцию на нескольких объектах

🔗 Шаг 15:

Миссия 15

Наталья разрабатывает generic-функцию для объектов с полями name и role . Как правильно ограничить generic?

Подсказка: Обратите внимание на несколько обязательных полей.

Ответ:

- **function processUser(user: T) {}**
- function processUser(user: T) {}
- function processUser(user: T) {}
- function processUser(user: any) {}

🔗 Шаг 16:

Миссия 16

📖 Сопоставьте персонажей с их типами generic:

Подсказка: Вспомните, какие свойства они используют.

Персонажи:

Ответ:

- Наталья
- Павел
- Мария
- Аня

Типы:

1. T extends { name: string; role: string }
2. T extends { id: number }
3. T extends { role: string }
4. T extends { name: string }

- **Наталья** — `T extends { name: string; role: string }`
- **Павел** — `T extends { id: number }`
- **Мария** — `T extends { role: string }`
- **Аня** — `T extends { name: string }`

💡 Шаг 17:

Миссия 17

📄 Расположите шаги по созданию сложной generic-функции с несколькими обязательными свойствами:

1. Определяем все обязательные свойства объекта
2. Объявляем generic-параметр с `extends`, включающим все свойства
3. Используем свойства внутри функции
4. Проверяем функцию на разных объектах

Подсказка: Сначала требования, потом тип, потом логика функции и проверка на данных.

Ответ:

- Определяем все обязательные свойства объекта
- Объявляем generic-параметр с `extends`, включающим все свойства
- Используем свойства внутри функции
- Проверяем функцию на разных объектах

📖 Урок 3 - Default types и conditional defaults

💡 Шаг 01:

Миссия 1

Алекс разрабатывает generic-функцию с дефолтным типом `T = string`. Как указать default type в TypeScript?

Подсказка: Default type используется прямо при объявлении generic-параметра.

Ответ:

- **`function identity(value: T): T { return value; }`**
- `function identity(value: T = string): T { return value; }`
- `function identity(value: string): string { return value; }`
- `function identity(value: T) { return value; }`

💡 Шаг 02:

Миссия 2

Ирина создает функцию с условным default type: если generic не указан, должен использоваться тип `number`. Как это реализовать?

Подсказка: В TypeScript можно назначать значение по умолчанию для generic-параметра.

Ответ:

- **`function calculate(value: T): T { return value; }`**
- `function calculate(value: T): T { return value; }`
- `function calculate(value: number): number { return value; }`
- `function calculate(value: T = 0): T { return value; }`

💡 Шаг 03:

Миссия 3

Сергей разрабатывает generic-функцию для массива и хочет, чтобы по умолчанию элементы были строками. Как указать default type?

Подсказка: Default type задается в угловых скобках при объявлении generic.

Ответ:

- **function wrapArray(arr: T[]): T[] { return arr; }**
- `function wrapArray(arr: T[] = string[]): T[] { return arr; }`
- `function wrapArray(arr: string[]): string[] { return arr; }`
- `function wrapArray(arr: T[]): T[] { return arr; }`

💡 Шаг 04:

Миссия 4

📄 Сопоставьте условное назначение default type с его примером

Подсказка: Вспомните синтаксис conditional defaults: `T = U extends X ? Y : Z`.

Условие:

Ответ:

- `T = U extends string ? number : boolean`
- `T = U extends number ? string : number`
- `T = U extends boolean ? string : boolean`
- `T = U extends object ? string : number`

Примеры:

1. Если U строка, T = число; иначе boolean
2. Если U число, T = строка; иначе число
3. Если U boolean, T = строка; иначе boolean
4. Если U объект, T = строка; иначе число

- **`T = U extends string ? number : boolean`** — Если U строка, T = число; иначе boolean
- **`T = U extends number ? string : number`** — Если U число, T = строка; иначе число
- **`T = U extends boolean ? string : boolean`** — Если U boolean, T = строка; иначе boolean
- **`T = U extends object ? string : number`** — Если U объект, T = строка; иначе число

💡 Шаг 05:

Миссия 5

📄 Расположите шаги создания generic-функции с default type:

1. Определяем тип, который хотим по умолчанию
2. Объявляем generic-параметр с default
3. Используем generic внутри функции
4. Тестируем функцию на разных типах

Подсказка: Сначала выбираем default, потом тип, потом логика и тестирование.

Ответ:

- Определяем тип, который хотим по умолчанию
- Объявляем generic-параметр с default
- Используем generic внутри функции

- Тестируем функцию на разных типах

Шаг 06:

Миссия 6

Ольга хочет создать функцию с generic-параметром и условным default: если тип не указан, использовать boolean . Как это правильно сделать?

Подсказка: Default type можно назначить прямо в объявлении generic.

Ответ:

- **function flag(value: T): T { return value; }**
- function flag(value: T): T { return value; }
- function flag(value: boolean): boolean { return value; }
- function flag(value: T = true): T { return value; }

Шаг 07:

Миссия 7

 Расположите шаги создания generic-функции с условным default type:

1. Определяем условие для default
2. Объявляем generic-параметр с условным default
3. Используем generic внутри функции
4. Тестируем функцию на разных типах

Подсказка: Сначала задаем условие, потом тип, потом логика и проверка.

Ответ:

- Определяем условие для default
- Объявляем generic-параметр с условным default
- Используем generic внутри функции
- Тестируем функцию на разных типах

Шаг 08:

Миссия 8

Игорь хочет, чтобы generic-параметр T по умолчанию был string , но если указана boolean, тогда default = number . Как реализовать?

Подсказка: Используйте условный default с extends ? : .

Ответ:

- **function process(value: T) { return value; }**
- function process(value: T) { return value; }
- function process(value: T = string) { return value; }
- function process(value: T) { return value; }

Шаг 09:

Миссия 9

 Сопоставьте условные default types с примерами:

Подсказка: Внимательно посмотрите на условие в generic.

Условные default:

Ответ:

- `T = boolean extends T ? number : string`
- `T = string extends T ? boolean : number`
- `T = number extends T ? string : boolean`
- `T = object extends T ? boolean : string`

Примеры:

1. Если `T boolean`, `default = number`; иначе `string`
2. Если `T string`, `default = boolean`; иначе `number`
3. Если `T number`, `default = string`; иначе `boolean`
4. Если `T object`, `default = boolean`; иначе `string`

- **`T = boolean extends T ? number : string`** — Если `T boolean`, `default = number`; иначе `string`
- **`T = string extends T ? boolean : number`** — Если `T string`, `default = boolean`; иначе `number`
- **`T = number extends T ? string : boolean`** — Если `T number`, `default = string`; иначе `boolean`
- **`T = object extends T ? boolean : string`** — Если `T object`, `default = boolean`; иначе `string`

Шаг 10:

Миссия 10

 Расположите шаги создания generic-функции с условным default type по сложной логике:

1. Определяем условие для default type
2. Объявляем generic-параметр с условным default
3. Используем generic внутри функции
4. Тестируем функцию с разными типами

Подсказка: Сначала условие, потом тип, потом логика, затем проверка.

Ответ:

- Определяем условие для default type
- Объявляем generic-параметр с условным default
- Используем generic внутри функции
- Тестируем функцию с разными типами

Шаг 11:

Миссия 11

Олег хочет, чтобы generic-параметр `T` имел default type `string`, но если передан объект, использовать `boolean`. Как это записать?

Подсказка: Используйте условие через `extends` при объявлении generic.

Ответ:

- **`function check(value: T) { return value; }`**
- `function check(value: T) { return value; }`
- `function check(value: T = string) { return value; }`
- `function check(value: T) { return value; }`

Шаг 12:

Миссия 12

 Сопоставьте персонажей с условными default types:

Подсказка: Вспомните их задачи и какие типы они хотели использовать по умолчанию.

Персонажи:

Ответ:

- Олег
- Игорь
- Ольга
- Мария

Default types:

1. object → boolean, иначе string
 2. boolean → number, иначе string
 3. boolean
 4. string
- **Олег** — object → boolean, иначе string
 - **Игорь** — boolean → number, иначе string
 - **Ольга** — boolean
 - **Мария** — string

Шаг 13:

Миссия 13

 Расположите шаги создания сложной generic-функции с условными default types:

1. Определяем все условия для default type
2. Объявляем generic с условными default
3. Используем generic в функции
4. Тестируем с разными типами

Подсказка: Сначала требования, потом тип, потом логика, затем проверка.

Ответ:

- Определяем все условия для default type
- Объявляем generic с условными default
- Используем generic в функции
- Тестируем с разными типами

Шаг 14:

Миссия 14

 Расположите шаги комплексного использования default и conditional default types в функции:

1. Определяем базовый default type
2. Определяем условные default types для разных сценариев
3. Объявляем generic с default и условными default
4. Используем generic внутри функции
5. Тестируем функцию с различными входными данными

Подсказка: Сначала типы, потом объявление, логика и проверка.

Ответ:

- Определяем базовый default type
- Определяем условные default types для разных сценариев

- Объявляем generic с default и условными default
- Используем generic внутри функции
- Тестируем функцию с различными входными данными

Урок 4 - Использование в функциях, типах, классах

Шаг 01:

Миссия 1

Алекс создаёт функцию, которая должна работать с любым типом данных. Как правильно объявить generic-функцию?

Подсказка: Generic-параметры указываются в угловых скобках.

Ответ:

- **function identity(value: T): T { return value; }**
- function identity(value: any): any { return value; }
- function identity(value: T): T { return value; }
- function identity(value: T) { return value; }

Шаг 02:

Миссия 2

Ирина разрабатывает тип, который может быть массивом любого типа. Как это указать через generic?

Подсказка: Используйте generic-параметр при объявлении типа массива.

Ответ:

- **type MyArray = T[];**
- type MyArray = T[];
- type MyArray = Array;
- type MyArray = T[];

Шаг 03:

Миссия 3

Сергей создаёт класс-хранилище данных, который должен быть универсальным для любого типа данных. Как правильно объявить generic-класс?

Подсказка: Generic-параметр указывается сразу после имени класса.

Ответ:

- **class Storage { private data: T[] = []; }**
- class Storage { private data: T[] = []; }
- class Storage { private data: T[] = []; }
- class Storage { private data: T; }

Шаг 04:

Миссия 4

 Сопоставьте персонажей с их задачей использования generic:

Подсказка: Подумайте, для чего каждому нужен generic: функция, тип или класс.

Персонажи:

Ответ:

- Алекс
- Ирина
- Сергей
- Мария

Задачи:

1. Generic-функция
 2. Generic-тип
 3. Generic-класс
 4. Generic-функция с ограничением
- **Алекс** — Generic-функция
 - **Ирина** — Generic-тип
 - **Сергей** — Generic-класс
 - **Мария** — Generic-функция с ограничением

 Шаг 05:

Миссия 5

 Сопоставьте типы generic с их синтаксисом:

Подсказка: Внимательно посмотрите, как задаются параметры generic в функциях, типах и классах.

Типы:

Ответ:

- Generic-функция
- Generic-тип
- Generic-класс
- Generic-интерфейс

Синтаксис:

1. `function func(value: T): T`
 2. `type MyType = T[]`
 3. `class MyClass { }`
 4. `interface MyInterface { value: T; }`
- **Generic-функция** — `function func(value: T): T`
 - **Generic-тип** — `type MyType = T[]`
 - **Generic-класс** — `class MyClass { }`
 - **Generic-интерфейс** — `interface MyInterface { value: T; }`

 Шаг 06:

Миссия 6

 Сопоставьте ограничения generic с примером:

Подсказка: Ограничения позволяют задавать типы, которые наследуют другие типы.

Ограничение:

Ответ:

- `T extends string`

- T extends number
- T extends object
- T extends any[]

Примеры:

1. function logLength(value: T)
 2. function double(value: T)
 3. function clone(obj: T)
 4. function firstElement(arr: T)
- **T extends string** — function logLength(value: T)
 - **T extends number** — function double(value: T)
 - **T extends object** — function clone(obj: T)
 - **T extends any[]** — function firstElement(arr: T)

Шаг 07:

Миссия 7

 Расположите шаги создания generic-функции с ограничением:

1. Определяем ограничение для generic
2. Объявляем generic-параметр с ограничением
3. Используем generic внутри функции
4. Тестируем функцию на соответствующих типах

Подсказка: Сначала ограничение, потом объявление, потом логика, потом тестирование.

Ответ:

- Определяем ограничение для generic
- Объявляем generic-параметр с ограничением
- Используем generic внутри функции
- Тестируем функцию на соответствующих типах

Шаг 08:

Миссия 8

Мария хочет создать generic-функцию, которая возвращает первый элемент массива любого типа. Как правильно объявить функцию?

Подсказка: Используйте generic-параметр с ограничением на массив.

Ответ:

- **function first(arr: T): T[0] { return arr[0]; }**
- function first(arr: T): T { return arr[0]; }
- function first(arr: T): T[0] { return arr[0]; }
- function first(arr: any[]): any { return arr[0]; }

Шаг 09:

Миссия 9

 Сопоставьте generic-функции с их результатом:

Подсказка: Подумайте, какой тип возвращается для каждого generic-параметра.

Функции:

Ответ:

- `identity(value: T)`
- `first(arr: T)`
- `double(value: T)`
- `clone(obj: T)`

Результаты:

1. Возвращает значение того же типа
2. Возвращает первый элемент массива
3. Удваивает числовое значение
4. Создает копию объекта

- **`identity(value: T)`** — Возвращает значение того же типа
- **`first(arr: T)`** — Возвращает первый элемент массива
- **`double(value: T)`** — Удваивает числовое значение
- **`clone(obj: T)`** — Создает копию объекта

Шаг 10:

Миссия 10

 Расположите шаги создания generic-класса `Storage`:

1. Объявляем generic-параметр класса
2. Создаем массив данных типа `T`
3. Добавляем методы для работы с массивом
4. Тестируем класс на разных типах

Подсказка: Сначала generic, потом структура, методы и проверка.

Ответ:

- Объявляем generic-параметр класса
- Создаем массив данных типа `T`
- Добавляем методы для работы с массивом
- Тестируем класс на разных типах

Шаг 11:

Миссия 11

Олег хочет, чтобы generic-функция принимала только объекты с полем `id`. Как правильно объявить тип?

Подсказка: Используйте ограничение `extends` для объекта.

Ответ:

- **`function getId(obj: T) { return obj.id; }`**
- `function getId(obj: T) { return obj.id; }`
- `function getId(obj: { id: number }) { return obj.id; }`
- `function getId(obj: T) { return obj.id; }`

Шаг 12:

Миссия 12

 Сопоставьте классы с их назначением:

Подсказка: Подумайте, какие данные они могут хранить и как использовать generic.

Классы:

Ответ:

- Storage
- Queue
- Stack
- MapStorage

Назначение:

1. Хранение элементов с доступом по ключу
 2. Класс для хранения данных типа T
 3. Очередь элементов типа T
 4. Стек элементов типа T
- **Storage** — Класс для хранения данных типа T
 - **Queue** — Очередь элементов типа T
 - **Stack** — Стек элементов типа T
 - **MapStorage** — Хранение элементов с доступом по ключу

Шаг 13:

Миссия 13

 Расположите шаги создания generic-интерфейса User с полем data:

1. Определяем generic-параметр T
2. Создаем поле data типа T
3. Добавляем методы для работы с data
4. Используем интерфейс для разных типов данных

Подсказка: Сначала generic, потом структура, методы, проверка.

Ответ:

- Определяем generic-параметр T
- Создаем поле data типа T
- Добавляем методы для работы с data
- Используем интерфейс для разных типов данных

Шаг 14:

Миссия 14

Лена хочет, чтобы функция clone принимала объект любого типа и возвращала копию. Как объявить generic?

Подсказка: Используйте generic с ограничением object.

Ответ:

- **function clone(obj: T): T { return { ...obj }; }**
- function clone(obj: T) { return { ...obj }; }
- function clone(obj: object): object { return { ...obj }; }
- function clone(obj: T): T { return [...obj]; }

Шаг 15:

Миссия 15

 Сопоставьте функции с типами параметров:

Подсказка: Подумайте, какое ограничение применено к generic.

Функции:

Ответ:

- `firstElement(arr: T)`
- `getId(obj: T)`
- `double(value: T)`
- `clone(obj: T)`

Типы параметров:

1. Массив элементов
2. Объект с полем `id`
3. Число
4. Объект

- **`firstElement(arr: T)`** — Массив элементов
- **`getId(obj: T)`** — Объект с полем `id`
- **`double(value: T)`** — Число
- **`clone(obj: T)`** — Объект

Шаг 16:

Миссия 16

 Расположите шаги использования generic-класса `Stack`:

1. Объявляем generic-параметр `T`
2. Создаем внутренний массив элементов типа `T`
3. Добавляем методы `push` и `pop`
4. Тестируем стек с разными типами данных

Подсказка: Сначала generic, потом структура, методы, проверка.

Ответ:

- Объявляем generic-параметр `T`
- Создаем внутренний массив элементов типа `T`
- Добавляем методы `push` и `pop`
- Тестируем стек с разными типами данных

Урок 5 - Generic constraints

Шаг 01:

Миссия 1

Аня работает над библиотекой для обработки данных, и хочет заставить функцию принимать только те типы, у которых есть поле `id`. Какой синтаксис `Generic constraints` ей подойдет?

Подсказка: `Constraint` указывается после имени типа в скобках `extends`.

Ответ:

- **`function foo(arg: T): T { ... }`**
- `function foo(arg: T has id: string): T { ... }`
- `function foo(arg: T): T { ... }`
- `function foo(arg: T extends { id: string }): T { ... }`

🔗 Шаг 02:

Миссия 2

Сергей разрабатывает утилиту, которая фильтрует массив значений по полю `length`. Какой `constraint` позволяет работать только с типами, у которых есть `length`?

Подсказка: Встроенный интерфейс `ArrayLike` может помочь, но можно и объявить собственный тип.

Ответ:

- **`function process(arg: T): number { return arg.length; }`**
- `function process(arg: T has length: number): number { return arg.length; }`
- `function process(arg: T): number { ... }` // где `Length` не определён
- `function process(arg: { length: number }): number { ... }` without generics

🔗 Шаг 03:

Миссия 3

Юлия хочет ограничить дженерик функцией, чтобы тип был либо `string` либо `number`. Какой пример верен?

Подсказка: Можно использовать `union`-тип вместе с `extends`.

Ответ:

- **`function parse(input: T): T { return input; }`**
- `function parse(input: T) where T in string|number: T { ... }`
- `function parse(input: T): T { ... }`
- `function parse(input: string | number): string | number` without generics

🔗 Шаг 04:

Миссия 4

Максим строит API-клиент, и хочет синглить тип так, чтобы он был либо `Person`, либо `Company`, оба имеют поле `createdAt`. Как правильно поставить `constraint`?

Подсказка: Можно описать общую часть “has `createdAt`” и применять к обоим.

Ответ:

- **`function log(item: T): void { console.log(item.createdAt); }`**
- `function log(item: T): void { console.log(item.createdAt); }` without verifying types
- `function log(item: Person | Company): void { console.log(item.createdAt); }` without generics
- `function log(item: T): void { ... }` missing type of `createdAt`

🔗 Шаг 05:

Миссия 5

📄 Сопоставьте задачу героя с подходящим Generic constraint.

Понятия:

Ответ:

- валидация объекта с `email`
- функция, принимающая только строки или числа
- работа с коллекцией, у которой есть метод `push`
- чтение свойства `length`

Варианты:

1. (user: T)
2. (value: T)
3. (arr: T)
4. (collection: T)

🔗 Подсказка:

Подумайте, какие свойства нужны для задач: email – свойство объекта; push – метод массива; length – свойство длины.

- **валидация объекта с email** — (user: T)
- **функция, принимающая только строки или числа** — (value: T)
- **работа с коллекцией, у которой есть метод push** — (arr: T)
- **чтение свойства length** — (collection: T)

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте интерфейс constraint с примером использования в жизни программиста.

Понятия:

Ответ:

- для логирования ресурса
- для геометрической фигуры с площадью
- для элемента DOM с id
- для массива чисел

Примеры:

1. (r: T): void
2. (shape: T): number
3. (el: T): string
4. >(arr: T): T

🔗 Подсказка:

Кто что логирует? Что измеряет? DOM-элемент часто имеет id; массив чисел — типизированный Array.

- **для логирования ресурса** — (r: T): void
- **для геометрической фигуры с площадью** — (shape: T): number
- **для элемента DOM с id** — (el: T): string
- **для массива чисел** — >(arr: T): T

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте сравнительные утверждения про ограничения Generic с примерами.

Утверждения:

Ответ:

- constraint помогает безопасно обращаться к свойства типа
- union constraint уменьшают допустимые типы
- constraint может потребовать метод, не только поле

- интерфейс `constraint` позволяет переиспользование

Примеры:

1. `T extends { prop: number }`
2. `T extends A | B`
3. `T extends { toJSON(): string }`
4. `T extends SomeInterface`

🔔 Подсказка:

Смотрите: если у типа есть свойство, если требует метод, если `union`, если интерфейс объявлен заранее.

- **`constraint` помогает безопасно обращаться к свойства типа** — `T extends { prop: number }`
- **`union constraint` уменьшают допустимые типы** — `T extends A | B`
- **`constraint` может потребовать метод, не только поле** — `T extends { toJSON(): string }`
- **интерфейс `constraint` позволяет переиспользование** — `T extends SomeInterface`

🔔 Шаг 08:

Миссия 8

📖 Сопоставьте реальные проблемы с неправильно заданными `constraint`.

Проблемы:

Ответ:

- Компилятор ругается, что у `T` нет метода `talk()`
- Ошибка: свойство `length` не найдено у типа `T`
- Невозможно использовать метод `.map`, потому что `T` может быть не массивом
- `T` может быть любым объектом, но нужен доступ к `name`

Решения (constraints):

1. `T extends { talk(): void }`
2. `T extends { length: number }`
3. `T extends Array`
4. `T extends { name: string }`

🔔 Подсказка:

Подумайте, какая ошибка — про метод, про свойство, или про то, что ожидается массив.

- **Компилятор ругается, что у `T` нет метода `talk()`** — `T extends { talk(): void }`
- **Ошибка: свойство `length` не найдено у типа `T`** — `T extends { length: number }`
- **Невозможно использовать метод `.map`, потому что `T` может быть не массивом** — `T extends Array`
- **`T` может быть любым объектом, но нужен доступ к `name`** — `T extends { name: string }`

🔔 Шаг 09:

Миссия 9

Рассортируйте шаги, которые программист выполняет, чтобы добавить корректный `Generic constraint` в существующую функцию.

1. Добавить `extends` с нужным типом или интерфейсом
2. Проанализировать ошибку компилятора, что свойство не найдено

3. Проверить, какой тип ожидается по логике функции
4. Запустить сборку/тесты после внесения изменений

 Подсказка:

Сначала должна быть проблема, потом мы определяем что именно нужно, затем применяем `constraint`, в конце проверяем.

Ответ:

- Проанализировать ошибку компилятора, что свойство не найдено
- Проверить, какой тип ожидается по логике функции
- Добавить `extends` с нужным типом или интерфейсом
- Запустить сборку/тесты после внесения изменений

 Шаг 10:

Миссия 10

Сергей хочет реализовать `generic` утилиту, сначала проектируем `constraint`, потом опции, затем тестируем. Расположите шаги в правильной последовательности:

1. Написать тесты для функции с ожиданием, что доступно свойство `size`
2. Добавить `constraint`
3. Определить тип, используемый пользователем
4. Реализовать логику доступа к `item.size`

 Подсказка: Тип и требования — сначала, потом реализация, затем проверка.

Ответ:

- Определить тип, используемый пользователем
- Добавить `constraint`
- Реализовать логику доступа к `item.size`
- Написать тесты для функции с ожиданием, что доступно свойство `size`

 Шаг 11:

Миссия 11

Виталий готовит API, где `generic` ограничение влияет на сериализацию. Расположите стадии:

1. Использовать `constraint`, чтобы гарантировать, что есть метод `toJson`
2. Сериализовать объект, вызывая `toJson`
3. Написать интерфейс, описывающий `toJson(): string`
4. Применить этот интерфейс как `Generic constraint`

 Подсказка: Интерфейс описывается до его использования; цель — сериализация.

Ответ:

- Написать интерфейс, описывающий `toJson(): string`
- Применить этот интерфейс как `Generic constraint`
- Сериализовать объект, вызывая `toJson`
- Использовать `constraint`, чтобы гарантировать, что есть метод `toJson`

 Шаг 12:

Миссия 12

Элина разрабатывает модуль валидации, её задача — обработка массива элементов с timestamp. Расположите шаги:

1. Конструировать constraint, что T extends { timestamp: number }
2. Определить структуру элемента с timestamp
3. Перебрать массив и читать elem.timestamp
4. Проверить, что функция работает корректно на разных типах элементов

🔗 **Подсказка: Сначала обобщаем требования, затем работаем с данными, потом проверяем.**

Ответ:

- Определить структуру элемента с timestamp
- Конструировать constraint, что T extends { timestamp: number }
- Перебрать массив и читать elem.timestamp
- Проверить, что функция работает корректно на разных типах элементов

🔗 **Шаг 13:**

Миссия 13

Иван реализует функцию, принимающую ключ объекта. Нужно гарантировать, что ключ действительно существует. Расположите шаги:

1. Определить функцию с K extends keyof T
2. Обработать ситуацию, если пользователь передал неверный ключ
3. Задать generic параметры
4. Вернуть значение свойства obj[key]

🔗 **Подсказка: Ключ определяется после типа объекта, затем используется.**

Ответ:

- Задать generic параметры
- Определить функцию с K extends keyof T
- Вернуть значение свойства obj[key]
- Обработать ситуацию, если пользователь передал неверный ключ

🔗 **Шаг 14:**

Миссия 14

Олег работает над композицией типов: хочет сначала задать default для generic, затем constraint, потом использовать условные типы. Расположите этапы:

1. Создать объявление
2. Определить условный тип внутри generic
3. Применить generic функцию к реальному типу, без указания всех параметров
4. Установить constraint extends U и default = DefaultType

🔗 **Подсказка: Default задаётся вместе с constraint, затем условие, затем применение.**

Ответ:

- Создать объявление
- Установить constraint extends U и default = DefaultType
- Определить условный тип внутри generic
- Применить generic функцию к реальному типу, без указания всех параметров

Урок 6 - Типы-обертки (Wrapper, Validator и пр.)

Шаг 01:

Миссия 1

Аня пишет утилиту Wrapper, которая просто хранит значение любого типа и возвращает его по запросу. Какой сигнатуре класса соответствует такой Wrapper?

Подсказка: подумайте, как класс принимает обобщённый тип и сохраняет значение, как метод его возвращает.

Ответ:

- **class Wrapper { constructor(private value: T) {} get(): T { return this.value; } }**
- class Wrapper { constructor(value: any) {} get(): any { return this.value; } }
- function Wrapper(value: T): T { return value; }
- class Wrapper { constructor(private value: T) {} get(): T { return this.value; } }

Шаг 02:

Миссия 2

Борис хочет сделать валидатор (Validator) для строки: он хочет, чтобы валидатор принимал строку и возвращал либо ошибку, либо строку, если всё прошло. Как можно типизировать такой Validator?

Подсказка: думайте о функциях, которые принимают string и возвращают string | Error.

Ответ:

- **type Validator = (input: string) => string | Error;**
- type Validator = (input: T) => boolean;
- type Validator = (input: T) => string;
- type Validator = (input: T) => string | Error;

Шаг 03:

Миссия 3

Олег создаёт класс SafeResult, который оборачивает функцию и возвращает объект { ok: boolean; value: T } или { ok: false; error: string }. Какой тип параметров подходит для метода, сохраняющего функцию внутри SafeResult?

Подсказка: метод хранит функцию, которая может быть любого возвращаемого типа — используйте дженерик.

Ответ:

- **class SafeResult { run(fn: () => T): { ok: boolean; value?: T; error?: string } { ... } }**
- class SafeResult { run(fn: () => T): { ok: boolean; value?: T; error?: string } { ... } }
- class SafeResult { run(fn: () => T): { ok: boolean; value?: T } { ... } }
- class SafeResult { run(fn: () => any): { ok: boolean; value?: any; error?: string } { ... } }

Шаг 04:

Миссия 4

Юлия хочет, чтобы её Wrapper принимал только функции, и возвращал обёртку, которая имеет ту же сигнатуру. Как можно типизировать такой wrapper?

Подсказка: используйте F extends (...args: any[]) => any.

Ответ:

- **function wrap any>(fn: F): F { return fn; }**
- function wrap(fn: Function): Function { return fn; }
- function wrap(fn: F): any { return fn; }
- function wrap(fn: F): F { return fn; }

🔗 Шаг 05:

Миссия 5

Максим делает валидатор, который принимает один тип T и возвращает T или null если невалидно. Какая сигнатура подходит?

Подсказка: тип возвращаемого значения должен учитывать возможность null .

Ответ:

- **type Validator = (input: T) => T | null;**
- type Validator = (input: any) => T | null;
- type Validator = (input: T) => T | Error;
- type Validator = (input: T) => boolean;

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте сценарий с подходящим Wrapper или Validator типом.

Сценарии:

Ответ:

- хранить значение любого типа и вернуть его
- оборачивать функцию, сохранять аргументы и возвращаемое значение
- валидировать строку, возвращать строку или ошибку
- валидировать объект, возвращать объект с тем же типом или null

Типы:

1. class Wrapper { ... get(): T }
2. function wrap any>(fn: F): F
3. type Validator = (input: string) => string | Error
4. type Validator = (input: T) => T | null

🔗 **Подсказка: Обратите внимание на то, что хранится, что возвращается, и как обрабатывать ошибку или невалидность.**

- **хранить значение любого типа и вернуть его** — class Wrapper { ... get(): T }
- **оборачивать функцию, сохранять аргументы и возвращаемое значение** — function wrap any>(fn: F): F
- **валидировать строку, возвращать строку или ошибку** — type Validator = (input: string) => string | Error
- **валидировать объект, возвращать объект с тем же типом или null** — type Validator = (input: T) => T | null

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте тип обёртки с ситуацией из жизни программиста.

Сценарии:

Ответ:

- декорировать функцию логированием входа и выхода
- обработать возможный сбой и вернуть либо значение либо ошибку
- обернуть значение в контейнер без изменений
- валидатор, который возвращает null при ошибке

Типы:

1. `function wrap any>(fn: F): F`
2. `class SafeResult { run(fn: () => T): { ok: boolean; value?: T; error?: string } }`
3. `class Wrapper { get(): T }`
4. `type Validator = (input: T) => T | null`

💡 **Подсказка: Кто что делает: логирование — это обёртка для функций; сбой — SafeResult; контейнер — Wrapper; null — вариант валидатора.**

- **декорировать функцию логированием входа и выхода** — `function wrap any>(fn: F): F`
- **обработать возможный сбой и вернуть либо значение либо ошибку** — `class SafeResult { run(fn: () => T): { ok: boolean; value?: T; error?: string } }`
- **обернуть значение в контейнер без изменений** — `class Wrapper { get(): T }`
- **валидатор, который возвращает null при ошибке** — `type Validator = (input: T) => T | null`

💡 **Шаг 08:**

Миссия 8

📖 Сопоставьте оборачивающий тип с ключевой характеристикой.

Характеристики:

Ответ:

- сохраняет аргументы и возвращаемое значение оригинальной функции
- возвращает ошибку при сбое функции
- контейнер просто держит значение
- валидатор возвращает null, если невалидно

Типы:

1. `function wrap any>(fn: F): F`
2. `SafeResult`
3. `Wrapper`
4. `Validator = (input: T) => T | null`

💡 **Подсказка: Подумайте, что делает каждый тип: error, null, просто хранение, сохранение сигнатуры функции.**

- **сохраняет аргументы и возвращаемое значение оригинальной функции** — `function wrap any>(fn: F): F`
- **возвращает ошибку при сбое функции** — `SafeResult`
- **контейнер просто держит значение** — `Wrapper`
- **валидатор возвращает null, если невалидно** — `Validator = (input: T) => T | null`

💡 **Шаг 09:**

Миссия 9

 Сопоставьте неудачные реализации с ошибками при использовании Wrapper / Validator.

Проблемы:

Ответ:

- функция обёртки теряет тип возвращаемого значения
- валидатор возвращает Error вместо string
- Wrapper сохраняет значение, но get возвращает any
- Validator допускает undefined, хотя тип не допускает

Ошибочные реализации:

1. function wrap(fn: Function): Function
2. type Validator = (input: string) => Error
3. class Wrapper { get(): any }
4. type Validator = (input: T) => T | undefined

 **Подсказка: Какая ошибка соответствует какому сценарию: потеря типа, неправильный возвращаемый тип, использование any, unexpected undefined.**

- **функция обёртки теряет тип возвращаемого значения** — function wrap(fn: Function): Function
- **валидатор возвращает Error вместо string** — type Validator = (input: string) => Error
- **Wrapper сохраняет значение, но get возвращает any** — class Wrapper { get(): any }
- **Validator допускает undefined, хотя тип не допускает** — type Validator = (input: T) => T | undefined

 **Шаг 10:**

Миссия 10

Пётр внедряет Wrapper в код: сначала он определяет класс, потом прописывает тесты, затем использует его и проверяет. Расположите шаги в порядке, как это должно быть.

1. Создать класс Wrapper с конструктором и методом get()
2. Написать тест, который создаёт экземпляр Wrapper и проверяет get() возвращает число
3. Использовать Wrapper в коде, передав строку или число
4. Запустить тесты и убедиться, что всё проходит

 **Подсказка: Сначала определение, затем тесты, затем использование, потом проверка.**

Ответ:

- Создать класс Wrapper с конструктором и методом get()
- Написать тест, который создаёт экземпляр Wrapper и проверяет get() возвращает число
- Использовать Wrapper в коде, передав строку или число
- Запустить тесты и убедиться, что всё проходит

 **Шаг 11:**

Миссия 11

Светлана делает SafeResult: её план такой — сначала определить тип, затем реализовать run, потом тестировать с функцией, которая бросает, потом обрабатывать error. Расположите шаги в правильной последовательности:

1. Определить класс SafeResult с методом run(fn: () => T)
2. Реализовать внутри run вызов fn и ловить исключения

3. Написать тест с функцией, которая выбрасывает exception
4. Обработать error в возвращаемом объекте, выставив ok: false и error

🔗 **Подсказка: Сначала структура, потом логика, потом сценарий исключения, потом обработка ошибки.**

Ответ:

- Определить класс SafeResult с методом run(fn: () => T)
- Реализовать внутри run вызов fn и ловить исключения
- Написать тест с функцией, которая выбрасывает exception
- Обработать error в возвращаемом объекте, выставив ok: false и error

🔗 Шаг 12:

Миссия 12

Настя делает обёртку для функций с логированием: её порядок — сначала определить сигнатуру wrap, потом имплементировать обёртку, затем протестировать на конкретной функции, потом убедиться, что signature совпадает. Расположите шаги:

1. Определить функцию wrap any>(fn: F): F
2. Имплементировать логирование до и после вызова fn внутри wrap
3. Использовать wrap на конкретной функции, например (a: number) => a * 2
4. Написать тест, что wrap сохраняет возвращаемый тип и параметры

🔗 **Подсказка: Сначала тип, потом реализация, потом использование, потом проверка типов и тестов.**

Ответ:

- Определить функцию wrap any>(fn: F): F
- Имплементировать логирование до и после вызова fn внутри wrap
- Использовать wrap на конкретной функции, например (a: number) => a * 2
- Написать тест, что wrap сохраняет возвращаемый тип и параметры

🔗 Шаг 13:

Миссия 13

Игорь делает вложенный валидатор, сначала тип alias, затем сложная структура, потом проверка, потом возвращение ошибки или структуры. Расположите шаги:

1. Объявить alias типа, описывающий объект с несколькими полями
2. Использовать Validator для объекта этого alias
3. Написать тест, передающим неверный объект (отсутствует поле или неправильный тип)
4. Обрабатывать результат, возвращая null или валидный объект

🔗 **Подсказка: Тип alias до использования, тест до обработки ошибок.**

Ответ:

- Объявить alias типа, описывающий объект с несколькими полями
- Использовать Validator для объекта этого alias
- Написать тест, передающим неверный объект (отсутствует поле или неправильный тип)
- Обрабатывать результат, возвращая null или валидный объект

🔗 Шаг 14:

Миссия 14

Ольга создает композицию обёрток: `Wrapper` → `Validator` → `SafeResult`. Её порядок действий: сначала `Wrapper`, потом `Validator`, затем `SafeResult`, потом тесты. Расположите шаги:

1. Создать `Wrapper`
2. Объявить `Validator = (input: T) => T | null`
3. Имплементировать `SafeResult` с методом `run`
4. Написать тесты, которые комбинируют все три: свойство `validate` внутри `wrapper`, `run` внутри `SafeResult`

💡 **Подсказка: Постройте базовые обёртки, потом сложите их, потом проверьте.**

Ответ:

- Создать `Wrapper`
- Объявить `Validator = (input: T) => T | null`
- Имплементировать `SafeResult` с методом `run`
- Написать тесты, которые комбинируют все три: свойство `validate` внутри `wrapper`, `run` внутри `SafeResult`

💡 **Шаг 15:**

Миссия 15

Антон хочет, чтобы `Wrapper` и `SafeResult` работали вместе: сначала оборачивает значение, потом обрабатывает функцию через `SafeResult`, потом возвращает обёрнутое значение или ошибку, потом тесты. Расположите:

1. Создать `Wrapper` и его метод `get()`
2. Имплементировать `SafeResult` с `run(fn: () => T)`
3. Использовать `Wrapper` внутри `SafeResult`, передав функцию, возвращающую число
4. Написать тест, что результат `SafeResult.run` возвращает обёрнутый `Wrapper` или ошибку

💡 **Подсказка: Сначала определение обоих типов, затем их совместное использование, затем проверка.**

Ответ:

- Создать `Wrapper` и его метод `get()`
- Имплементировать `SafeResult` с `run(fn: () => T)`
- Использовать `Wrapper` внутри `SafeResult`, передав функцию, возвращающую число
- Написать тест, что результат `SafeResult.run` возвращает обёрнутый `Wrapper` или ошибку

💡 **Шаг 16:**

Миссия 16

Кирилл строит расширение: имеет `Validator`, `SafeResult`, и ещё хочет автоматическую цепочку: сначала валидирует, если валидно — работает `SafeResult`, если нет — ошибка. Расположите этапы:

1. Объявить `Validator = (input: T) => T | null`
2. Объявить `SafeResult` с `run(fn: () => T)`
3. Создать функцию `validateThenRun(input: T, fn: () => T): { ok: boolean; value?: T; error?: string }`
4. **Внутри `validateThenRun` сначала вызвать `Validator`, затем если результат `null` — ошибка, иначе вызвать `SafeResult.run` с `fn`**

💡 **Подсказка: Построение по слоям: валидатор, `SafeResult`, потом объединяющая логика.**

Ответ:

- Объявить `Validator = (input: T) => T | null`
- Объявить `SafeResult` с `run(fn: () => T)`
- Создать функцию `validateThenRun(input: T, fn: () => T): { ok: boolean; value?: T; error?: string }`
- **Внутри `validateThenRun` сначала вызвать `Validator`, затем если результат `null` — ошибка, иначе вызвать `SafeResult.run` с `fn`**

Урок 7 - Квест Построить generic FormBuilder

Шаг 01:

Миссия 1

Аня решила начать строить FormBuilder. Первый шаг — определить тип для значений формы, которая хранит поля с разными типами (`string`, `number`, `boolean`). Какой тип подойдёт?

Подсказка: подумайте, как описать объект, где ключи — строки, а значения — разные типы.

Ответ:

- **`type FormValues = { [key: string]: string | number | boolean; }`**
- `type FormValues = Record;`
- `type FormValues = Array;`
- `type FormValues = unknown;`

Шаг 02:

Миссия 2

Борис хочет, чтобы FormBuilder принимал generic-параметр `T`, который ограничивает форму к объекту с ключами и значениями строк или чисел. Какую сигнатуру класса выбрать?

Подсказка: используйте `>`.

Ответ:

- **`class FormBuilder> { ... }`**
- `class FormBuilder { (...)`
- `type FormBuilder = Record;`
- `class FormBuilder { ... }`

Шаг 03:

Миссия 3

Юлия хочет добавить метод `setField`, который принимает имя поля и значение, и обновляет внутреннее хранилище формы. Как должна выглядеть типизация?

Подсказка: имя из `keyof T`, значение типа `T[Name]`, и метод ничего не возвращает.

Ответ:

- **`setField(name: Name, value: T[Name]): void`**
- `setField(name: keyof T, value: any): void`
- `setField(name: Name, value: T): void`
- `setField(name: Name, value: string | number): void`

Шаг 04:

Миссия 4

 Сопоставьте части API FormBuilder с их назначениями.

Части API:

Ответ:

- `getField(name: Name): T[Name]`
- `setField(name: Name, value: T[Name]): void`
- `validate(): boolean`
- `reset(values: Partial): void`

Назначения:

1. получить текущее значение поля
2. установить новое значение поля
3. проверить, что все поля валидны
4. сбросить форму к новым значениям либо default

💡 **Подсказка: Какие методы обычно встречаются в формах: `get`, `set`, `validate` и `reset`.**

- **`getField(name: Name): T[Name]`** — получить текущее значение поля
- **`setField(name: Name, value: T[Name]): void`** — установить новое значение поля
- **`validate(): boolean`** — проверить, что все поля валидны
- **`reset(values: Partial): void`** — сбросить форму к новым значениям либо default

💡 **Шаг 05:**

Миссия 5

📄 Сопоставьте примеры интерфейсов типов, используемых в FormBuilder, с описаниями.

Интерфейсы:

Ответ:

- `type FormValues = Record`
- `type PartialFormValues = { [K in keyof T]?: T[K] }`
- `type RequiredFormValues = { [K in keyof T]-?: T[K] }`
- `type ValidatorsMap = { [K in keyof T]: (value: T[K]) => boolean }`

Назначения:

1. все поля обязательны
2. только часть полей из формы может быть задана
3. отображение функций-валидаторов для каждого поля
4. тип для всех полей

💡 **Подсказка: Обратите внимание на `?` и `-?` при обработке обязательности, и на использование `Record`.**

- **`type FormValues = Record`** — тип для всех полей
- **`type PartialFormValues = { [K in keyof T]?: T[K] }`** — только часть полей из формы может быть задана
- **`type RequiredFormValues = { [K in keyof T]-?: T[K] }`** — все поля обязательны
- **`type ValidatorsMap = { [K in keyof T]: (value: T[K]) => boolean }`** — отображение функций-валидаторов для каждого поля

💡 **Шаг 06:**

Миссия 9 📄 Сопоставьте варианты валидации с полем формы и типом. Поля формы: `username: string` `age: number` `email: string` `subscribe: boolean` Валидации: `(value: string)`

**=> value.length > 0 (value: number) => value >= 18 (value: string) => value.includes("@")
(value: boolean) => value === true**

🔗 **Подсказка:** Для каждого типа поля нужна валидатор, работающий именно с этим типом.

Ответ:

- **username: string** — (value: string) => value.length > 0
- **age: number** — (value: number) => value >= 18
- **email: string** — (value: string) => value.includes("@")
- **subscribe: boolean** — (value: boolean) => value === true

🔗 **Шаг 07:**

Миссия 7

📖 Сопоставьте типы настроек FormBuilder с их возможными значениями.

Настройки:

Ответ:

- optional fields
- validators map
- initial values
- disabling all fields

Типы настроек:

1. PartialFormValues
2. ValidatorsMap
3. Partial
4. boolean

🔗 **Подсказка:** Какая настройка отвечает за необязательность, какая — за валидацию и т.п.

- **optional fields** — PartialFormValues
- **validators map** — ValidatorsMap
- **initial values** — Partial
- **disabling all fields** — boolean

🔗 **Шаг 08:**

Миссия 8

Олег хочет построить форму: сначала определить generic класс, затем определить методы (getField, setField), потом валидаторы, потом сброс (reset). Расположите шаги:

1. Определить class FormBuilder
2. Добавить методы getField и setField
3. Добавить структуру валидаторов (ValidatorsMap)
4. Добавить метод reset(values: Partial): void

🔗 **Подсказка:** Строим API постепенно: сначала класс и базовые методы, потом расширения.

Ответ:

- Определить class FormBuilder
- Добавить методы getField и setField

- Добавить структуру валидаторов (ValidatorsMap)
- Добавить метод reset(values: Partial): void

🔗 Шаг 09:

Миссия 9

Наташа хочет, чтобы FormBuilder работал с формой, где есть вложенные объекты и массивы. Порядок задач: сначала типы вложенности, потом методы getField для вложенных, потом validate, потом тест вложенных. Расположите шаги:

1. Определить вложенную структуру типа T с объектами и массивами
2. Расширить getField, чтобы работать с путями вложенных ключей
3. Добавить validation, учитывающую вложенные объекты и массивы
4. Написать тесты, использующие вложенные поля и массивы

🔗 **Подсказка: Сначала тип, потом методы, потом валидация, затем тестирование.**

Ответ:

- Определить вложенную структуру типа T с объектами и массивами
- Расширить getField, чтобы работать с путями вложенных ключей
- Добавить validation, учитывающую вложенные объекты и массивы
- Написать тесты, использующие вложенные поля и массивы

🔗 Шаг 10:

Миссия 10

Иван хочет сделать метод validate() , который проверяет все поля по ValidatorsMap и возвращает true/false. Порядок: определить тип ValidatorsMap, потом собрать validators, затем реализовать validate, потом тесты. Расположите шаги:

1. Определить type ValidatorsMap
2. Создать объект с валидаторскими функциями для каждого поля
3. Реализовать validate(): boolean в классе
4. Написать тесты, которые проверяют false, когда хотя бы один валидатор не проходит

🔗 **Подсказка: Тип — структура — логика — проверка.**

Ответ:

- Определить type ValidatorsMap
- Создать объект с валидаторскими функциями для каждого поля
- Реализовать validate(): boolean в классе
- Написать тесты, которые проверяют false, когда хотя бы один валидатор не проходит

🔗 Шаг 11:

Миссия 11

Виталий хочет добавить метод toJSON() , чтобы форму можно было сериализовать только с валидными полями. Порядок: определить, как сериализовать, фильтровать невалидные, then возвращать чистый объект. Расположите шаги:

1. Определить возвращаемый тип type ValidFormData = { [K in keyof T]: T[K] }
2. В классе FormBuilder реализовать toJSON(): ValidFormData
3. Внутри toJSON проверять validate() === true
4. Возвращать объект значений формы

🔗 **Подсказка:** Сначала тип результата, потом реализация, затем проверка, потом возврат.

Ответ:

- Определить возвращаемый тип `ValidFormData = { [K in keyof T]: T[K] }`
- В классе `FormBuilder` реализовать `toJSON(): ValidFormData`
- Внутри `toJSON` проверять `validate() === true`
- Возвращать объект значений формы

🔗 **Шаг 12:**

Миссия 12

Дмитрий хочет готовую версию `FormBuilder`: всё: класс, методы `get/set`, валидация, `reset`, `toJSON`, `asyncValidate`. Он собирается: сначала собрать типы, потом методы, потом тесты, потом интеграцию в UI.

1. Определить все типы: `FormValues`, `ValidatorsMap`, `AsyncValidatorsMap`, `ValidFormData`
2. Имплементировать методы класса: `getField`, `setField`, `validate`, `validateAsync`, `reset`, `toJSON`
3. Написать тесты, покрывающие все методы и случаи (валидные, невалидные, `async`)
4. Интегрировать `FormBuilder` в UI-компонент или пример использования

🔗 **Подсказка:** Сначала типы, потом реализация, затем проверка, и только потом применение в интерфейсе.

Ответ:

- Определить все типы: `FormValues`, `ValidatorsMap`, `AsyncValidatorsMap`, `ValidFormData`
- Имплементировать методы класса: `getField`, `setField`, `validate`, `validateAsync`, `reset`, `toJSON`
- Написать тесты, покрывающие все методы и случаи (валидные, невалидные, `async`)
- Интегрировать `FormBuilder` в UI-компонент или пример использования

📁 Модуль 5 - Работа с типами и утилитами

📖 Урок 1 - `keyof`, `typeof`, `in`, `as`

🔗 **Шаг 01:**

Миссия 1

Аня нашла объект `const user = { name: "Alice", age: 30 }`. Она хочет тип `UserKeys`, состоящий из ключей этого объекта. Какой тип подойдёт?

Подсказка: используйте `typeof` чтобы получить тип объекта, и `keyof` чтобы из него извлечь ключи.

Ответ:

- **`type UserKeys = keyof typeof user;`**
- `type UserKeys = typeof user;`
- `type UserKeys = keyof user;`
- `type UserKeys = "name" | "age";`

🔗 **Шаг 02:**

Миссия 2

Борис создаёт функцию `getProp`, которая принимает объект `obj` и ключ `key`, и возвращает значение по ключу. Как правильно написать её типы?

Подсказка: используйте `generic` и `K extends keyof T` и индексный доступ.

Ответ:

- **function getProp(obj: T, key: K): T[K];**
- function getProp(obj: any, key: string): any;
- function getProp(obj: T, key: keyof any): unknown;
- function getProp(obj: T, key: string): T[string];

 **Шаг 03:**

Миссия 3

Сергей делает коллекцию статуса, представленных как объект: `const Status = { LOADING: "loading", SUCCESS: "success", ERROR: "error" }` и хочет тип `StatusKeys` быть `"LOADING" | "SUCCESS" | "ERROR"`. Как его получить?

Подсказка: объект — `value`, для типа — `typeof`, и затем извлечь ключи.

Ответ:

- **type StatusKeys = keyof typeof Status;**
- type StatusKeys = keyof Status;
- type StatusKeys = typeof Status;
- type StatusKeys = "loading" | "success" | "error";

 **Шаг 04:**

Миссия 4

Юлия хочет создать тип, который берёт ключи интерфейса `Person { name: string; age: number; }` и возвращает новый интерфейс с теми же ключами, но все поля делают `readonly`. Как можно описать такой mapped type?

Подсказка: используйте `in` `keyof` и модификатор `readonly`.

Ответ:

- **type ReadonlyPerson = { readonly [K in keyof Person]: Person[K] }**
- type ReadonlyPerson = Person;
- type ReadonlyPerson = { [K in Person]: Person[K] }
- type ReadonlyPerson = { readonly Person[K] in keyof Person }

 **Шаг 05:**

Миссия 5

Максим хочет использовать оператор `as` в типах: у него есть тип `type A = "x" | "y" | "z"` и он хочет создать тип, который переименует `"z"` в `"ZED"`. Как примерно выглядит такая конструкция?

Подсказка: подумайте про `conditional types` и `as` в `mapped types`.

Ответ:

- **type RenameZ = { [K in T as K extends "z" ? "ZED" : K]: K }[T];**
- type RenameZ = "x" | "y" | "ZED";
- type RenameZ = { [K in T]: K as "ZED" };
- type RenameZ = T;

 **Шаг 06:**

Миссия 6

 Сопоставьте выражения с их результатами типов.

Выражения:

Ответ:

- `keyof { id: number; name: string; }`
- `typeof [1, 2, 3][number]`
- `type Values = T[keyof T]`
- `typeof "hello"`

Результаты:

1. `"id" | "name"`
2. `number`
3. тип значений всех свойств `T`
4. `"string"`

💡 **Подсказка: Разберитесь: чему равен индекс тип массива, что даёт "keyof", и как работает typeof на литералах.**

- **`keyof { id: number; name: string; }`** — `"id" | "name"`
- **`typeof [1, 2, 3][number]`** — `number`
- **`type Values = T[keyof T]`** — тип значений всех свойств `T`
- **`typeof "hello"`** — `"string"`

💡 Шаг 07:

Миссия 7

📖 Сопоставьте случаи использования `keyof`, `typeof`, `in`, `as` с их описаниями.

Операторы:

Ответ:

- `keyof`
- `typeof`
- `in`
- `as`

Назначения:

1. создаёт объединение ключей типа объекта
2. получает тип значения переменной или свойства во время компиляции из runtime-значения
3. перебирает ключи при создании `mapped type`
4. переименовывает или утверждает типы в `mapped types` или утверждениях

💡 **Подсказка: Обратите внимание на синтаксис `mapped` и на ключевые слова.**

- **`keyof`** — создаёт объединение ключей типа объекта
- **`typeof`** — получает тип значения переменной или свойства во время компиляции из runtime-значения
- **`in`** — перебирает ключи при создании `mapped type`
- **`as`** — переименовывает или утверждает типы в `mapped types` или утверждениях

💡 Шаг 08:

Миссия 8

Ольга хочет научиться применять `keyof` и `typeof` в правильном порядке: сначала `typeof`, потом `keyof`, потом `indexed access`. Расположите шаги:

1. иметь `value` (объект или константа)
2. использовать `typeof value`
3. применить `keyof typeof value`
4. использовать `value[key]` или `typeof value[key]` для получения типа значения

🔗 **Подсказка: Сначала получают тип объекта, потом ключи, потом значение по ключу.**

Ответ:

- иметь `value` (объект или константа)
- использовать `typeof value`
- применить `keyof typeof value`
- использовать `value[key]` или `typeof value[key]` для получения типа значения

🔗 **Шаг 09:**

Миссия 9

Иван изучает `mapped types` с `in` и `as`. Ему надо: выбрать ключи, затем переопределить часть ключей, затем задать значение, потом проверить тип. Расположите шаги:

1. определить `union` типов ключей
2. создать `mapped type` с `[K in Keys as ...]`
3. указать, что значение свойства зависит от `K`
4. проверить, что новый тип соответствует ожидаемому

🔗 **Подсказка: Mapped type строится сначала на ключах, then you can rename ключи через `as`, then описать тип значений, потом тесты или проверка.**

Ответ:

- определить `union` типов ключей
- создать `mapped type` с `[K in Keys as ...]`
- указать, что значение свойства зависит от `K`
- проверить, что новый тип соответствует ожидаемому

🔗 **Шаг 10:**

Миссия 10

Катя хочет сначала определить типизированный объект, потом `type alias` с `indexed access`, потом выйти через `keyof` и `typeof`, затем использовать всё в функции. Расположите шаги:

1. объявить объект `const config = { host: "localhost", port: 8080 }`
2. определить `type Config = typeof config;`
3. определить `type ConfigKeys = keyof Config;`
4. функция `function getConfig(key: K): Config[K]`

🔗 **Подсказка: Константа → `typeof` → `keyof` → функция с `generic`.**

Ответ:

- объявить объект `const config = { host: "localhost", port: 8080 }`
- определить `type Config = typeof config;`
- определить `type ConfigKeys = keyof Config;`
- функция `function getConfig(key: K): Config[K]`

🔗 Шаг 11:

Миссия 11

Вика хочет составить mapped type, который делает все свойства интерфейса необязательными и ключи строками в верхнем регистре. Её шаги: взять ключи через keyof, as Uppercase<>, mapped type, проверка. Расположите:

1. определить union ключей: keyof T
2. создать mapped type с [K in keyof T as Uppercase]?: T[K]
3. написать интерфейс T с несколькими свойствами
4. протестировать, что ключи изменены и свойства необязательные

🔗 **Подсказка: Обратите внимание, что as может менять ключ, ? делает необязательность.**

Ответ:

- определить union ключей: keyof T
- создать mapped type с [K in keyof T as Uppercase]?: T[K]
- написать интерфейс T с несколькими свойствами
- протестировать, что ключи изменены и свойства необязательные

🔗 Шаг 12:

Миссия 12

Рома хочет сделать функцию pick, которая принимает объект и массив ключей, и возвращает новый объект только с этими ключами. Он планирует: generic с T и K extends keyof T, затем функция возвращает Record, потом тесты, потом использование. Расположите шаги:

1. объявить сигнатуру function pick(obj: T, keys: K[]): Record
2. реализовать логику: создавать новый объект, копировать выбранные свойства
3. написать тесты, что pick работает с разным набором ключей
4. использовать typeof + keyof для объекта literals, передав конкретные ключи

🔗 **Подсказка: Generic → реализация → тесты → применение на literal объектах.**

Ответ:

- объявить сигнатуру function pick(obj: T, keys: K[]): Record
- реализовать логику: создавать новый объект, копировать выбранные свойства
- написать тесты, что pick работает с разным набором ключей
- использовать typeof + keyof для объекта literals, передав конкретные ключи

🔗 Шаг 13:

Миссия 13

Даша хочет построить утилиту, которая переименует ключи объекта по правилу: если ключ начинается с "old_" — убрать префикс и заглавную букву, иначе оставить. План: определить union ключей, mapped type с as, функция, тест. Расположите шаги:

1. определить union ключей: keyof T
2. создать mapped type { [K in keyof T as K extends `old\_\${infer Rest}` ? Capitalize : K]: T[K] }
3. написать функцию, принимающую объект и возвращающую объект с новым типом ключей
4. протестировать на примере объекта с ключами "old_name", "age" и др.

🔗 **Подсказка: Используйте условные типы и as для переименования; тесты нужны, чтобы убедиться, что типы совпадают.**

Ответ:

- определить union ключей: `keyof T`
- создать mapped тип `{ [K in keyof T as K extends `old_${infer Rest}` ? Capitalize : K]: T[K] }`
- написать функцию, принимающую объект и возвращающую объект с новым типом ключей
- протестировать на примере объекта с ключами "old_name", "age" и др.

🔗 Шаг 14:

Миссия 14

Женя хочет, чтобы утилита `omit` убирала из типа поля по ключам. Её порядок: сначала `generic`, затем mapped type с `in` и `conditional excluding`, затем реализовать функцию, потом тест. Расположите шаги:

1. написать `type Omit = Pick;`
2. реализовать функцию `omit(obj, keys)`
3. написать тест, что `omit` убирает правильные ключи
4. использовать `typeof + keyof` на конкретном объекте и вызвать `omit`

🔗 Подсказка: `Exclude/Exclude` как конструкция `useful` с `keyof` и `mapped types`.

Ответ:

- написать `type Omit = Pick<>;`
- реализовать функцию `omit(obj, keys)`
- написать тест, что `omit` убирает правильные ключи
- использовать `typeof + keyof` на конкретном объекте и вызвать `omit`

📖 Урок 2 - Маппинг типов (Mapped Types)

🔗 Шаг 01:

Миссия 1

Максим решил, что хочет поменять имена ключей у типа `Person { first: string; last: string }`, чтобы они начинались с "get" и были функциями, возвращающими исходное значение. Как может выглядеть такой mapped type?

Подсказка: используйте `as` с `template literal types` и `mapped type`.

Ответ:

- `type Getters = { [K in keyof T as `get${Capitalize}<K>`]: () => T[K] }`
- `type Getters = { [K in keyof T]: get${Capitalize}<K> }`
- `type Getters = { getK: () => T[K] } // не mapped`
- `type Getters = { [K in keyof T]: () => any }`

🔗 Шаг 02:

Миссия 2

📄 Сопоставьте mapped типы с их значениями на примере:

Типы:

Ответ:

- `{ [K in keyof Person]: Person[K] | null }`
- `{ readonly [K in keyof Person]: Person[K] }`
- `{ [K in keyof Person]?: Person[K] }`
- `{ [K in keyof Person as `get${Capitalize}<K>`]: () => Person[K] }`

Примеры:

1. все поля могут быть null
2. все поля readonly
3. все поля optional
4. для каждого свойства создать метод getX

💡 **Подсказка:** Сравните, что делает mapped тип: `optional?`, `readonly`, `value` | `null`, или добавление метода.

- `{ [K in keyof Person]: Person[K] | null }` — все поля могут быть null
- `{ readonly [K in keyof Person]: Person[K] }` — все поля readonly
- `{ [K in keyof Person]?: Person[K] }` — все поля optional
- `{ [K in keyof Person as `get${Capitalize}`]: () => Person[K] }` — для каждого свойства создать метод getX

🔗 **Шаг 03:**

Миссия 3

📄 Сопоставьте описания с mapped типами:

Описания:

Ответ:

- сделать все свойства объекта `readonly`
- сделать все свойства объекта `optional`, `nullable`
- переименовать ключи с помощью `template literal`
- убрать `optional` свойства

Mapped типы:

1. `{ readonly [K in keyof T]: T[K] }`
2. `{ [K in keyof T]?: T[K] | null }`
3. `{ [K in keyof T as `get${Capitalize}`]: () => T[K] }`
4. `{ [K in keyof Maybe]-?: Maybe[K] }`

💡 **Подсказка:** Смотрите на `readonly`, `?`, `as`, и `-?` модификатор.

- **сделать все свойства объекта `readonly`** — `{ readonly [K in keyof T]: T[K] }`
- **сделать все свойства объекта `optional`, `nullable`** — `{ [K in keyof T]?: T[K] | null }`
- **переименовать ключи с помощью `template literal`** — `{ [K in keyof T as `get${Capitalize}`]: () => T[K] }`
- **убрать `optional` свойства** — `{ [K in keyof Maybe]-?: Maybe[K] }`

🔗 **Шаг 04:**

Миссия 4

📄 Сопоставьте mapped type constructions с их предназначением:

Конструкции:

Ответ:

- `{ [K in keyof T]: string }`
- `{ [K in keyof T as Uppercase]: T[K] }`
- `{ [K in keyof T]-?: T[K] }`
- `{ readonly [K in keyof T]?: T[K] | undefined }`

Назначения:

1. все значения стали string
2. ключи в верхнем регистре
3. все свойства обязательные
4. поля readonly + optional + могут быть undefined

🔗 **Подсказка: Обратите внимание на комбинации модификаторов readonly, ?, -?, и использование as и Uppercase.**

- **{ [K in keyof T]: string }** — все значения стали string
- **{ [K in keyof T as Uppercase]: T[K] }** — ключи в верхнем регистре
- **{ [K in keyof T]-?: T[K] }** — все свойства обязательные
- **{ readonly [K in keyof T]?: T[K] | undefined }** — поля readonly + optional + могут быть undefined

🔗 **Шаг 05:**

Миссия 5

Ольга хочет выстроить процесс создания mapped type постепенно: сначала копирование типа, затем добавление optional, затем readonly, затем nullable. Расположите шаги:

1. начать с **{ [K in keyof T]: T[K] }**
2. сделать поля optional: **?:**
3. сделать их readonly
4. добавить **| null** к каждому полю

🔗 **Подсказка: Стройте трансформации одну за другой: базовый mapped → optional → readonly → value changes.**

Ответ:

- начать с **{ [K in keyof T]: T[K] }**
- сделать поля optional: **?:**
- сделать их readonly
- добавить **| null** к каждому полю

🔗 **Шаг 06:**

Миссия 6

Иван хочет сначала сделать mapped тип, переименовав ключи, затем сделать поля optional, потом значение через условный тип (например number → string), затем тестирование. Расположите шаги:

1. создать mapped type с as для переименования ключей
2. добавить **?:** к свойствам
3. использовать условный тип для типов полей
4. написать тесты, чтобы проверить, что переименование, optional и условие работают

🔗 **Подсказка: Ключи → optional → изменение типа → проверка.**

Ответ:

- создать mapped type с as для переименования ключей
- добавить **?:** к свойствам
- использовать условный тип для типов полей
- написать тесты, чтобы проверить, что переименование, optional и условие работают

🔗 Шаг 07:

Миссия 7

Катя хочет инструмент, который сначала делает все свойства nullable, затем убирает null у определённых ключей, потом делает readonly, в конце optional. Расположите этапы:

1. тип типа с { [K in keyof T]: T[K] | null }
2. убрать null для ключей, например через условный тип и mapping
3. сделать все readonly
4. применить optional к свойствам

🔗 **Подсказка: Выстраивайте сложные mapped виды: сначала простое, затем модификации.**

Ответ:

- тип типа с { [K in keyof T]: T[K] | null }
- убрать null для ключей, например через условный тип и mapping
- сделать все readonly
- применить optional к свойствам

🔗 Шаг 08:

Миссия 8

Настя хочет mapped тип, который фильтрует свойства: оставить только те, которые типом string. Её план: использование conditional type внутри mapped, затем тест. Расположите шаги:

1. определить generic тип type OnlyStrings = { [K in keyof T]: T[K] extends string ? T[K] : never }
2. применить OnlyStrings к примеру интерфейса
3. написать тест, что свойства не-строковых стали never
4. использовать keyof + mapped to проверить ключи с never

🔗 **Подсказка: Условные типы внутри mapped позволяют фильтровать.**

Ответ:

- определить generic тип type OnlyStrings = { [K in keyof T]: T[K] extends string ? T[K] : never }
- применить OnlyStrings к примеру интерфейса
- написать тест, что свойства не-строковых стали never
- использовать keyof + mapped to проверить ключи с never

🔗 Шаг 09:

Миссия 9

Виталий хочет создать mapped тип, который переименует ключи, добавив префикс "old_" перед каждым, и делает все свойства readonly. Шаги: ключи → as с template literal → readonly модификатор → тест. Расположите шаги:

1. определить generic mapped тип с [K in keyof T as `old\_\${string & K}`]
2. добавить readonly перед каждым свойством
3. описать интерфейс T с несколькими свойствами
4. написать тест, что новый тип имеет ключи с "old_" и readonly свойства

🔗 **Подсказка: Префикс + readonly — комбинируются; тест должен проверять оба.**

Ответ:

- определить generic mapped тип с [K in keyof T as `old\_\${string & K}`]

- добавить readonly перед каждым свойством
- описать интерфейс T с несколькими свойствами
- написать тест, что новый тип имеет ключи с "old_" и readonly свойства

🔧 Шаг 10:

Миссия 10

Ольга хочет комбинировать несколько mapped трансформаций: optional + nullable + uppercase ключи + make all values Promise. Её порядок: uppercase ключей, values в Promise, optional + nullable, тест. Расположите шаги:

1. создать mapped с [K in keyof T as Uppercase]
2. значения сделать Promise
3. поля сделать optional и nullable
4. написать тест, что все трансформации применены

🔧 Подсказка: Ключи → значения → модификаторы → проверка.

Ответ:

- создать mapped с [K in keyof T as Uppercase]
- значения сделать Promise
- поля сделать optional и nullable
- написать тест, что все трансформации применены

🔧 Шаг 11:

Миссия 11

Антон хочет mapped тип, который убирает все свойства, тип которых не соответствует string, посредством conditional types внутри mapped. Процесс: описать тип, пример, тест, использование. Расположите шаги:

1. написать type FilterStringProps = { [K in keyof T]: T[K] extends string ? T[K] : never }
2. создать интерфейс с mix типов (string, number, boolean)
3. написать тест, что только строковые свойства остаются не never
4. использовать результат типа в функции с generic

🔧 Подсказка: Условный тип внутри mapped useful для фильтрации.

Ответ:

- написать type FilterStringProps = { [K in keyof T]: T[K] extends string ? T[K] : never }
- создать интерфейс с mix типов (string, number, boolean)
- написать тест, что только строковые свойства остаются не never
- использовать результат типа в функции с generic

🔧 Шаг 12:

Миссия 12

Лёша хочет всё вместе: переименование ключей, optional/nullable, фильтрация по типу, readonly, и ещё values превращаются в функции. Шаги: описать тип, mapped с as + фильтрация, readonly + optional + nullable, применение функций-значений, тестирование. Расположите шаги:

1. описать generic утилиту type ComplexMapped = { ... }
2. внутри mapped использовать as для переименования ключей и conditional для фильтрации по типу

3. назначить модификаторы `readonly` и `optional` и добавить `| null`
4. обратить значения в функции, например `() => T[K]`
5. написать тест, проверяющий все характеристики результата типа

💡 **Подсказка: Композиция `mapped` трансформаций — комбинация `as`, `conditional`, модификаторов, и проверка.**

Ответ:

- описать generic утилиту `type ComplexMapped = { ... }`
- внутри `mapped` использовать `as` для переименования ключей и `conditional` для фильтрации по типу
- назначить модификаторы `readonly` и `optional` и добавить `| null`
- обратить значения в функции, например `() => T[K]`
- написать тест, проверяющий все характеристики результата типа

📖 Урок 3 - Условные типы (`extends`)

💡 **Шаг 01:**

Миссия 1

Аня объявила тип `type IsString = T extends string ? true : false`. Что будет типом `IsString` ?

Подсказка: число не является строкой — ветка `false`.

Ответ:

- **false**
- true
- string
- never

💡 **Шаг 02:**

Миссия 2

Борис хочет создать утилиту `type ElementType = T extends Array ? U : T`. Что даст `ElementType` ?

Подсказка: если `T` — массив, то берётся внутренний тип через `infer`.

Ответ:

- **string**
- string[]
- never
- unknown

💡 **Шаг 03:**

Миссия 3

Юлия пишет `type NonNullableString = T extends string ? T : never`. Что будет типом `NonNullableString` ?

Подсказка: условные типы распространяются на каждый элемент `union`.

Ответ:

- **string**
- string | number
- never
- number

🔧 Шаг 04:

Миссия 4

Максим делает тип `type Flatten = T extends any[] ? T[number] : T`. Какой тип будет у `Flatten` ?

Подсказка: индекс `[number]` даёт тип элементов массива.

Ответ:

- **number**
- `number[]`
- Flatten itself
- never

🔧 Шаг 05:

Миссия 5

Сергей создаёт тип `type IsFunction = T extends (...args: any[]) => any ? true : false`. Что будет `IsFunction<() => void>` ?

Подсказка: проверяется, является ли `T` функцией.

Ответ:

- **true**
- false
- never
- void

🔧 Шаг 06:

Миссия 6

📄 Сопоставьте условные типы с их поведением:

Условные типы:

Ответ:

- `type OnlyStrings = T extends string ? T : never`
- `type NotString = T extends string ? never : T`
- `type MaybeArray = T extends any[] ? T[number] : T`
- `type IsFunction = T extends (...args: any[]) => any ? true : false`

Поведение:

1. оставляет только строки из `union`
2. удаляет строки из `union`, оставляя всё недлинное
3. расплющивает массивы в элемент либо возвращает сам тип если не массив
4. определяет, является ли тип функцией

🔧 **Подсказка: Подумайте, что делает ветка `true` и ветка `false` в каждом условном типе.**

- **`type OnlyStrings = T extends string ? T : never`** — оставляет только строки из `union`
- **`type NotString = T extends string ? never : T`** — удаляет строки из `union`, оставляя всё недлинное
- **`type MaybeArray = T extends any[] ? T[number] : T`** — расплющивает массивы в элемент либо возвращает сам тип если не массив
- **`type IsFunction = T extends (...args: any[]) => any ? true : false`** — определяет, является ли тип функцией

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте примеры с тем, что условные типы делают:

Примеры:

Ответ:

- Flatten
- Extract
- Exclude
- ReturnType

Описание:

1. **если T** — массив, получить элемент, иначе T
2. избрать из T те члены, которые совместимы с U
3. удалить из T те члены, которые совместимы с U
4. получить тип возвращаемого значения функции

🔗 **Подсказка: Extract и Exclude — стандартные conditional-утилиты; ReturnType тоже использует infer.**

- **Flatten** — если T — массив, получить элемент, иначе T
- **Extract** — избрать из T те члены, которые совместимы с U
- **Exclude** — удалить из T те члены, которые совместимы с U
- **ReturnType** — получить тип возвращаемого значения функции

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте условные типы с результатами, когда T = string | number | boolean:

Типы:

Ответ:

- Extract
- Exclude
- NonNullable
- type ToString = T extends string ? T : never

Результаты:

1. number
2. string | number
3. string | number | boolean (если нет null/undefined)
4. string

🔗 **Подсказка: Как Extract и Exclude работают с union, а как NonNullable убирает null/undefined.**

- **Extract** — number
- **Exclude** — string | number
- **NonNullable** — string | number | boolean (если нет null/undefined)
- **type ToString = T extends string ? T : never** — string

🔗 Шаг 09:

Миссия 9

Ольга хочет построить тип, который проверяет, является ли тип функции или нет, и вернуть соответствующий текст, затем протестировать оба случая. Расположите шаги:

1. объявить `type IsFuncOrNot = T extends (...args: any[]) => any ? "func" : "not"`
2. применить к типу функции, например `() => void`
3. применить к не-функции, например `string`
4. **написать тест, что первый даёт "func", второй — "not"**

🔗 Подсказка: Сначала объявление, потом применение на разных типах, потом проверка.

Ответ:

- объявить `type IsFuncOrNot = T extends (...args: any[]) => any ? "func" : "not"`
- применить к типу функции, например `() => void`
- применить к не-функции, например `string`
- **написать тест, что первый даёт "func", второй — "not"**

🔗 Шаг 10:

Миссия 10

Антон хочет утилиту `type MaybePromise = T extends Promise ? Awaited : T`. Его шаги: объявить тип, затем пример с `Promise`, пример без, затем тесты. Расположите шаги:

1. объявить `type MaybePromise = T extends Promise ? Awaited : T`
2. использовать тип с `Promise`
3. использовать тип с `number`
4. **написать тесты, что первый даёт `string`, второй — `number`**

🔗 Подсказка: Проверьте оба случая: когда условие `true` и `false`.

Ответ:

- объявить `type MaybePromise = T extends Promise ? Awaited : T`
- использовать тип с `Promise`
- использовать тип с `number`
- **написать тесты, что первый даёт `string`, второй — `number`**

🔗 Шаг 11:

Миссия 11

Катя хочет тип `type ResponseType = T extends { status: "ok" } ? { data: any } : { error: string }`. Её сценарий: сначала объявить тип, потом пример где `status: "ok"`, потом где другой статус, потом тест. Расположите шаги:

1. объявить `type ResponseType = T extends { status: "ok" } ? { data: any } : { error: string }`
2. применить к `{ status: "ok", data: { x: number } }`
3. применить к `{ status: "error", message: "oops" }`
4. **написать тесты, что первый соответствует `{ data: any }`, второй — `{ error: string }`**

🔗 Подсказка: Condition проверяет поле `status`; `true`-ветка — `data`, `false` — `error`.

Ответ:

- объявить `type ResponseType = T extends { status: "ok" } ? { data: any } : { error: string }`

- применить к `{ status: \"ok\", data: { x: number } }`
- применить к `{ status: \"error\", message: \"oops\" }`
- **написать тесты, что первый соответствует `{ data: any }`, второй — `{ error: string }`**

🔗 Шаг 12:

Миссия 12

Настя хочет тип `type IfStringElseNumber = T extends string ? string : number`, и применить его к `union`. Порядок: объявление, применить к `string`, затем к `number`, затем к `union`, тесты. Расположите шаги:

1. объявить `type IfStringElseNumber = T extends string ? string : number`
2. применить к `"hello"`
3. применить к `42`
4. применить к `string | number` и проверить результат

🔗 Подсказка: **Union распределяется: для `string` ветка `true`, для `number` — `false`.**

Ответ:

- объявить `type IfStringElseNumber = T extends string ? string : number`
- применить к `"hello"`
- применить к `42`
- применить к `string | number` и проверить результат

🔗 Шаг 13:

Миссия 13

Виталий хочет сделать тип `type FilterPromise = T extends Promise ? U : T`, и применить к нескольким вариантам. Порядок: объявление, `Promise`, `number`, `Promise`, тесты.

1. объявить `type FilterPromise = T extends Promise ? U : T`
2. применить к `Promise`
3. применить к `number`
4. применить к `Promise`
5. написать тесты, что результат для `Promise` дает вложенный тип, иначе сам тип

🔗 Подсказка: **Условие проверяется: если `Promise`, вытаскиваем внутренний тип, иначе — оставить как есть.**

Ответ:

- объявить `type FilterPromise = T extends Promise ? U : T`
- применить к `Promise`
- применить к `number`
- применить к `Promise`
- написать тесты, что результат для `Promise` дает вложенный тип, иначе сам тип

🔗 Шаг 14:

Миссия 14

Оля хочет комбинировать условные типы: сначала проверить, является ли тип `null` или `undefined`, если да — `never`, иначе оставить тип. Потом применить `Extract`, потом тесты. Расположите:

1. объявить `type NonNullableCustom = T extends null | undefined ? never : T`
2. применить к типу `string | null | undefined`
3. применить к `number`

4. написать тесты, что первый даёт string, второй — number

🔗 **Подсказка:** Условие проверяет «null | undefined» верхний предел; простые случаи тоже важны.

Ответ:

- объявить type NonNullableCustom = T extends null | undefined ? never : T
- применить к типу string | null | undefined
- применить к number
- **написать тесты, что первый даёт string, второй — number**

🔗 **Шаг 15:**

Миссия 15

Лёша хочет мощный условный тип: если T — массив, возвращать массив из Flatten элементов; иначе, если T — функция, возвращать её возвращаемый тип; в остальных случаях — never. Ему нужно: объявить тип, проверить массив, проверить функцию, проверить другое, написать тесты. Расположите шаги:

1. объявить type DeepType = T extends any[] ? DeepFlatten : T extends (...args: any[]) => infer R ? R : never
2. применить к типу массива, например string[][]
3. применить к функции, например () => boolean
4. применить к простому типу, например number
5. написать тесты, что каждое даёт ожидаемый результат

🔗 **Подсказка:** Условные ветки можно вложить; ветка true — проверка, ветка false может быть новым условным.

Ответ:

- объявить type DeepType = T extends any[] ? DeepFlatten : T extends (...args: any[]) => infer R ? R : never
- применить к типу массива, например string[][]
- применить к функции, например () => boolean
- применить к простому типу, например number
- написать тесты, что каждое даёт ожидаемый результат

📖 Урок 4 - Инференция и шаблонные литералы

🔗 **Шаг 01:**

Миссия 1

Аня нашла строковый literal type type Greeting = `Hello \${string}`. Что будет типом Greeting ?

Подсказка: шаблон литерал с \${string} даёт все строки, начинающиеся с "Hello ".

Ответ:

- **"Hello " + any string**
- exactly "Hello \${string}"
- just "Hello"
- never

🔗 **Шаг 02:**

Миссия 2

Борис пишет тип type PrefixId = `id\_\${T}`. Что будет PrefixId<"123"> ?

Подсказка: подставьте "123" вместо generic параметра.

Ответ:

- **"id_123"**
- PrefixId<"123">
- "123"
- never

Шаг 03:

Миссия 3

Сергей использует infer внутри шаблонного литерала: `type ExtractName = S extends `name:${infer Rest}` ? Rest : never`. Что выдаст `ExtractName<"name:John">?`

Подсказка: условие с шаблоном ``name:${...}`` вытягивает всё после "name:".

Ответ:

- **"John"**
- never
- "name:"
- Rest

Шаг 04:

Миссия 4

Юлия хочет тип `type ToUpperCasePrefix = S extends `${infer First}_${infer Rest}` ? `${Uppercase}_${Rest}` : S`. Что будет `ToUpperCasePrefix<"hello_world">?`

Подсказка: First — часть до "_", Rest — после; Uppercase влияет только на первую часть.

Ответ:

- **"HELLO_world"**
- "hello_world"
- "HELLO_WORLD"
- never

Шаг 05:

Миссия 5

Максим объявляет тип `type EventName = `${K}Changed``. Что будет `EventName<"age">? ?`

Подсказка: просто добавляется "Changed" в конец литерала.

Ответ:

- **"ageChanged"**
- "Changedage"
- age
- never

Шаг 06:

Миссия 6

 Сопоставьте определения шаблонов с примерами типов:

Описания:

Ответ:

- тип, добавляющий префикс через шаблонный литерал
- тип, извлекающий часть строки после двоеточия
- тип, меняющий регистр первой буквы до подчёркивания
- тип, добавляющий суффикс "Changed"

Примеры:

1. `type WithPrefix = `pre${S}``
2. `type ExtractName = S extends `name:${infer Rest}` ? Rest : never`
3. `type ToUpperCasePrefix = S extends `${infer First}_${infer Rest}` ? `${Uppercase}${Rest}` : S`
4. `type EventName = `${K}Changed``

🔗 **Подсказка:** Смотрите на `template literal` с `infer`, префиксы, суффиксы, и манипуляции с частью строки.

- **тип, добавляющий префикс через шаблонный литерал** — `type WithPrefix = `pre${S}``
- **тип, извлекающий часть строки после двоеточия** — `type ExtractName = S extends `name:${infer Rest}` ? Rest : never`
- **тип, меняющий регистр первой буквы до подчёркивания** — `type ToUpperCasePrefix = S extends `${infer First}_${infer Rest}` ? `${Uppercase}${Rest}` : S`
- **тип, добавляющий суффикс "Changed"** — `type EventName = `${K}Changed``

🔗 Шаг 07:

Миссия 7

Ольга хочет узнать, как выстроить инференцию с шаблонами: сначала базовый тип строки, потом шаблон, затем `infer`, затем тест. Расположите шаги:

1. объявить `generic` тип, например `type WithPrefix = `pre${S}``
2. затем шаблонный литерал с `${S}`
3. использовать `infer` в шаблоне для извлечения части
4. написать тест, что правильная часть вытягивается

🔗 **Подсказка:** `Inference` через шаблон требует шаблонного литерала с `infer`, затем проверка.

Ответ:

- объявить `generic` тип, например `type WithPrefix = `pre${S}``
- затем шаблонный литерал с `${S}`
- использовать `infer` в шаблоне для извлечения части
- написать тест, что правильная часть вытягивается

🔗 Шаг 08:

Миссия 8

Иван планирует более сложную инференцию: извлекать слово до точки. Его план: объявить тип, шаблон с `${infer P}.${infer Q}`, применить к строкам с точкой, применить к строкам без точки, протестировать. Расположите шаги:

1. объявить `type BeforeDot = S extends `${infer P}.${infer Q}` ? P : S`
2. применить к `"hello.world"`
3. применить к `"noworld"`
4. **написать тесты, что первый даёт "hello", второй — "noworld"**

🔗 **Подсказка:** Шаблон с точкой — разбивает; если нет точки — возвращает исход.

Ответ:

- объявить `type BeforeDot = S extends `${infer P}.${infer Q}` ? P : S`
- применить к `"hello.world"`
- применить к `"noworld"`
- **написать тесты, что первый даёт "hello", второй — "noworld"**

🔗 **Шаг 09:**

Миссия 9

Катя хочет составить тип, который разбивает строку вида `"key1_key2_key3"` и вытягивает последний сегмент. План: объявить шаблон с три `infer`, затем применить к строке с тремя сегментами, затем к строке с двумя сегментами, потом тесты. Расположите:

1. объявить `type LastSegment = S extends `${infer A}_${infer B}_${infer C}` ? C : S`
2. применить к `"one_two_three"`
3. применить к `"hello_world"`
4. **написать тесты, что первый даёт "three", второй — "hello_world"**

🔗 **Подсказка:** `infer` работает только если структура соответствует шаблону; иначе — исходный.

Ответ:

- объявить `type LastSegment = S extends `${infer A}_${infer B}_${infer C}` ? C : S`
- применить к `"one_two_three"`
- применить к `"hello_world"`
- **написать тесты, что первый даёт "three", второй — "hello_world"**

🔗 **Шаг 10:**

Миссия 10

Дима хочет тип, который проверяет префикс, например строки, начинающиеся с `"on"` + что-то, вытягивает все после `"on"`. Шаги: объявление, шаблон с `infer`, пример с `prefix`, пример без, тесты.

1. объявить `type AfterOn = S extends `on${infer Rest}` ? Rest : never`
2. применить к `"onclick"`
3. применить к `"off"`
4. **написать тесты, что первый даёт "click", второй — never**

🔗 **Подсказка:** Префикс `matching` + `infer` вытягивает `Rest` только если префикс совпадает.

Ответ:

- объявить `type AfterOn = S extends `on${infer Rest}` ? Rest : never`
- применить к `"onclick"`
- применить к `"off"`
- **написать тесты, что первый даёт "click", второй — never**

🔗 **Шаг 11:**

Миссия 11

Лена хочет утилиту `type ExtractDomain = S extends `${infer Sub}@${infer Domain}` ? Domain : never`, и проверить: сначала email, потом строка без @, потом тесты. Расположите шаги:

1. объявить `type ExtractDomain = S extends `${infer Sub}@${infer Domain}` ? Domain : never`
2. применить к `"user@example.com"`
3. применить к `"noatsymbol"`
4. **написать тесты, что первый даёт "example.com", второй — never**

🔗 **Подсказка: Инференция между @, всё до @ — Sub, всё после — Domain.**

Ответ:

- объявить `type ExtractDomain = S extends `${infer Sub}@${infer Domain}` ? Domain : never`
- применить к `"user@example.com"`
- применить к `"noatsymbol"`
- **написать тесты, что первый даёт "example.com", второй — never**

🔗 **Шаг 12:**

Миссия 12

Паша хочет тип, работающий с путями: строки вроде `"src/file.ts"`, извлечь расширение. Шаги: шаблон с ``${infer Ext}``, пример с файлом, пример без расширения, тесты.

1. объявить `type GetExtension = S extends `${infer Name}.${infer Ext}` ? Ext : never`
2. применить к `"index.ts"`
3. применить к `"file"`
4. **написать тесты, что первый даёт "ts", второй — never**

🔗 **Подсказка: Шаблон с точкой разбивает имя и расширение.**

Ответ:

- объявить `type GetExtension = S extends `${infer Name}.${infer Ext}` ? Ext : never`
- применить к `"index.ts"`
- применить к `"file"`
- **написать тесты, что первый даёт "ts", второй — never**

🔗 **Шаг 13:**

Миссия 13

Саша хочет тип, который проверяет, заканчивается ли строка на `"_id"` и вытягивает часть до него, или возвращает `entire string`. Шаги: описать шаблон с ``${infer P}_id``, потом применить к `"user_id"`, к `"userid"`, тест.

1. объявить `type StripId = S extends `${infer P}_id` ? P : S`
2. применить к `"order_id"`
3. применить к `"orderid"`
4. **написать тесты, что первый даёт "order", второй — "orderid"**

🔗 **Подсказка: Suffix matching с шаблоном; если суффикс не совпадёт — вернуть исход.**

Ответ:

- объявить `type StripId = S extends `${infer P}_id` ? P : S`
- применить к `"order_id"`
- применить к `"orderid"`
- **написать тесты, что первый даёт "order", второй — "orderid"**

🔮 Шаг 14:

Миссия 14

Денис хочет тип, который из строк вида "env:PROD" | "env:DEV" вытаскивает суффикс после ":" и делает всё lowercase: пример, объявление, тесты. Расположите:

1. объявить type AfterColonLower = S extends `\${infer L}:\${infer R}` ? Lowercase : never
2. создать union type Env = "env:PROD" | "env:DEV"
3. применить AfterColonLower
4. написать тест, что получаем "prod" | "dev"

🔮 Подсказка: Lowercase влияет на R; шаблон с ":" разделяет; union → каждый вариант.

Ответ:

- объявить type AfterColonLower = S extends `\${infer L}:\${infer R}` ? Lowercase : never
- создать union type Env = "env:PROD" | "env:DEV"
- применить AfterColonLower
- написать тест, что получаем "prod" | "dev"

📖 Урок 5 - infer и satisfies

🔮 Шаг 01:

Миссия 1

Программист Алекс внедряет условные типы с оператором infer . Что делает infer в TypeScript?

Подсказка: infer помогает вытянуть тип из другой конструкции.

Ответ:

- Позволяет вывести тип внутри условного типа
- Создаёт новый тип с точно заданными значениями
- Определяет функцию с конкретным типом параметров
- Автоматически преобразует типы к строкам

🔮 Шаг 02:

Миссия 2

📖 Сопоставьте примеры использования infer с их назначением.

Примеры:

Ответ:

- Извлечение типа аргумента функции
- Выделение типа возвращаемого значения
- Получение типа элемента массива
- Определение типа промиса

Назначения:

1. Тип, который функция возвращает после вызова
2. Тип, который передаётся функции при вызове
3. Тип, который хранится в массиве
4. Тип, который промис оборачивает внутри себя

🔮 Подсказка:

Обратите внимание, как infer «вынимает» тип из сложных структур.

- **Извлечение типа аргумента функции** — Тип, который передаётся функции при вызове
- **Выделение типа возвращаемого значения** — Тип, который функция возвращает после вызова
- **Получение типа элемента массива** — Тип, который хранится в массиве
- **Определение типа промиса** — Тип, который промис оборачивает внутри себя

🔗 Шаг 03:

Миссия 3

Разработчица Ирина использует оператор `satisfies` для проверки объекта на соответствие интерфейсу. В чем преимущество `satisfies` перед обычной проверкой типов через `as` ?

Подсказка: `satisfies` гарантирует корректность типов без потери информации.

Ответ:

- **Сохраняет конкретные типы, при этом проверяя соответствие интерфейсу**
- Приводит тип к интерфейсу без проверки
- Автоматически преобразует типы к `any`
- Удаляет все дополнительные свойства объекта

🔗 Шаг 04:

Миссия 4

Сопоставьте конструкции с описанием, как работает `satisfies` :

Ответ:

- Проверка структуры объекта
 - Сохранение конкретных типов в объекте
 - Отказ от приведения типов
 - Обеспечение безопасности типов
1. Гарантирует, что объект соответствует интерфейсу без искажения типов
 2. Позволяет компилятору проверить тип, не меняя конкретные свойства
 3. Не приводит тип объекта к интерфейсу, а лишь проверяет
 4. Помогает избежать ошибок во время компиляции

🔗 Подсказка:

Помните, что `satisfies` — это проверка, а не преобразование.

- **Проверка структуры объекта** — Гарантирует, что объект соответствует интерфейсу без искажения типов
- **Сохранение конкретных типов в объекте** — Позволяет компилятору проверить тип, не меняя конкретные свойства
- **Отказ от приведения типов** — Не приводит тип объекта к интерфейсу, а лишь проверяет
- **Обеспечение безопасности типов** — Помогает избежать ошибок во время компиляции

🔗 Шаг 05:

Миссия 5

Отладчик Павел пытается понять порядок работы условных типов с `infer` и `satisfies` .

Расположите этапы применения этих операторов в правильной последовательности:

1. Проверка объекта на соответствие типу с помощью `satisfies`
2. Определение типа внутри условного типа с помощью `infer`

3. Извлечение нужного подтипа из сложной структуры
4. Использование результата типа в функции или классе

💡 Подсказка:

Подумайте, что сначала — проверка соответствия, а что — извлечение внутри типа.

Ответ:

- Проверка объекта на соответствие типу с помощью `satisfies`
- Определение типа внутри условного типа с помощью `infer`
- Извлечение нужного подтипа из сложной структуры
- Использование результата типа в функции или классе

💡 Шаг 06:

Миссия 6

Дмитрий пишет утилиту, которая должна извлекать типы параметров функции через `infer`. Какой тип параметра он сможет получить с помощью `infer`?

Подсказка: Внимательно посмотрите, как в условных типах можно выделить типы аргументов.

Ответ:

- **Тип первого аргумента функции**
- Тип возвращаемого значения функции
- Тип имени функции
- Тип объекта, содержащего функцию

💡 Шаг 07:

Миссия 7

🔗 Сопоставьте оператор `infer` с его типичным применением в примерах:

Ответ:

- Извлечение типа промиса
 - Получение типа элементов массива
 - Выделение типа аргумента функции
 - Определение возвращаемого типа функции
1. Тип, который оборачивает `Promise`
 2. Тип, который содержится в массиве
 3. Тип, передаваемый функции
 4. Тип, который функция возвращает

💡 Подсказка:

Подумайте, что `infer` позволяет «вытягивать» типы из разных шаблонов.

- **Извлечение типа промиса** — Тип, который оборачивает `Promise`
- **Получение типа элементов массива** — Тип, который содержится в массиве
- **Выделение типа аргумента функции** — Тип, передаваемый функции
- **Определение возвращаемого типа функции** — Тип, который функция возвращает

💡 Шаг 08:

Миссия 8

Анна использует оператор `satisfies`, чтобы гарантировать соответствие объекта интерфейсу, но сохранить более конкретные типы. Что произойдет, если объект не соответствует интерфейсу?

Подсказка: Подумайте, как TypeScript реагирует на несоответствие с помощью `satisfies` .

Ответ:

- **Компилятор выдаст ошибку**
- Объект будет приведён к интерфейсу без ошибок
- Программа выполнится с предупреждением
- **Ничего не произойдет** — это только для документации

🔗 Шаг 09:

Миссия 9

🧐 Сопоставьте описания с ключевыми особенностями оператора `satisfies` :

Ответ:

- Сохраняет детали типов
 - Проверяет соответствие типу
 - Не приводит тип объекта
 - Выявляет ошибки во время компиляции
1. Оператор сохраняет конкретные типы, а не упрощает до интерфейса
 2. Компилятор проверяет, что объект удовлетворяет интерфейсу
 3. Тип объекта остается без преобразования
 4. Ошибки несоответствия показываются на этапе компиляции

🔗 Подсказка:

Помните, `satisfies` — именно проверка, а не преобразование.

- **Сохраняет детали типов** — Оператор сохраняет конкретные типы, а не упрощает до интерфейса
- **Проверяет соответствие типу** — Компилятор проверяет, что объект удовлетворяет интерфейсу
- **Не приводит тип объекта** — Тип объекта остается без преобразования
- **Выявляет ошибки во время компиляции** — Ошибки несоответствия показываются на этапе компиляции

🔗 Шаг 10:

Миссия 10

Сергей создает сложный условный тип с `infer` и проверяет объекты с `satisfies` . Расположите этапы процесса в правильном порядке:

1. Создание условного типа с `infer` для извлечения типа
2. Проверка объекта с помощью `satisfies` на соответствие типу
3. Использование извлечённого типа в функции
4. Отладка и исправление ошибок компиляции

🔗 Подсказка:

Проанализируйте сначала определение, потом проверку, затем использование и исправление ошибок.

Ответ:

- Создание условного типа с `infer` для извлечения типа
- Проверка объекта с помощью `satisfies` на соответствие типу
- Использование извлечённого типа в функции

- Отладка и исправление ошибок компиляции

Урок 6 - Типизация ошибок

Шаг 01:

Миссия 1

Анна пишет функцию, которая получает от сервера данные, но не уверена, что в объекте всегда будет свойство `age`. Какой тип будет безопаснее всего указать для этого свойства в интерфейсе?

Подсказка: Подумайте, что делать, если значение отсутствует или `undefined`.

Ответ:

- **`age?: number`**
- `age: number`
- `age: string`
- `age: any`

Шаг 02:

Миссия 2

Борис создал тип функции: `(input: string) => number`, но иногда функция может выбрасывать ошибку и не возвращать число. Какой тип нужно использовать, чтобы явно указать, что функция может не вернуть значение?

Подсказка: Одна из возможностей TypeScript позволяет указать возвращаемый тип, равный либо числу, либо чему-то другому.

Ответ:

- **`(input: string) => number | undefined`**
- `(input: string) => any`
- `(input: string) => never`
- `(input: string) => void`

Шаг 03:

Миссия 3

Катя хочет быть уверена, что никогда не передаст `null` значением параметра функции, но TypeScript её код допускает `null`. Какой строгий режим или оператор поможет избежать `null` при вызове параметра?

Подсказка: Есть оператор, ставится после имени переменной.

Ответ:

- **non-null assertion operator: `param!`**
- `null as any`
- `strictNullChecks: false`
- using any type

Шаг 04:

Миссия 4

Давид ловит ошибки в `try/catch`, но переменная ошибки по умолчанию имеет тип `any`. Как указать, что ошибка — это объект `Error` или его потомок?

Подсказка: Нужно сузить тип внутри блока `catch`.

Ответ:

- **catch (e: unknown) then check instanceof Error**
- catch (e: Error) regardless
- catch (e: any)
- catch (e: object)

 **Шаг 05:**

Миссия 5

Елена возвращает из API список пользователей, но иногда сервер возвращает ошибку вида строки. Какой тип лучше использовать для результата функции: список пользователей или строка ошибки?

Подсказка: Рассмотрите использование union-технологий.

Ответ:

- **User[] | string**
- User[]
- string
- never

 **Шаг 06:**

Миссия 6

Игорь хочет, чтобы компилятор предупреждал об ошибках типов при обращении к свойствам объекта, которых нет в интерфейсе. Какой флаг TypeScript можно включить?

Подсказка: Этот флаг связан с проверкой лишних свойств при присваивании литерала.

Ответ:

- **"strict" или "noImplicitAny" и "noUncheckedIndexedAccess"**
- allowJs
- skipLibCheck
- isolatedModules

 **Шаг 07:**

Миссия 7

✂ Сопоставьте понятия с примерами ошибок типов.

Понятия:

Ответ:

- Type 'string' is not assignable to type 'number'
- Cannot assign to 'foo' because it is a constant or readonly
- Object is possibly 'null' or 'undefined'
- Property 'bar' does not exist on type 'Baz'

Объяснения:

1. Вы пытаетесь использовать свойство, которого нет в объявленном типе объекта.
2. Вы присваиваете значение неправильного базового типа.
3. Попытка изменить readonly или const значение.
4. Нужно проверить, что значение не null или undefined перед использованием.

Подсказка: Подумайте, какой тип проверяется в каждом сообщении об ошибке — базовый тип, наличие свойства, readonly, или nullable.

- **Type 'string' is not assignable to type 'number'** — Вы присваиваете значение неправильного базового типа.
- **Cannot assign to 'foo' because it is a constant or readonly** — Попытка изменить readonly или const значение.
- **Object is possibly 'null' or 'undefined'** — Нужно проверить, что значение не null или undefined перед использованием.
- **Property 'bar' does not exist on type 'Baz'** — Вы пытаетесь использовать свойство, которого нет в объявленном типе объекта.

Шаг 08:

Миссия 8

Сопоставьте тип ошибки с ситуацией, когда она возникает.

Типы ошибок:

Ответ:

- Excess property error
- Union type mismatch
- Nullable property access error
- Readonly violation

Ситуации:

1. Объект литерал содержит ключ, которого нет в интерфейсе.
2. Переменная union-типа получает значение, не входящее в union.
3. Попытка прочесть свойство на потенциально null значении.
4. Попытка присвоить значение свойству, объявленному как readonly.

Подсказка: Сравните причину ошибки: лишнее свойство vs несовпадающий union vs readonly vs nullable.

- **Excess property error** — Объект литерал содержит ключ, которого нет в интерфейсе.
- **Union type mismatch** — Переменная union-типа получает значение, не входящее в union.
- **Nullable property access error** — Попытка прочесть свойство на потенциально null значении.
- **Readonly violation** — Попытка присвоить значение свойству, объявленному как readonly.

Шаг 09:

Миссия 9

 Сопоставьте сообщения компилятора с нужной стратегией исправления.

Сообщения:

Ответ:

- Argument of type 'undefined' is not assignable to parameter of type 'string'
- Property 'length' does not exist on type 'number'
- Type 'never' is not assignable to type 'User'
- Type 'any' is assigned to type 'unknown'

Стратегии:

1. Добавить проверку или сделать параметр optional и обрабатывать undefined.

2. Убедиться, что вы работаете со строкой, а не с числом, либо привести тип.
3. Использовать защиту типов или уточнение (type guards).
4. Избегать any, использовать более строгий тип вроде unknown или конкретного интерфейса.

Подсказка: Внимательно читайте, какой тип ожидается и что передаётся.

- **Argument of type 'undefined' is not assignable to parameter of type 'string'** — Добавить проверку или сделать параметр optional и обрабатывать undefined.
- **Property 'length' does not exist on type 'number'** — Убедиться, что вы работаете со строкой, а не с числом, либо привести тип.
- **Type 'never' is not assignable to type 'User'** — Использовать защиту типов или уточнение (type guards).
- **Type 'any' is assigned to type 'unknown'** — Избегать any, использовать более строгий тип вроде unknown или конкретного интерфейса.

🔮 Шаг 10:

Миссия 10

🔍 Сопоставьте кодовую ситуацию с типом ошибки, который произойдёт при компиляции.

Кодовые ситуации:

Ответ:

- `let x: string = 5;`
- `const obj: { a: number } = { a: 1, b: 2 };`
- `function f(s: string | null) { return s.length; }`
- `interface I { readonly id: number }; let i: I; i.id = 3;`

Типы ошибок:

1. Type 'number' is not assignable to type 'string'
2. Excess property error
3. Object is possibly 'null'
4. Cannot assign to 'id' because it is a read-only property

Подсказка: Свяжите каждую строку к сообщению, которое TypeScript выдаёт при такой ошибке.

- **let x: string = 5;** — Type 'number' is not assignable to type 'string'
- **const obj: { a: number } = { a: 1, b: 2 };** — Excess property error
- **function f(s: string | null) { return s.length; }** — Object is possibly 'null'
- **interface I { readonly id: number }; let i: I; i.id = 3;** — Cannot assign to 'id' because it is a read-only property

🔮 Шаг 11:

Миссия 11

⚠️ Сопоставьте типы ошибок с примерами API ответа, когда что-то идёт не так.

Типы ошибок:

Ответ:

- Invalid input error
- Authentication error
- Not found error
- Server error

Примеры ответов:

1. { status: 400, message: "Email is invalid" }
2. { status: 401, message: "Token expired" }
3. { status: 404, message: "User not found" }
4. { status: 500, message: "Internal server error" }

Подсказка: Статусный код даёт подсказку, к какому типу относится ошибка.

- **Invalid input error** — { status: 400, message: "Email is invalid" }
- **Authentication error** — { status: 401, message: "Token expired" }
- **Not found error** — { status: 404, message: "User not found" }
- **Server error** — { status: 500, message: "Internal server error" }

Шаг 12:

Миссия 12

 Сопоставьте утилиты TypeScript с преимуществами их использования при типизации ошибок.

Утилиты:

Ответ:

- unknown
- never
- Readonly
- Partial

Преимции:

1. Означает, что функция никогда не возвращает нормальный результат.
2. Позиционирует значение как Readonly, защищает от изменений.
3. Позволяет описать объект, где поля могут отсутствовать.
4. Не позволяет работать с типом, пока не проверишь его структуру.

Подсказка: Подумайте, когда вы не знаете тип заранее, и когда нужно запретить модификацию.

- **unknown** — Не позволяет работать с типом, пока не проверишь его структуру.
- **never** — Означает, что функция никогда не возвращает нормальный результат.
- **Readonly** — Позиционирует значение как Readonly, защищает от изменений.
- **Partial** — Позволяет описать объект, где поля могут отсутствовать.

Шаг 13:

Миссия 13

Упорядочите шаги корректного использования unknown при обработке ошибки:

1. Провести проверку типа или использовать type guard
2. Получить ошибку как unknown
3. Бросить ошибку или вернуть конкретный тип
4. Работать с результатом внутри гарантированного типа

Подсказка: Сначала принимаем неизвестное, затем проверяем, затем используем.

Ответ:

- Получить ошибку как unknown
- Провести проверку типа или использовать type guard
- Работать с результатом внутри гарантированного типа

- Бросить ошибку или вернуть конкретный тип

🔗 Шаг 14:

Миссия 14

Упорядочите шаги, если вы хотите создать тип ошибки, объединяющий разные виды ошибок (union):

1. Определить все виды ошибок, которые могут произойти
2. Создать union типов для всех ошибок
3. Использовать этот union в возвращаемом типе функции
4. Обрабатывать каждый вариант в коде с помощью switch или type guard

Подсказка: Сначала собрать виды, затем описать, затем применять.

Ответ:

- Определить все виды ошибок, которые могут произойти
- Создать union типов для всех ошибок
- Использовать этот union в возвращаемом типе функции
- Обрабатывать каждый вариант в коде с помощью switch или type guard

🔗 Шаг 15:

Миссия 15

Упорядочите шаги отладки ошибки «Object is possibly 'undefined'»:

1. Посмотреть, где объект может быть undefined
2. Добавить проверку на undefined или использовать optional chaining
3. Сменить тип на non-nullable, если уверены, что не будет undefined
4. Перекомпилировать и убедиться, что ошибка исчезла

Подсказка: Проверьте корень undefined, затем примените средство защиты.

Ответ:

- Посмотреть, где объект может быть undefined
- Добавить проверку на undefined или использовать optional chaining
- Сменить тип на non-nullable, если уверены, что не будет undefined
- Перекомпилировать и убедиться, что ошибка исчезла

🔗 Шаг 16:

Миссия 16

Упорядочите шаги, если вы используете утилиту never для обработки ситуации, когда по логике код не должен быть выполнен:

1. Определить все возможные случаи в union
2. Добавить кандидат на never для невозможного случая
3. Внутри switch/fn бросить ошибку или never в default ветке

Подсказка: Union case —> default/исключение —> never.

Ответ:

- Определить все возможные случаи в union
- Добавить кандидат на never для невозможного случая
- Внутри switch/fn бросить ошибку или never в default ветке

🔮 Шаг 17:

Миссия 17

Упорядочите этапы, как действовать, когда функция может вернуть либо результат, либо ошибку:

1. Задать возврат как ResultType | ErrorType
2. Вызвать функцию и получить union тип
3. Проверить, какой вариант возвращён (например, через discriminated union или instanceof)
4. Обработать ошибку или использовать результат

Подсказка: Тип функции —> вызов —> проверка —> использование.

Ответ:

- Задать возврат как ResultType | ErrorType
- Вызвать функцию и получить union тип
- Проверить, какой вариант возвращён (например, через discriminated union или instanceof)
- Обработать ошибку или использовать результат

📖 Урок 7 - Квест Построить типизированный DSL (например, конфиг API)

🔮 Шаг 01:

Миссия 1

Максим хочет описать простой конфиг API, где нужно указать базовый URL как строку и флаг useSsl как булево значение. Какой интерфейс подойдет?

Подсказка: Интерфейс должен точно отражать ключи и типы, без лишних.

Ответ:

- **interface ApiConfig { baseUrl: string; useSsl: boolean; }**
- type ApiConfig = { baseUrl: string; useSsl: string; }
- interface ApiConfig { url: string; ssl: boolean; }
- type ApiConfig = { baseUrl?: string; useSsl?: boolean; }

🔮 Шаг 02:

Миссия 2

Ольга хочет, чтобы в конфиге API был объект headers, где ключи — строки, а значения — либо строка, либо массив строк. Как описать тип headers?

Подсказка: Используйте индексную сигнатуру и union.

Ответ:

- **headers: { [key: string]: string | string[] }**
- headers: Record
- headers: { [key: string]: any }
- headers: Map

🔮 Шаг 03:

Миссия 3

Сергей хочет, чтобы параметр timeout был опциональным, по умолчанию 5000. Что должно быть в типе конфига?

Подсказка: Optional свойства в интерфейсах/типа union с undefined могут помочь.

Ответ:

- **timeout?: number**
- timeout: number
- timeout: number | null
- timeout: any

🔮 Шаг 04:

Миссия 4

Ирина хочет, чтобы в конфиге можно было указать метод API: GET | POST | PUT | DELETE . Как описать это в типе?

Подсказка: Используйте литеральные типы или union строк.

Ответ:

- **method: "GET" | "POST" | "PUT" | "DELETE"**
- method: string
- method: HttpMethod (enum) without defining enum
- method: any

🔮 Шаг 05:

Миссия 5

Дмитрий хочет, чтобы конфиг API мог быть либо простым объектом с url + method, либо функцией, возвращающей такой объект. Какой тип определить для главного параметра apiConfig ?

Подсказка: Используйте union функций и объектов.

Ответ:

- **apiConfig: ApiConfig | (() => ApiConfig)**
- apiConfig: () => ApiConfig
- apiConfig: ApiConfig[]
- apiConfig: any

🔮 Шаг 06:

Миссия 6

Сопоставьте части конфигурации с их назначением в API-DSL.

Части конфига:

Ответ:

- baseUrl
- method
- headers
- timeout

Назначения:

1. Указывает протокол и домен сервера
2. Определяет способ передачи данных (GET, POST и т.д.)
3. Дополнительные HTTP заголовки, например Content-Type или Authorization
4. Сколько миллисекунд ждать ответа перед ошибкой таймаута

Подсказка: Прочитайте названия: baseUrl — не method, timeout — не headers.

- **baseUrl** — Указывает протокол и домен сервера
- **method** — Определяет способ передачи данных (GET, POST и т.д.)
- **headers** — Дополнительные HTTP заголовки, например Content-Type или Authorization
- **timeout** — Сколько миллисекунд ждать ответа перед ошибкой таймаута

🔗 Шаг 07:

Миссия 7

Сопоставьте ключи в утилите DSL с возможностью конфигурации, которую они предоставляют.

Ключи/утилиты:

Ответ:

- optional()
- default()
- sensitive()
- array()

Возможность конфигурации:

1. Позволяет параметру быть необязательным в конфиге
2. Предоставляет значение по умолчанию, если параметр не передан
3. Помечает, что значение не должно логироваться полностью
4. **Определяет, что значение** — массив элементов указанного типа

Подсказка: Подумайте, как эти утилиты применяются к чтению и валидации конфигов.

- **optional()** — Позволяет параметру быть необязательным в конфиге
- **default()** — Предоставляет значение по умолчанию, если параметр не передан
- **sensitive()** — Помечает, что значение не должно логироваться полностью
- **array()** — Определяет, что значение — массив элементов указанного типа

🔗 Шаг 08:

Миссия 8

Сопоставьте строки к коду TypeScript, который лучше всего описывает DSL-конфиг с reader-комбинатором, где ключ port обязателен, env — строка по умолчанию, debug — булево необязательное.

Строки:

Ответ:

- port: number;
- env?: string; default "production"
- debug?: boolean;
- timeout: string | string[];

Код:

1. port: number;
2. env: string;
3. debug?: boolean;
4. timeout? ... (не сюда)

Подсказка: Строки должны соответствовать свойствам, которые входят в DSL; лишнего — не должно быть.

- **port: number;** — port: number;
- **env?: string; default "production"** — env: string;
- **debug?: boolean;** — debug?: boolean;
- **timeout: string | string[];** — (не часть описыв. DSL)

🔧 Шаг 09:

Миссия 9

Упорядочите шаги, как строится типизированный DSL для конфига API от идеи до использования:

1. Определить все возможные ключи и их типы
2. Решить, какие ключи обязательны, какие опциональны, и указать defaults, если нужно

Ответ:

- Определить все возможные ключи и их типы
- Решить, какие ключи обязательны, какие опциональны, и указать defaults, если нужно
- Создать утилиты или комбинаторы (optional, default и т.д.)
- Использовать тип DSL в реальных функциях, и проверять на соблюдение интерфейса

🔧 Шаг 10:

Миссия 10

Упорядочите шаги, если вы хотите, чтобы конфиг API поддерживал два варианта: объект и функцию, возвращающую объект:

1. Определить тип объекта APIConfig
2. Определить тип функции, возвращающей APIConfig
3. Составить union этих двух типов для главного параметра
4. В коде функции, когда передано значение, отличное от функции, просто его использовать напрямую

Подсказка: Сначала типы, затем union, затем реализацию.

Ответ:

- Определить тип объекта APIConfig
- Определить тип функции, возвращающей APIConfig
- Составить union этих двух типов для главного параметра
- В коде функции, когда передано значение, отличное от функции, просто его использовать напрямую

🔧 Шаг 11:

Миссия 11

Упорядочите шаги для добавления проверки литерального типа метода (GET|POST...), чтобы DSL не позволял другие строки:

1. Определить union литералов для метода
2. Использовать этот тип в интерфейсе
3. При попытке передать другую строку TypeScript выдаёт ошибку
4. Добавить тесты или примеры, которые проверяют, что неправильный метод не компилируется

Подсказка: Литеральные типы → интерфейс → проверка → примеры ошибок.

Ответ:

- Определить union литералов для метода
- Использовать этот тип в интерфейсе
- При попытке передать другую строку TypeScript выдаёт ошибку
- Добавить тесты или примеры, которые проверяют, что неправильный метод не компилируется

🔧 Шаг 12:

Миссия 12

Упорядочите шаги, когда один из ключей конфигурации — массив, и нужно гарантировать, что массив элементов одного типа:

1. Определить тип элемента массива
2. **Определить, что поле** — массив этого типа (например, `string[]`)
3. Указать это поле в интерфейсе или типе DSL

Ответ:

- Определить тип элемента массива
- **Определить, что поле** — массив этого типа (например, `string[]`)
- Указать это поле в интерфейсе или типе DSL
- Использовать его в примерах и убедиться, что несоответствия вызывают ошибки

🔧 Шаг 13:

Миссия 13

Упорядочите шаги, если конфиг DSL должен поддерживать вложенные конфиги для разных API ресурсов (например, `user`, `auth`, `data`):

1. Определить отдельные интерфейсы для ресурсов `user`, `auth`, `data`
2. Объединить их в общий тип, например `Record<"user"|"auth"|"data", ResourceConfig>`
3. Указать, что общий конфиг имеет поле `perResource` с этим `Record`
4. В коде использовать свойство `perResource.user` / `.auth` и т.д., с гарантией типов ресурсных конфигов

Подсказка: Модульность помогает: думаю о ресурсах как о маленьких конфигурациях.

Ответ:

- Определить отдельные интерфейсы для ресурсов `user`, `auth`, `data`
- Объединить их в общий тип, например `Record<"user"|"auth"|"data", ResourceConfig>`
- Указать, что общий конфиг имеет поле `perResource` с этим `Record`
- В коде использовать свойство `perResource.user` / `.auth` и т.д., с гарантией типов ресурсных конфигов

📁 Модуль 6 - Классы и ООП в TypeScript

📖 Урок 1 - Классы, интерфейсы и абстрактные классы

🔧 Шаг 01:

Миссия 1

Алексей пишет класс User , который должен содержать публичное имя name: string и приватный возраст age: number , доступный только через метод getAge() . Как должна выглядеть объявка класса?

Подсказка: Используйте модификаторы доступа public и private , подумайте, где конструктор.

Ответ:

- **class User { public name: string; private age: number; constructor(name: string, age: number) { this.name = name; this.age = age; } getAge(): number { return this.age; } }**
- class User { name: string; age: number; constructor(name: string, age: number) { this.name = name; this.age = age; } }
- class User { public name: string; age: number; constructor(name: string, age: number) { this.name = name; this.age = age; } getAge(): string { return this.age; } }
- class User { private name: string; private age: number; constructor(name: string, age: number) { this.name = name; this.age = age; } getName(): string { return this.name; } }

🔗 Шаг 02:

Миссия 2

Мария создает интерфейс IShape с методом area() , и класс Circle , который должен реализовать этот интерфейс. Как правильно объявить интерфейс и класс?

Подсказка: Класс должен использовать ключевое слово implements и обеспечить реализацию метода.

Ответ:

- **interface IShape { area(): number; } class Circle implements IShape { constructor(public radius: number) {} area(): number { return Math.PI * this.radius * this.radius; } }**
- interface IShape { area(): string; } class Circle implements IShape { constructor(public radius: number) {} area(): number { return Math.PI * this.radius * this.radius; } }
- interface IShape { area(): number; } class Circle { constructor(public radius: number) {} }
- interface IShape { area(): number; } class Circle extends IShape { constructor(public radius: number) {} area(): number { return Math.PI * this.radius * this.radius; } }

🔗 Шаг 03:

Миссия 3

Илья хочет, чтобы класс Logger выступал базовым классом, и чтобы никто не мог его напрямую создать, только расширять. Что нужно использовать?

Подсказка: Подумайте, как сделать так, чтобы класс нельзя было инстанцировать, но дать возможность наследовать.

Ответ:

- **abstract class Logger { abstract log(msg: string): void; }**
- class Logger { constructor() { if (new.target === Logger) throw Error(); } log(msg: string): void {} }
- interface Logger { log(msg: string): void; }
- class Logger { log(msg: string): void {} }

🔗 Шаг 04:

Миссия 4

Олег создал абстрактный класс Vehicle с методом start(): void , и класс Car наследует Vehicle. Что обязательно должна делать Car?

Подсказка: Проверьте, какой тип метода объявлен в абстрактном, и что Car должен с этим сделать.

Ответ:

- **Класс Car должен реализовать метод start(): void**
- Car может не реализовать start(), если не вызывает super()
- Car может оставить start абстрактным
- Car должен переопределить constructor базового класса Vehicle без метода start

Шаг 05:

Миссия 5

Наташа создает интерфейсы IFlyable и ISwimmable , и хочет, чтобы класс Duck мог и летать, и плавать. Как лучше описать Duck?

Подсказка: Классы могут реализовать несколько интерфейсов.

Ответ:

- **class Duck implements IFlyable, ISwimmable { fly(): void { /*...*/ } swim(): void { /*...*/ } }**
- **class Duck extends IFlyable, ISwimmable { fly(): void; swim(): void; }**
- **class Duck implements IFlyable { fly(): void { /*...*/ } }**
- **interface Duck extends IFlyable, ISwimmable { }**

Шаг 06:

Миссия 6 Сопоставьте определения с ключевыми понятиями класса/интерфейса/абстрактного класса в TypeScript. Понятия: Интерфейс Абстрактный класс Класс Наследование Определения: Представляет контракт (сигнатуры), не содержит реализации методов или свойств, только типы. Можно определить методы, часть которых реализована, часть — без реализации; класс нельзя инстанцировать напрямую. Шаблон для создания объектов, может быть инстанцирован, реализует методы и хранит состояние. Механизм, с помощью которого один класс получает свойства и методы другого. Подсказка: Сравните интерфейс и абстрактный класс: что общего, что разного.

Ответ:

- **Интерфейс** — Представляет контракт (сигнатуры), не содержит реализации методов или свойств, только типы.
- **Абстрактный класс** — Можно определить методы, часть которых реализована, часть — без реализации; класс нельзя инстанцировать напрямую.
- **Класс** — Шаблон для создания объектов, может быть инстанцирован, реализует методы и хранит состояние.
- **Наследование** — Механизм, с помощью которого один класс получает свойства и методы другого.

Шаг 07:

Миссия 7

Сопоставьте следующие члены класса с их возможным модификатором доступа в TypeScript.

Члены класса:

Ответ:

- свойство, доступное только внутри класса
- свойство, доступное в классе и в наследниках, но не снаружи

- метод, который всегда публичен по умолчанию
- конструктор, если не указан, будет какой по умолчанию?

Модификаторы:

1. private
2. protected
3. public
4. public (конструктор по умолчанию)

Подсказка: Подумайте о значениях по умолчанию в классах и как работают protected / private.

- **свойство, доступное только внутри класса** — private
- **свойство, доступное в классе и в наследниках, но не снаружи** — protected
- **метод, который всегда публичен по умолчанию** — public
- **конструктор, если не указан, будет какой по умолчанию?** — public (конструктор по умолчанию)

🔗 Шаг 08:

Миссия 8

Сопоставьте примеры с тем, используется ли там абстрактный класс или интерфейс.

Примеры:

Ответ:

- `abstract class Animal { abstract makeSound(): void; move(): void { /* ... */ } }`
- `interface Serializable { serialize(): string; }`
- `class Car implements Serializable { serialize(){ return "..."; } }`
- `abstract class Base implements Serializable { abstract serialize(): string; }`

Что это:

1. Абстрактный класс с конкретным и абстрактным методом
2. **Интерфейс** — только сигнатуры без реализации
3. **Класс** — конкретная реализация интерфейса
4. Абстрактный класс, реализующий интерфейс

Подсказка: Обратите внимание, есть ли реализация метода и можно ли создать экземпляр.

- **`abstract class Animal { abstract makeSound(): void; move(): void { /* ... */ } }`** — Абстрактный класс с конкретным и абстрактным методом
- **`interface Serializable { serialize(): string; }`** — Интерфейс — только сигнатуры без реализации
- **`class Car implements Serializable { serialize(){ return "..."; } }`** — Класс — конкретная реализация интерфейса
- **`abstract class Base implements Serializable { abstract serialize(): string; }`** — Абстрактный класс, реализующий интерфейс

🔗 Шаг 09:

Миссия 9

Сопоставьте сообщения компилятора с ошибками, которые получит Павел, если он сделает следующие нарушения.

Ситуации:

Ответ:

- попытка создать экземпляр абстрактного класса
- класс не реализует абстрактный метод базового класса
- класс объявлен implements интерфейс, но не реализует метод интерфейса
- метод с неправильной сигнатурой при наследовании

Сообщения:

1. Cannot create an instance of an abstract class
2. Non-abstract class must implement inherited abstract method
3. Class incorrectly implements interface: missing method
4. Method signature mismatch: types of parameters or return type differ

Подсказка: Помните, что ошибки связаны с тем, что нарушается контракт интерфейса или абстрактного класса.

- **попытка создать экземпляр абстрактного класса** — Cannot create an instance of an abstract class
- **класс не реализует абстрактный метод базового класса** — Non-abstract class must implement inherited abstract method
- **класс объявлен implements интерфейс, но не реализует метод интерфейса** — Class incorrectly implements interface: missing method
- **метод с неправильной сигнатурой при наследовании** — Method signature mismatch: types of parameters or return type differ

Урок 2 - Наследование, super, композиция

Шаг 01:

Миссия 1

Петя делает класс Animal с конструктором, принимающим имя, и класс Dog , который наследует Animal . Что он должен вызвать первым в конструкторе Dog ?

Подсказка: Подумайте, как работает ключевое слово super в TypeScript.

Ответ:

- **super(name);**
- this.name = name;
- Dog constructor без вызова super
- Не нужно определять конструктор в Dog вообще

Шаг 02:

Миссия 2

Саша создал класс Logger , а затем класс FileLogger , который наследует Logger и хочет расширить метод log() . Как правильно внутри FileLogger.log() вызвать оригинальный log() из класса-родителя?

Подсказка: Используйте super и имя метода.

Ответ:

- **super.log(message);**
- Logger.log(message);
- this.log(message);
- FileLogger.log(message);

🔧 Шаг 03:

Миссия 3

Маша делает композицию: класс Car содержит объект Engine , а не наследует от него. Какой принцип она реализует?

Подсказка: Это когда один объект внутри другого, не через extends.

Ответ:

- **Композиция**
- Наследование
- Интерфейс
- Абстрактный класс

🔧 Шаг 04:

Миссия 4

Вася создал класс Base с защищённым свойством protected value и класс Derived extends Base . Что может сделать Derived с value ?

Подсказка: Подумайте, что означает protected vs private .

Ответ:

- **Обращаться к this.value внутри Derived**
- Обращаться к value из вне через экземпляр Derived
- Сделать this.value = some without super
- Наследовать, но не иметь доступа к value никак

🔧 Шаг 05:

Миссия 5

Денис хочет, чтобы класс Bird мог летать, но не через наследование от FlyingAnimal , а через композицию. Что следует сделать?

Подсказка: Один из способов: вставить объект с поведением полёта вместо extends.

Ответ:

- **Добавить внутри Bird поле flyBehavior: FlyBehavior и вызывать flyBehavior.fly()**
- Наследовать от FlyingAnimal несмотря ни на что
- Переопределить метод fly в Bird без super
- Использовать интерфейс Flyable и наследоваться от Bird

🔧 Шаг 06:

Миссия 6

Сопоставьте сообщения ошибок компилятора с тем, что нарушено.

Сообщения:

Ответ:

- Must call super constructor in derived class before accessing 'this'
- Property 'x' does not exist on type 'Base' when accessed via child instance
- Cannot access private member of Base from Derived
- super.log is not a function if parent has no method log

Нарушения:

1. Конструктор дочернего класса не вызывает super до this

2. Попытка вызвать свойство, которого нет в базовом классе через экземпляр наследника
3. Свойство объявлено `private` в родителе, и пытаются к нему получить доступ в наследнике
4. Вызывается `super.method()`, которого нет в родительском классе

Подсказка: Ошибки связаны с порядком `super`, видимостью, и наличием метода.

- **Must call super constructor in derived class before accessing 'this'** — Конструктор дочернего класса не вызывает `super` до `this`
- **Property 'x' does not exist on type 'Base' when accessed via child instance** — Попытка вызвать свойство, которого нет в базовом классе через экземпляр наследника
- **Cannot access private member of Base from Derived** — Свойство объявлено `private` в родителе, и пытаются к нему получить доступ в наследнике
- **super.log is not a function if parent has no method log** — Вызывается `super.method()`, которого нет в родительском классе

Урок 3 - Приватность `private`, `protected`, `#`

Шаг 01:

Миссия 1

Настя создаёт класс `User`, и хочет, чтобы поле `password` было доступно только внутри этого класса. Какое объявление использовать?

Подсказка: Поле не должно быть видно ни в наследниках, ни извне.

Ответ:

- **`private password: string;`**
- `protected password: string;`
- `public password: string;`
- `#password: string;`

Шаг 02:

Миссия 2

Вася создал класс `Base` с защищённым полем `protected data`. Он хочет, чтобы классы-наследники могли читать и менять `data`, но не снаружи. Какой модификатор выбрать?

Подсказка: Поле должно быть доступно в классах, которые наследуются.

Ответ:

- **`protected data: string;`**
- `private data: string;`
- `public data: string;`
- `#data: string;`

Шаг 03:

Миссия 3

Олег хочет, чтобы приватность проверялась не только при компиляции, но и в рантайме — даже если кто-то попытается обратиться к полю напрямую через скобочную нотацию или через JSON. Какое объявление ему подойдёт?

Подсказка: Один способ обеспечивает «жёсткую» приватность, встроенную в JS.

Ответ:

- **`#secretField: number;`**
- `private secretField: number;`

- protected secretField: number;
- public secretField: number;

🔗 Шаг 04:

Миссия 4

Катя использует приватное поле `private count` и хочет, чтобы её тесты могли проверять значение через экземпляр класса. Что произойдёт, если она попытается обратиться к `instance.count` из тестового файла?

Подсказка: Это проверка типов, а не рантайм приватность.

Ответ:

- **Ошибка компиляции: свойство `private` недоступно извне.**
- Работает нормально, доступ есть.
- Будет `undefined` в рантайме.
- Используется `protected` вместо `private`.

🔗 Шаг 05:

Миссия 5

Ира пытается определить два поля в базовом и наследнике с одним и тем же именем, оба с модификатором `private`. Что скажет TypeScript?

Подсказка: Подумайте о том, можно ли скрыть приватное поле в наследнике.

Ответ:

- **Ошибка: нельзя объявлять `private` поле с тем же именем в подклассе.**
- Разрешено, они будут разные поля.
- Компиляция, но `warning`.
- То же поле будет переопределено.

🔗 Шаг 06:

Миссия 6

Сопоставьте термины с их описанием приватности в TypeScript.

Термины:

Ответ:

- `private` (TS)
- `protected`
- `public`
- `#private field`

Описание:

1. Доступно только внутри класса, проверяется только на этапе компиляции.
2. Доступно внутри класса и его подклассов, но не извне.
3. Доступно откуда угодно, если есть экземпляр класса.
4. Жёсткая приватность, поле невидимо даже через скобочную нотацию и JSON, соблюдается в рантайме.

Подсказка: Подумайте, какой модификатор работает только на этапе компиляции, а какой — и в рантайме. Истории приватности сейчас развиваются.

- **`private` (TS)** — Доступно только внутри класса, проверяется только на этапе компиляции.

- **protected** — Доступно внутри класса и его подклассов, но не извне.
- **public** — Доступно откуда угодно, если есть экземпляр класса.
- **#private field** — Жёсткая приватность, поле невидимо даже через скобочную нотацию и JSON, соблюдается в рантайме.

🔮 Шаг 07:

Миссия 7

Сопоставьте ситуации с тем, какой модификатор приватности подходит лучше всего.

Ситуации:

Ответ:

- Поле секретного токена, которое ни наследники, ни внешние не должны видеть.
- Поле конфигурации, доступное только наследникам.
- Свойство, доступное всем пользователям объекта.
- Поле, которое должно быть скрыто при сериализации в JSON и недоступно через bracket notation.

Модификатор:

1. private
2. protected
3. public
4. #private field

Подсказка: Подумайте, что делает поле доступным, что делает его невидимым при JSON.stringify.

- **Поле секретного токена, которое ни наследники, ни внешние не должны видеть.** — private
- **Поле конфигурации, доступное только наследникам.** — protected
- **Свойство, доступное всем пользователям объекта.** — public
- **Поле, которое должно быть скрыто при сериализации в JSON и недоступно через bracket notation.** — #private field

🔮 Шаг 08:

Миссия 8

Сопоставьте ошибки компилятора с нарушениями приватности.

Ошибки:

Ответ:

- Property 'x' is private and only accessible within class 'A'
- Property 'y' is protected and only accessible within class 'B' and its subclasses
- Cannot access private field #z outside class
- Cannot assign to 'a' because it is a readonly property

Ситуации:

1. Попытка получить доступ к private полю извне класса.
2. Попытка доступа к protected полю из экземпляра, не наследника.
3. Попытка доступа к #private полю из вне, даже через bracket notation.
4. **Readonly не относится к приватности напрямую** — другое ограничение.

Подсказка: Обратите внимание, что # поля защищены намного сильнее, и не видны даже через скобочную нотацию.

- **Property 'x' is private and only accessible within class 'A'** — Попытка получить доступ к private полю извне класса.
- **Property 'y' is protected and only accessible within class 'B' and its subclasses** — Попытка доступа к protected полю из экземпляра, не наследника.
- **Cannot access private field #z outside class** — Попытка доступа к #private полю из вне, даже через bracket notation.
- **Cannot assign to 'a' because it is a readonly property** — Readonly не относится к приватности напрямую — другое ограничение.

Урок 4 - Статические свойства и методы

Шаг 01:

Миссия 1

Вика пишет класс Counter , и хочет, чтобы свойство count было общим для всех экземпляров, а не привязанным к каждому объекту. Какой вариант объявления подойдёт?

Подсказка: Используйте ключевое слово, которое делает член «принадлежащим классу», не экземпляру.

Ответ:

- **static count: number = 0;**
- count: static number = 0;
- public staticCount: number = 0;
- static: count = 0;

Шаг 02:

Миссия 2

Толя хочет, чтобы метод reset() сбрасывал статический счётчик count у класса Counter , не создавая новый экземпляр. Как он должен объявить этот метод?

Подсказка: Метод должен быть вызываемым через имя класса, без new.

Ответ:

- **static reset(): void { Counter.count = 0; }**
- reset(): void { this.count = 0; }
- public reset() { Counter.count = 0; }
- static reset(count: number): number { return count; }

Шаг 03:

Миссия 3

Женя создал класс Util с несколькими статическими методами и хочет, чтобы нельзя было создавать экземпляры этого класса. Что он может сделать?

Подсказка: Посмотрите, какой модификатор конструктора подходит, чтобы запретить new извне.

Ответ:

- **private constructor() { }**
- protected constructor() { }
- constructor() { throw new Error(); }
- public constructor() { }

🔗 Шаг 04:

Миссия 4

Света делает класс `Config` с константой `PI = 3.14159`, которая должна быть общей и неизменяемой. Как объявить это свойство?

Подсказка: Используйте `static` и модификатор, запрещающий изменение.

Ответ:

- **`static readonly PI: number = 3.14159;`**
- `readonly PI: number = 3.14159;`
- `static PI = "3.14159";`
- `const PI: number;`

🔗 Шаг 05:

Миссия 5

Андрей хочет, чтобы статический метод `getHeadcount()` возвращал, сколько экземпляров класса `Employee` создано. Что внутри метода должно быть?

Подсказка: Используйте статическое свойство, увеличенное в конструкторе, и обратитесь к нему из метода.

Ответ:

- **`return Employee.headcount;`**
- `return this.headcount;`
- `return instance.headcount;`
- `return this.constructor.headcount;`

🔗 Шаг 06:

Миссия 6

Сопоставьте примеры к тому, как используются статические члены.

Примеры:

Ответ:

- `class MathUtil { static PI = 3.14; }`
- `class Logger { static countCalls(): number { ... } }`
- `instance.getSomething()`
- `ClassName.someStaticMethod()`

Что это:

1. Статическое свойство
2. Статический метод
3. Обычный метод экземпляра
4. Вызов статического метода через имя класса

Подсказка: Статические члены объявляются один раз на классе, методы экземпляра — на объекте.

- **`class MathUtil { static PI = 3.14; }`** — Статическое свойство
- **`class Logger { static countCalls(): number { ... } }`** — Статический метод
- **`instance.getSomething()`** — Обычный метод экземпляра
- **`ClassName.someStaticMethod()`** — Вызов статического метода через имя класса

🔧 Шаг 07:

Миссия 7

Сопоставьте ситуации с ошибками, которые TypeScript выдаст при неправильном использовании статических членов.

Ситуации:

Ответ:

- Попытка обратиться к статическому свойству через экземпляр
- Статический метод пытается использовать нестатическое свойство через `this`
- Переопределение статического метода в подклассе с `super`
- Обычный метод экземпляра обращается к статическому свойству

Ожидаемые ошибки или поведение:

1. Ошибка компиляции: свойство не существует на типе экземпляра
2. Ошибка компиляции: `Property 'x' does not exist on type typeof ClassName`
3. Разрешено, вызывается метод подкласса с `super`
4. Работает нормально через имя класса

Подсказка: Помните, что статический метод работает на уровне класса, не экземпляра.

- **Попытка обратиться к статическому свойству через экземпляр** — Ошибка компиляции: свойство не существует на типе экземпляра
- **Статический метод пытается использовать нестатическое свойство через `this`** — Ошибка компиляции: `Property 'x' does not exist on type typeof ClassName`
- **Переопределение статического метода в подклассе с `super`** — Разрешено, вызывается метод подкласса с `super`
- **Обычный метод экземпляра обращается к статическому свойству** — Работает нормально через имя класса

📖 Урок 5 - GettersSetters

🔧 Шаг 01:

Миссия 1

Влад сделал класс `Person` с приватным свойством `_name: string`, и хочет, чтобы внешняя часть кода могла получить имя через `person.name`, но не менять напрямую `_name`. Какой элемент следует добавить в класс?

Подсказка: Используй `get` — это способ, чтобы читать значение как свойство.

Ответ:

- **`get name(): string { return this._name; }`**
- `set name(value: string) { this._name = value; }`
- `public name: string;`
- `private name: string;`

🔧 Шаг 02:

Миссия 2

Маша хочет запретить устанавливать отрицательный возраст в классе `User`. У неё есть приватное поле `_age`. Что должно быть в сеттере `set age(...)`?

Подсказка: Проверяй перед присвоением, если значение `< 0` — игнорируй или выбрасывай ошибку.

Ответ:

- **if (value >= 0) { this._age = value; }**
- this._age = Math.abs(value);
- this._age = value;
- if (value > 18) { this._age = value; }

🔗 **Шаг 03:**

Миссия 3

Олег сделал геттер-only свойство fullName в классе Person , без сеттера. Что будет, если кто-то попытается написать person.fullName = "New Name" ?

Подсказка: Отсутствие сеттера делает свойство доступным только для чтения.

Ответ:

- **Ошибка компиляции: нельзя присваивать значению свойства без set-метода**
- Присваивание просто игнорируется в рантайме
- Присваивание работает, без предупреждений
- Будет warning, но код исполнится

🔗 **Шаг 04:**

Миссия 4

Света хочет, чтобы setter set age() не возвращал значение (void). Что произойдёт, если она укажет тип возвращаемого значения, например number ?

Подсказка: В TypeScript есть правило, что setter не должен возвращать значение.

Ответ:

- **Ошибка компиляции: setter не может иметь возвращаемого типа**
- Просто игнорируется возвращаемое значение
- TypeScript автоматически подставит void
- Метод станет обычным методом, а не setter

🔗 **Шаг 05:**

Миссия 5

Игорь хочет, чтобы геттер get isAdult() возвращал true , когда возраст ≥ 18 , иначе false . Как должно выглядеть тело геттера?

Подсказка: Используй приватное поле _age , сравнение и булев результат.

Ответ:

- **return this._age >= 18;**
- return this._age > 18;
- return this.age >= 18;
- if (this._age >= 18) return true; else return null;

🔗 **Шаг 06:**

Миссия 6

Сопоставьте части класса с их назначением, когда используются get/set.

Части:

Ответ:

- `private _value`
- `get value()`
- `set value(v)`
- отсутствие `setter`

Назначения:

1. Хранит внутреннее состояние, недоступное напрямую.
2. Возвращает `_value`, возможно с дополнительной логикой.
3. Позволяет изменять `_value` с проверкой или логикой.
4. Делает свойство `read-only` снаружи.

Подсказка: Думай над тем, что делает `get`, что делает `set`, и что если `setter` нет.

- **`private _value`** — Хранит внутреннее состояние, недоступное напрямую.
- **`get value()`** — Возвращает `_value`, возможно с дополнительной логикой.
- **`set value(v)`** — Позволяет изменять `_value` с проверкой или логикой.
- **отсутствие `setter`** — Делает свойство `read-only` снаружи.

 Шаг 07:**Миссия 7**

Сопоставьте примеры с тем, какое поведение они демонстрируют, когда используются геттеры и сеттеры.

Примеры:

Ответ:

- `person.name` читает приватное поле через геттер
- `person.age = -5` через сеттер, который игнорирует отрицательные значения
- `person.fullName = "John Doe"` где нет сеттера
- `person.isAdult` вычисляется на основе возраста через геттер

Поведение:

1. Чтение приватного поля
2. Валидация в сеттере
3. Ошибка присваивания, т.к. нет сеттера
4. Вычисленное свойство

Подсказка: Подумайте, что делает сеттер и когда он может не существовать.

- **`person.name`** — Чтение приватного поля
- **`person.age = -5`** — Валидация в сеттере
- **`person.fullName = "John Doe"`** — Ошибка присваивания, т.к. нет сеттера
- **`person.isAdult`** — Вычисленное свойство

 Шаг 08:**Миссия 8**

Сопоставьте ошибки/поведения с ситуациями, связанными с `get/set` в TypeScript.

Ситуации:

Ответ:

- Сеттер указан с возвращаемым типом отличным от `void`

- Отсутствует сеттер, но свойство устанавливается извне
- Сеттер проверяет значение перед присвоением
- Геттер объединяет два приватных поля в одно публичное

Что происходит:

1. Ошибка компиляции: setter не может возвращать тип, который не void
2. Ошибка: нет setter, свойство не определяется как устанавливаемое извне
3. Значение фильтруется или не применяется, если не проходит проверку
4. Геттер возвращает конкатенацию или вычисленное значение

Подсказка: Различия между тем, что делает сеттер и что делает геттер — критически важны.

- **Сеттер указан с возвращаемым типом отличным от void** — Ошибка компиляции: setter не может возвращать тип, который не void
- **Отсутствует сеттер, но свойство устанавливается извне** — Ошибка: нет setter, свойство не определяется как устанавливаемое извне
- **Сеттер проверяет значение перед присвоением** — Значение фильтруется или не применяется, если не проходит проверку
- **Геттер объединяет два приватных поля в одно публичное** — Геттер возвращает конкатенацию или вычисленное значение

Урок 6 - Типизация this

Шаг 01:

Миссия 1

Оля создала класс Person с приватным полем `_name: string`. Она хочет, чтобы свойство `name` можно было читать как `person.name`. Что нужно добавить в класс?

Подсказка: Используй ключевое слово, которое делает метод похожим на свойство.

Ответ:

- **`get name(): string { return this._name; }`**
- `set name(value: string) { this._name = value; }`
- `public name: string;`
- `getName(): string { return this._name; }`

Шаг 02:

Миссия 2

Сергей хочет, чтобы возраст `age` в классе User не мог быть отрицательным. У него есть приватное поле `_age`. Что должно быть в сеттере `set age(...)` ?

Подсказка: Проверяй значение перед присвоением.

Ответ:

- **`if (value >= 0) { this._age = value; }`**
- `this._age = value;`
- `throw new Error("Invalid age");`
- `if (value > 18) { this._age = value; }`

Шаг 03:

Миссия 3

Наташа определила только геттер `get fullName()` в классе Person, где возвращается соединение `firstName` и `lastName`. Что произойдёт, если кто-то напишет `person.fullName = "Новое Имя"` ?

Подсказка: Нет сеттера → свойство только для чтения.

Ответ:

- **Ошибка компиляции: нет сеттера, нельзя присвоить**
- Работает, но значение игнорируется
- Будет warning, но код выполнится
- Set-метод автоматически создаётся

🔮 Шаг 04:

Миссия 4

Паша хочет, чтобы сеттер `set salary()` не возвращал значение. Что произойдёт, если в объявлении сеттера указать возвращаемый тип, например `number` ?

Подсказка: В TypeScript сеттер не может явно указывать возвращаемый тип, отличный от `void`.

Ответ:

- **Ошибка компиляции: setter не может возвращать тип**
- Тип будет игнорироваться
- Возвращает `undefined` автоматически
- Будет предупреждение, но выполнится

🔮 Шаг 05:

Миссия 5

Ирина хочет, чтобы геттер `get isAdult()` возвращал `true` , когда возраст ≥ 18 , иначе `false` . Как правильно реализовать тело геттера?

Подсказка: Используй приватное поле `_age` и возвращай булево значение.

Ответ:

- **`return this._age >= 18;`**
- `return this._age > 18;`
- `return Boolean(this._age == 18);`
- `return this._age = 18;`

🔮 Шаг 06:

Миссия 6

Сопоставьте части класса с их назначением, когда используются `getters/setters`.

Части:

Ответ:

- `private _value`
- `get value()`
- `set value(v)`
- отсутствие `setter`

Назначения:

1. Хранит внутреннее состояние, недоступное напрямую.
2. Возвращает значение, возможно с логикой.
3. Устанавливает значение с проверкой или преобразованием.
4. Свойство только для чтения извне.

Подсказка: Подумайте, что делает `get`, что делает `set` и что если сеттера нет.

- **private _value** — Хранит внутреннее состояние, недоступное напрямую.
- **get value()** — Возвращает значение, возможно с логикой.
- **set value(v)** — Устанавливает значение с проверкой или преобразованием.
- **отсутствие setter** — Свойство только для чтения извне.

🔗 Шаг 07:

Миссия 7

Сопоставьте примеры поведения с правильным описанием, когда используются getters/setters.

Примеры:

Ответ:

- `person.name` читает через геттер
- `person.age = -5` через сеттер, который блокирует отрицательное
- `person.fullName = "John Doe"` где нет сеттера
- `person.isAdult` вычисляется динамически через геттер

Поведение:

1. Чтение приватного поля
2. Валидация в сеттере
3. Ошибка: нельзя присвоить, нет сеттера
4. Вычисленное свойство

Подсказка: Подумайте, что делает сеттер и геттер, и какое поведение ожидается.

- **person.name** — Чтение приватного поля
- **person.age = -5** — Валидация в сеттере
- **person.fullName = "John Doe"** — Ошибка: нельзя присвоить, нет сеттера
- **person.isAdult** — Вычисленное свойство

🔗 Шаг 08:

Миссия 8

Сопоставьте ситуации с результатами, когда используются геттер/сеттер и разные типы.

Ситуации:

Ответ:

- Геттер возвращает `number`, сеттер принимает `string | number`
- Сеттер объявлен, геттер отсутствует
- Геттер отсутствует, но сеттер есть
- Геттер и сеттер оба есть, типы согласованы

Что происходит:

1. Типизация допускает разные типы установки и чтения (TS $\geq$ 4.3) при условии совместимости.
2. **Чтение возможно через приватное поле, но нельзя записать напрямую** — свойство только для установки или только для чтения
3. Свойство можно установить, но чтение вызовет ошибку компиляции
4. Свойство работает как обычный двусторонний геттер/сеттер

Подсказка: Обрати внимание на версию TypeScript и правило `related-getter-setter-pairs`. `:contentReference[oaicite:0]{index=0}`

- **Геттер возвращает number, сеттер принимает string | number** — Типизация допускает разные типы установки и чтения (TS $\geq$ 4.3) при условии совместимости.
- **Сеттер объявлен, геттер отсутствует** — Чтение возможно через приватное поле, но нельзя записать напрямую — свойство только для установки или только для чтения.
- **Геттер отсутствует, но сеттер есть** — Свойство можно установить, но чтение вызовет ошибку компиляции.
- **Геттер и сеттер оба есть, типы согласованы** — Свойство работает как обычное двусторонний геттер/сеттер.

Урок 7 - Квест Создание модели бизнес-логики в ООП стиле

Шаг 01:

Миссия 1

Антон моделирует систему заказов в TypeScript. У него есть класс Order , который хранит массив позиций заказа и должен вычислять общую сумму. Где логичнее всего разместить метод getTotal() ?

Подсказка: Метод относится к одному заказу, использует его данные — не общий для всех.

Ответ:

- **Внутри класса Order как экземплярный метод**
- Как статический метод Order.getTotal(orders[])
- В отдельной утилите вне модели
- В интерфейсе OrderService

Шаг 02:

Миссия 2

Оля создаёт класс Product с приватным полем _price . Она хочет, чтобы цена никогда не была отрицательной, даже если кто-то через код пытается изменить ее. Что нужно сделать?

Подсказка: Это правило — часть бизнес-логики модели.

Ответ:

- **В сеттере price проверять value $\geq$ 0, иначе выбрасывать ошибку**
- Оставить поле public и доверять, что код не передаст отрицательное
- В конструкторе, но не в сеттере
- Использовать static утилиту для проверки перед созданием продукта

Шаг 03:

Миссия 3

Максим хочет, чтобы при добавлении товара в корзину, если товара нет, он создавался, иначе увеличивалось количество. Куда логично поместить этот метод — в модель корзины (Cart) или вне её?

Подсказка: Это поведение относится к конкретной корзине, её состоянию.

Ответ:

- **Внутри класса Cart как экземплярный метод addItem(item)**
- В глобальной функции addCartItem(cart, item)
- В сервисе CartService только
- В утилите helper

🔮 Шаг 04:

Миссия 4

Юля моделирует сущность `UserAccount` с приватным балансом `_balance`. Как лучше всего реализовать метод `withdraw(amount)`, чтобы бизнес-правило “нельзя снять больше, чем баланс” соблюдалось?

Подсказка: Проверка обязательна до изменения состояния объекта.

Ответ:

- Если `amount <= this._balance`, то уменьшить, иначе бросить ошибку
- `this._balance -= amount; // довериться вызывающему`
- Баланс отрицательный станет — нормально
- Проверка после снятия

🔮 Шаг 05:

Миссия 5

Саша делает модель `Invoice`, у которого есть поле `status`, которое может быть только "pending", "paid" или "cancelled". Какой тип лучше использовать?

Подсказка: Ограничи возможность ошибок при присвоении.

Ответ:

- `type InvoiceStatus = 'pending' | 'paid' | 'cancelled';`
- `string`
- `enum InvoiceStatus { Pending, Paid, Cancelled }`
- `number`

🔮 Шаг 06:

Миссия 6

Сопоставьте сущности с бизнес-обязанностями, когда строится модель логики.

Сущности:

Ответ:

- User
- Order
- Product
- Cart

Обязанности:

1. Хранить товары, выбранные для покупки
2. Представлять покупателя, его имя, контакт, аккаунт
3. Содержать товары и вычислять сумму заказа, проверять статус
4. Представлять товар, его цену, описание, валидность

Подсказка: Подумайте, какая сущность отвечает за что в реальной системе магазина.

- **User** — Представлять покупателя, его имя, контакт, аккаунт
- **Order** — Содержать товары и вычислять сумму заказа, проверять статус
- **Product** — Представлять товар, его цену, описание, валидность
- **Cart** — Хранить товары, выбранные для покупки

🔗 Шаг 07:

Миссия 7

Сопоставьте методы с бизнес-правилами, которые они реализуют.

Методы:

Ответ:

- `addItem(quantity)`
- `removeItem(quantity)`
- `cancelOrder()`
- `pay(amount)`

Правила:

1. Не допустить отрицательного количества товара в корзине
2. При отмене заказа изменить статус и вернуть товары в наличии
3. Оплата возможна только если статус заказа "pending"
4. Удаление товара уменьшает, но не делает отрицательным

Подсказка: Какие бизнес-правила естественны в таких методах?

- **`addItem(quantity)`** — Не допустить отрицательного количества товара в корзине
- **`removeItem(quantity)`** — Удаление товара уменьшает, но не делает отрицательным
- **`cancelOrder()`** — При отмене заказа изменить статус и вернуть товары в наличии
- **`pay(amount)`** — Оплата возможна только если статус заказа "pending"

🔗 Шаг 08:

Миссия 8

Сопоставьте примеры ошибок с тем, что не было учтено в бизнес-логике модели.

Ошибки:

Ответ:

- Допускается отрицательный остаток после удаления товара
- Можно оплатить заказ, который уже отменён
- Добавление товара в корзину без проверки его наличия в каталоге
- Создание продукта с нулевой или отрицательной ценой

Что не учтено:

1. Проверка количества в `removeItem`
2. Статус заказа перед `pay()`
3. Наличие продукта перед добавлением в `Cart`
4. Валидация цены в конструкторе `Product`

Подсказка: В модели бизнес-логики важно предусмотреть все правила, иначе неконсистентность.

- **Допускается отрицательный остаток после удаления товара** — Проверка количества в `removeItem`
- **Можно оплатить заказ, который уже отменён** — Статус заказа перед `pay()`
- **Добавление товара в корзину без проверки его наличия в каталоге** — Наличие продукта перед добавлением в `Cart`
- **Создание продукта с нулевой или отрицательной ценой** — Валидация цены в конструкторе `Product`

Модуль 7 - Декораторы и метапрограммирование

Урок 1 - Декораторы классов, методов, свойств

Шаг 01:

Миссия 1

В TypeScript, что делает декоратор класса?

Подсказка: Подумайте, что касается всего класса, а не отдельных его членов.

Ответ:

- **Модифицирует либо заменяет конструктор класса**
- Обязывает методы класса быть статическими
- Применяется только к методам класса

Шаг 02:

Миссия 2

Какой декоратор в TypeScript позволяет изменить поведение метода перед его выполнением?

Подсказка: Название должно намекать, что метод будет обёрнут.

Ответ:

- **Method decorator**
- Property decorator
- Class decorator
- Parameter decorator

Шаг 03:

Миссия 3

Что можно изменять с помощью декоратора свойства (property)?

Подсказка: Рассмотрите, что хранится в метаданных свойства.

Ответ:

- **Изменить дескриптор свойства (writable, enumerable)**
- **Сохранить оригинальную мета-информацию о типе**
- Автоматически вызывать метод при установке свойства

Шаг 04:

Миссия 4

Что такое метапрограммирование в контексте TypeScript и декораторов?

Подсказка: Это про код, который манипулирует самим кодом.

Ответ:

- **Программирование, где код изменяет или генерирует другой код во время компиляции или выполнения**
- Простой вызов функций без классов
- Только работа с интерфейсами
- Использование только стандартных библиотек JS

🔮 Шаг 05:

Миссия 5

Когда декоратор метода получает target, propertyKey и descriptor, что такое descriptor?

Подсказка: Descriptor описывает как метод представлен в объекте.

Ответ:

- **Объект, содержащий value, writable, enumerable, configurable для метода**
- Тип возвращаемого метода
- Список параметров метода

🔮 Шаг 06:

Миссия 6

Сопоставьте декораторы с тем, что они делают:

Декораторы:

Ответ:

- Class decorator
- Method decorator
- Property decorator
- Parameter decorator

Что они делают:

1. Применяется к аргументу метода
2. Изменяет конструктор или прототип класса
3. **Влияет на метод** — может модифицировать поведение
4. Работает на уровне хранения данных свойства

Подсказка: Смотрите, к чему привязывается декоратор — к классу, к свойству, параметру, методу.

- **Class decorator** — Изменяет конструктор или прототип класса
- **Method decorator** — Влияет на метод — может модифицировать поведение
- **Property decorator** — Работает на уровне хранения данных свойства
- **Parameter decorator** — Применяется к аргументу метода

🔮 Шаг 07:

Миссия 7

Сопоставьте части метода декоратора с их назначением:

Части:

Ответ:

- target
- propertyKey
- descriptor.value
- descriptor.writable

Назначение:

1. Имя метода
2. Сама функция метода
3. Объект, содержащий метод/конструктор

4. Разрешение на изменение метода

Подсказка: descriptor — это объект, знакомый из Object.defineProperty.

- **target** — Объект, содержащий метод/конструктор
- **propertyKey** — Имя метода
- **descriptor.value** — Сама функция метода
- **descriptor.writable** — Разрешение на изменение метода

Шаг 08:

Миссия 8

Сопоставьте случаи использования классовых декораторов:

Кейс:

Ответ:

- Логирование создания экземпляра
- Добавление метода ко всем экземплярам
- Изменение прототипа базового класса
- Внедрение зависимостей в конструктор

Подходит для Class decorator или нет:

1. Да, логировать создание при new Class()
2. Да, добавить новый метод в prototype
3. Да, изменить базовый прототип
4. Да, но только через фабрику, не через декоратор класса

Подсказка: Class decorator влияет на сам класс, но может иметь ограничения.

- **Логирование создания экземпляра** — Да, логировать создание при new Class()
- **Добавление метода ко всем экземплярам** — Да, добавить новый метод в prototype
- **Изменение прототипа базового класса** — Да, изменить базовый прототип
- **Внедрение зависимостей в конструктор** — Да, но только через фабрику, не через декоратор класса

Шаг 09:

Миссия 9

Сопоставьте виды ошибок с ситуацией использования метаданных (Reflect.metadata):

Ситуация:

Ответ:

- Отсутствие метаданных типа аргумента метода
- Несовпадение типов при проверке времени выполнения
- Ошибка, что Reflect не определён
- Неправильный ключ метаданных

Причина:

1. Не установлено emitDecoratorMetadata в tsconfig
2. Использован неверный ключ в Reflect.getMetadata
3. Попытка читать метаданные для несуществующего свойства
4. Отсутствие полифила Reflect API в окружении

Подсказка: Проверьте конфигурации TypeScript и окружение выполнения.

- **Отсутствие метаданных типа аргумента метода** — Не установлено `emitDecoratorMetadata` в `tsconfig`
- **Ошибка, что Reflect не определён** — Отсутствие полифила `Reflect API` в окружении
- **Неправильный ключ метаданных** — Использован неверный ключ в `Reflect.getMetadata`
- **Несовпадение типов при проверке времени выполнения** — Попытка читать метаданные для несуществующего свойства

🔮 Шаг 10:

Миссия 10

Упорядочите шаги, как класс-декоратор может изменять конструктор:

1. Сохранить оригинальный конструктор класса
2. Создать новый класс, расширяющий или заменяющий оригинал
3. Возвращение нового конструктора из декоратора
4. Замена ссылки на класс в месте его объявления

Подсказка: Декоратор класса может вернуть новый конструктор, и тогда он будет использован вместо оригинала.

Ответ:

- Сохранить оригинальный конструктор класса
- Создать новый класс, расширяющий или заменяющий оригинал
- Возвращение нового конструктора из декоратора
- Замена ссылки на класс в месте его объявления

🔮 Шаг 11:

Миссия 11

Упорядочите, что происходит внутри метода-декоратора перед вызовом оригинального метода:

1. Сохранить оригинальный метод
2. Изменить `descriptor.value` на обёрнутую функцию
3. Выполнить дополнительную логику (лог, проверка, ...)
4. Вызвать оригинальный метод с контекстом и аргументами

Подсказка: Обёртка должна вызывать оригинал после своей логики, чтобы не нарушить поведение.

Ответ:

- Сохранить оригинальный метод
- Изменить `descriptor.value` на обёрнутую функцию
- Выполнить дополнительную логику (лог, проверка, ...)
- Вызвать оригинальный метод с контекстом и аргументами

🔮 Шаг 12:

Миссия 12

Упорядочите шаги включения `Property decorator`:

1. Получение `target` и `propertyKey` свойством
2. Изменение или определение дескриптора свойства через `Object.defineProperty` (опционально)
3. Чтение или запись значения свойства при доступе (`getter/setter`) (если задано)

4. Применение декоратора к свойству при объявлении класса

Подсказка: Декоратор свойства срабатывает в момент объявления класса, не при каждом обращении к свойству.

Ответ:

- Применение декоратора к свойству при объявлении класса
- Получение target и propertyKey свойством
- Изменение или определение дескриптора свойства через Object.defineProperty (опционально)
- Чтение или запись значения свойства при доступе (getter/setter) (если задано)

Шаг 13:

Миссия 13

Упорядочите этапы использования Parameter decorator:

1. Декоратор получает target, propertyKey и индекс параметра
2. Сохранение информации о параметре (например, в метаданных)
3. Метод вызывается как обычно при runtime
4. Использование сохранённых метаданных для проверки или логирования аргументов при вызове метода

Подсказка: Декоратор параметра не изменяет метод непосредственно, он лишь собирает информацию.

Ответ:

- Декоратор получает target, propertyKey и индекс параметра
- Сохранение информации о параметре (например, в метаданных)
- Метод вызывается как обычно при runtime
- Использование сохранённых метаданных для проверки или логирования аргументов при вызове метода

Шаг 14:

Миссия 14

Упорядочите стадии создания декоратора, который добавляет лог перед каждым методом класса:

1. Перебор всех методов в прототипе класса
2. Для каждого метода сохранить оригинал
3. Создать новую функцию-обёртку, вызывающую лог, потом оригинал
4. Заменить метод на обёртку в прототипе класса

Подсказка: Class decorator может модифицировать прототип, чтобы изменить методы всех экземпляров.

Ответ:

- Перебор всех методов в прототипе класса
- Для каждого метода сохранить оригинал
- Создать новую функцию-обёртку, вызывающую лог, потом оригинал
- Заменить метод на обёртку в прототипе класса

Шаг 15:

Миссия 15

Упорядочите полный жизненный цикл декоратора, от объявления до эффекта:

1. TypeScript читает метадекораторы в AST при компиляции
2. Компилируется JS-код с декораторными вызовами
3. При загрузке класса в рантайме декоратор вызывается
4. Метаданные сохраняются (если `emitDecoratorMetadata` включён)
5. Код метода/свойства/параметра изменён или обёрнут

Подсказка: Подумайте, когда происходят компиляция, когда — выполнение и когда — создание метаданных.

Ответ:

- TypeScript читает метадекораторы в AST при компиляции
- Компилируется JS-код с декораторными вызовами
- При загрузке класса в рантайме декоратор вызывается
- Метаданные сохраняются (если `emitDecoratorMetadata` включён)
- Код метода/свойства/параметра изменён или обёрнут

Шаг 16:

Миссия 16

Упорядочите шаги, как создать метод-декоратор, проверяющий что аргумент не null перед выполнением метода:

1. Декоратор получает `target`, `propertyKey` и `descriptor`
2. Сохранить оригинальный метод из `descriptor.value`
3. Переписать `descriptor.value` на функцию, проверяющую аргумент
4. При нарушении бросить ошибку, иначе вызвать оригинал

Подсказка: Проверьте порядок: получение, сохранение, замена, логика.

Ответ:

- Декоратор получает `target`, `propertyKey` и `descriptor`
- Сохранить оригинальный метод из `descriptor.value`
- Переписать `descriptor.value` на функцию, проверяющую аргумент
- При нарушении бросить ошибку, иначе вызвать оригинал

Шаг 17:

Миссия 17

Упорядочите все ключевые аспекты, когда комбинируются несколько декораторов (класс, метод, параметр):

1. Декораторы параметров вызываются сначала
2. Декораторы методов применяются после параметров, перед классом
3. Декораторы свойств обрабатываются при объявлении класса
4. Декораторы класса вызываются в конце последовательности

Подсказка: Есть установленный порядок: параметры → методы → свойства → класс.

Ответ:

- Декораторы параметров вызываются сначала
- Декораторы методов применяются после параметров, перед классом
- Декораторы свойств обрабатываются при объявлении класса
- Декораторы класса вызываются в конце последовательности

Урок 2 - Метаданные и reflect-metadata

Шаг 01:

Миссия 1

Ваша задача — разобраться в метаданных в TypeScript. Что из нижеперечисленного будет храниться в метаданных объекта?

Подсказка: Метаданные часто включают типы и другую информацию о структурных элементах.

Ответ:

- **Типы данных и их аннотации**
- Содержимое функций
- Список всех переменных программы
- Объекты и массивы данных программы

Шаг 02:

Миссия 2

Какую роль выполняет библиотека reflect-metadata при работе с метаданными?

Подсказка: Эта библиотека помогает работать с метаданными на уровне типов и декораторов.

Ответ:

- **Она позволяет считывать и записывать метаданные объектов**
- Она генерирует код для метаданных
- Она улучшает производительность программы
- Она упрощает создание новых типов данных

Шаг 03:

Миссия 3

 Задача: Сопоставьте ключевые элементы метаданных с их описаниями.

Элементы:

Ответ:

- Метаданные класса
- Декоратор параметра
- Декоратор метода
- Метаданные метода

Описание:

1. Хранит информацию о параметрах метода (например, их типы)
2. Предназначен для добавления дополнительной информации о методах класса.
3. Содержит метаданные, связанные с самим классом (например, его имя или родительский класс).
4. Применяется к методу, чтобы дать информацию о его характеристиках.

Подсказка:

Подумайте, как декораторы могут быть применены в TypeScript, чтобы обеспечить структуру данных для методов, параметров и самого класса.

- **Метаданные класса** — Содержит метаданные, связанные с самим классом (например, его имя или родительский класс).
- **Декоратор параметра** — Хранит информацию о параметрах метода (например, их типы)

- **Декоратор метода** — Предназначен для добавления дополнительной информации о методах класса.
- **Метаданные метода** — Применяется к методу, чтобы дать информацию о его характеристиках.

🔗 Шаг 04:

Миссия 4

В какой ситуации необходимо использовать декоратор `@Reflect.metadata`?

Подсказка: Этот декоратор используется для добавления метаданных к элементам.

Ответ:

- **Когда нужно добавить метаданные к параметрам или методам**
- Когда нужно изменить поведение функции
- Когда необходимо запустить асинхронный код
- Когда нужно проверить типы в JavaScript

🔗 Шаг 05:

Миссия 5

Как вы думаете, что произойдёт, если метаданные не будут добавлены к параметрам функции?

Подсказка: Без метаданных можно столкнуться с проблемами при анализе кода.

Ответ:

- **Параметры функции могут быть неправильно интерпретированы и обработаны**
- Код будет работать быстрее
- Метаданные будут автоматически добавлены компилятором
- Ничего не изменится, программа будет работать нормально

🔗 Шаг 06:

Миссия 6

Расположите шаги использования `reflect-metadata` в правильном порядке:

1. Импортируйте библиотеку `reflect-metadata` в файл
2. Применяйте декораторы с использованием `reflect.metadata`
3. Считывайте метаданные с помощью `Reflect.getMetadata`
4. Используйте полученные метаданные для анализа или логирования

🔗 **Подсказка:**

Подумайте, как организован процесс работы с библиотеками в TypeScript. Сначала вы подключаете нужную библиотеку, а затем уже начинаете использовать её функционал.

Ответ:

- Импортируйте библиотеку `reflect-metadata` в файл
- Применяйте декораторы с использованием `reflect.metadata`
- Считывайте метаданные с помощью `Reflect.getMetadata`
- Используйте полученные метаданные для анализа или логирования

🔗 Шаг 07:

Миссия 7

Что происходит, если вы пытаетесь получить метаданные для объекта, к которому не применен декоратор?

Подсказка: Отсутствие декоратора может привести к пустому результату или ошибке.

Ответ:

- **Метаданные не будут найдены**
- Будет выведена ошибка компиляции
- Метаданные будут автоматически добавлены
- Метаданные будут получены по умолчанию

Шаг 08:

Миссия 8

Что такое `Reflect.getMetadata` и как она работает в TypeScript?

Подсказка: Это важный метод для получения метаданных, связанных с объектом или его свойствами.

Ответ:

- **Она извлекает метаданные, связанные с указанным объектом или элементом**
- Она автоматически обновляет метаданные объектов
- Она позволяет манипулировать объектами и их методами
- Она генерирует метаданные для новых классов и функций

Шаг 09:

Миссия 9

 Задача: Сопоставьте метаданные с их правильным использованием в TypeScript.

Метаданные:

Ответ:

- `@Reflect.metadata`
- `Reflect.getMetadata`
- Декоратор параметра
- Декоратор класса

Использование:

1. Применяется для добавления метаданных к параметрам метода или функции.
2. Позволяет получить метаданные, сохранённые для определённых элементов.
3. Применяется для добавления метаданных к самому классу.
4. Используется для создания метаданных, привязанных к классу или его элементам.

Подсказка:

Обратите внимание на различие между декораторами для параметров, методов и классов. Все они имеют разные применения для метаданных.

- **@Reflect.metadata** — Используется для создания метаданных, привязанных к классу или его элементам.
- **Reflect.getMetadata** — Позволяет получить метаданные, сохранённые для определённых элементов.
- **Декоратор параметра** — Применяется для добавления метаданных к параметрам метода или функции.
- **Декоратор класса** — Применяется для добавления метаданных к самому классу.

🔗 Шаг 10:

Миссия 10

Что будет, если вы забудете импортировать библиотеку `reflect-metadata` в файл, где применяете декораторы?

Подсказка: Без этой библиотеки метаданные не будут работать корректно.

Ответ:

- **Метаданные не будут правильно извлекаться или записываться**
- Программа не скомпилируется
- Ошибки будут происходить только при запуске программы
- Метаданные будут записываться автоматически компилятором

🔗 Шаг 11:

Миссия 11

Какой метод позволяет проверить, существует ли для класса или его элемента метаданные?

Подсказка: Этот метод позволяет выполнить проверку перед получением метаданных.

Ответ:

- **Reflect.hasMetadata**
- Reflect.getMetadata
- Reflect.defineMetadata
- Reflect.setMetadata

🔗 Шаг 12:

Миссия 12

Когда вы впервые слышите о `reflect-metadata`, что лучше всего сделать первым?

Подсказка: Сначала изучить, как хранится и читается метаданные.

Ответ:

- **Игнорировать** — работать напрямую с `prototype`
- **Установить `reflect-metadata` и прочитать базовые API**
- Создать свой формат хранения метаданных вручную

🔗 Шаг 13:

Миссия 13

Вы хотите добавить метаданные к параметру метода. Какой декоратор из `reflect-metadata` поможет вам?

Подсказка: Подумайте о том, где хранится метадата: на параметрах, свойствах или классах.

Ответ:

- **Reflect.metadata для параметров**
- Reflect.defineMetadata на уровне класса
- Reflect.getMetadata на уровне свойства

🔗 Шаг 14:

Миссия 14

Где хранятся метаданные, определённые через `Reflect.defineMetadata`?

Подсказка: Хранение происходит на целевом объекте, часто через `WeakMap` внутри библиотеки.

Ответ:

- **На целевом объекте (или его прототипе) внутри библиотеки**
- В глобальной переменной
- В файле JSON на диске

Шаг 15:

Миссия 15

Сопоставьте виды метаданных с примерами:

Ответ:

- Класс
 - Метод
 - Параметр
 - Свойство
1. Метаданные, читаемые у функции внутри класса
 2. Аннотация типа свойства поля
 3. Информация о типе аргумента
 4. Общая метадата на весь класс

Подсказка: Подумайте, где `assign` применяется: декоратор класса, параметров, методов, свойств.

- **Класс** — Общая метадата на весь класс
- **Метод** — Метаданные, читаемые у функции внутри класса
- **Параметр** — Информация о типе аргумента
- **Свойство** — Аннотация типа свойства поля

Шаг 16:

Миссия 16

Сопоставьте методы `reflect-metadata` с их назначением:

Ответ:

- `defineMetadata`
 - `getMetadata`
 - `hasMetadata`
 - `deleteMetadata`
1. Удалить метаданные
 2. Проверить, есть ли метаданные у объекта или в прототипе
 3. Установить метаданные
 4. Получить метаданные, учитывая наследование

Подсказка: Названия методов часто прямо намекают на действие.

- **`defineMetadata`** — Установить метаданные
- **`getMetadata`** — Получить метаданные, учитывая наследование
- **`hasMetadata`** — Проверить, есть ли метаданные у объекта или в прототипе
- **`deleteMetadata`** — Удалить метаданные

Шаг 17:

Миссия 17

Вы применяете декоратор свойства, но не видите метаданные при вызове `getMetadata`. Что, скорее всего, забыли?

Подсказка: Задействуйте `import 'reflect-metadata'` в начале приложения.

Ответ:

- Не импортировали ``reflect-metadata`` перед декораторами
- Не объявили класс как `abstract`
- Не дали метаданные в JSON

Шаг 18:

Миссия 18

Вы хотите прочитать метаданные, определённые в суперклассе, из подкласса. Какой метод использовать?

Подсказка: Метод должен учитывать прототипное наследование.

Ответ:

- **`Reflect.getMetadata` (а не `getOwnMetadata`)**
- `Reflect.getOwnMetadata`
- `Reflect.defineMetadata`

Шаг 19:

Миссия 19

Какой встретившийся кейс не решится просто использованием `reflect-metadata`?

Подсказка: Подумайте о параметрах во время рантайма, которые не сохраняются системой типов.

Ответ:

- **Узнать имена параметров функции (`Token names`)**
- Хранить типы свойств
- Считывать декораторы

Урок 3 - Примеры DIValidation

Шаг 01:

Миссия 1

Анна хочет, чтобы при создании объекта `User` автоматически валидировались поля через декораторы. С чего ей лучше начать?

Подсказка: Сначала нужно определить, как хранить правила в метаданных.

Ответ:

- **Создать декоратор `@Validate`, внутри которого применять `Reflect.defineMetadata`**
- Писать валидацию вручную в конструкторе класса
- Использовать глобальный объект для хранения правил

Шаг 02:

Миссия 2

Борис применил @IsEmail() к полю email, но при проверке не видит правила. Что наиболее вероятно забыто?

Подсказка: Подумайте, как привязать декораторы к коду валидатора.

Ответ:

- **Не вызвать код чтения метаданных перед валидацией**
- Не импортировать модуль @IsEmail
- Не задать тип поля как string

🔗 Шаг 03:

Миссия 3

Когда вы используете DI-контейнер с декораторами @Inject, в каком месте целесообразно читать метаданные зависимости?

Подсказка: Там, где вы создаёте экземпляр через контейнер.

Ответ:

- **В момент разрешения зависимости в контейнере**
- При объявлении класса
- При экспорте модуля

🔗 Шаг 04:

Миссия 4

Сопоставьте валидатор и его назначение:

Ответ:

- @IsEmail
 - @MinLength(n)
 - @IsPositive
 - @Matches(regex)
1. Проверяет соответствие регулярному выражению
 2. Не позволяет отрицательные числа
 3. Минимальная длина строки
 4. **Проверяет, что значение — email**

Подсказка: Названия говорящие: подумайте, что делает каждый валидатор.

- **@IsEmail** — Проверяет, что значение — email
- **@MinLength(n)** — Минимальная длина строки
- **@IsPositive** — Не позволяет отрицательные числа
- **@Matches(regex)** — Проверяет соответствие регулярному выражению

🔗 Шаг 05:

Миссия 5

Сопоставьте части DI-системы и их роли:

Ответ:

- Провайдер
- Ключ зависимости
- Контейнер

- Токен

1. Объект, создающий экземпляры
2. Идентификатор, по которому ищут реализацию
3. Хранилище регистраций и логика разрешения
4. Абстрактная метадата, используемая в декораторах

Подсказка: Подумайте, что делает каждый элемент в контексте DI.

- **Провайдер** — Объект, создающий экземпляры
- **Ключ зависимости** — Идентификатор, по которому ищут реализацию
- **Контейнер** — Хранилище регистраций и логика разрешения
- **Токен** — Абстрактная метадата, используемая в декораторах

Шаг 06:

Миссия 6

Сопоставьте ошибку в DI / Validation и её вероятную причину:

Ответ:

- Проверки не выполняются
 - Контейнер возвращает undefined
 - Конфликт токенов
 - Отсутствует правило для поля
1. Не вызван валидатор перед сохранением
 2. Класс не зарегистрирован в контейнере
 3. Два @Inject используют один ключ
 4. Нет декоратора над полем

Подсказка: Проблемы часто между регистрацией, чтением и применением.

- **Проверки не выполняются** — Не вызван валидатор перед сохранением
- **Контейнер возвращает undefined** — Класс не зарегистрирован в контейнере
- **Конфликт токенов** — Два @Inject используют один ключ
- **Отсутствует правило для поля** — Нет декоратора над полем

Шаг 07:

Миссия 7

Даша хочет, чтобы при вызове метода validate(user) автоматически проверялись все применённые валидаторы. Что логично сделать внутри validate?

Подсказка: Нужно прочитать метаданные валидаторов и пройтись по полям.

Ответ:

- **Получить список валидаторов из метаданных и применить к полям**
- Вызвать JSON.stringify(user) и проверить строку
- Просто проверить все поля на null

Шаг 08:

Миссия 8

Когда вы строите DI + валидацию в одном сервисе, куда логично встроить проверку?

Подсказка: В момент, когда зависимости уже инжецированы, но до вызова логики.

Ответ:

- **В геттере или в методе init после инъекций**
- В конструкторе перед инъекциями
- После завершения логики метода

Шаг 09:

Миссия 9

Сопоставьте подходы DI/валидации и особенности:

Ответ:

- Lazy инъекция
 - Singleton scope
 - Проверка после установки зависимостей
 - Использование исключений в валидаторе
1. Реинициализирует при первом запросе
 2. Единый экземпляр на весь контейнер
 3. После свойства/поля инжектированы, запустить валидацию
 4. Останавливает цепочку при первом неверном поле

Подсказка: Подумайте, когда удобнее проверять и как реагировать на ошибки.

- **Lazy инъекция** — Реинициализирует при первом запросе
- **Singleton scope** — Единый экземпляр на весь контейнер
- **Проверка после установки зависимостей** — После свойства/поля инжектированы, запустить валидацию
- **Использование исключений в валидаторе** — Останавливает цепочку при первом неверном поле

Шаг 10:

Миссия 10

Сопоставьте сценарий и проблему, если DI + валидация не работают:

Ответ:

- Класс A зависит от B, но B не зарегистрирован
 - Поле user.email имеет @IsEmail, но значение пустое
 - Метод save() вызывает validate, но не ждёт результата
 - Декораторы используют один и тот же ключ метадаты
1. Получим undefined при инъекции
 2. Пройдёт как валидное значение
 3. Ошибки не будут обработаны
 4. Только один из валидаторов будет виден

Подсказка: Внимательно проверьте регистрацию, асинхронность и ключи.

- **Класс A зависит от B, но B не зарегистрирован** — Получим undefined при инъекции
- **Поле user.email имеет @IsEmail, но значение пустое** — Пройдёт как валидное значение
- **Метод save() вызывает validate, но не ждёт результата** — Ошибки не будут обработаны

- **Декораторы используют один и тот же ключ метадаты** — Только один из валидаторов будет виден

🔮 Шаг 11:

Миссия 11

Расположите шаги процесса валидации через декораторы:

1. Чтение метаданных валидаторов
2. Вызывание каждого валидатора на поле
3. Сбор ошибок по полям
4. Возврат или бросок исключения, если есть ошибки

Подсказка: Сначала чтение, потом применение, потом агрегация, потом результат.

Ответ:

- Чтение метаданных валидаторов
- Вызывание каждого валидатора на поле
- Сбор ошибок по полям
- Возврат или бросок исключения, если есть ошибки

🔮 Шаг 12:

Миссия 12

Расположите этапы разрешения зависимости с валидацией на входе:

1. Контейнер узнаёт токен и класс-провайдер
2. Создает экземпляр класса (без данных ещё)
3. Инъектит зависимости в конструктор
4. Запускает валидацию перед методом init или before use

Подсказка: Сначала DI, затем валидация перед использованием.

Ответ:

- Контейнер узнаёт токен и класс-провайдер
- Создает экземпляр класса (без данных ещё)
- Инъектит зависимости в конструктор
- Запускает валидацию перед методом init или before use

🔮 Шаг 13:

Миссия 13

Расположите фазы внедрения DI + валидации при HTTP-запросе:

1. Контроллер получает HTTP-тело в DTO
2. Валидация DTO через декораторы
3. Контейнер создаёт сервисы и инъектит зависимости
4. Сервис выполняет логику с валидным DTO

Подсказка: Сначала получение данных, затем проверка, затем DI, затем логика.

Ответ:

- Контроллер получает HTTP-тело в DTO
- Валидация DTO через декораторы
- Контейнер создаёт сервисы и инъектит зависимости

- Сервис выполняет логику с валидным DTO

🔧 Шаг 14:

Миссия 14

Расположите шаги создания кастомного валидатора с параметром:

1. Определить интерфейс валидатора (`validate(value)`): `boolean` или `string`)
2. В декораторе регистрировать валидатор вместе с параметром
3. При валидации получать параметр из метаданных и вызывать `validate(value, param)`
4. Добавить декоратор на поле с аргументом

Подсказка: Сначала интерфейс, потом регистрация, потом применение, потом вызов.

Ответ:

- Определить интерфейс валидатора (`validate(value)`): `boolean` или `string`)
- В декораторе регистрировать валидатор вместе с параметром
- При валидации получать параметр из метаданных и вызывать `validate(value, param)`
- Добавить декоратор на поле с аргументом

🔧 Шаг 15:

Миссия 15

Расположите шаги интеграции DI + валидатора для метода контроллера:

1. Контейнер создаёт экземпляр контроллера и инъецирует сервисы
2. Метод контроллера получает DTO-параметр с декораторами валидации
3. Сначала выполнить валидацию DTO
4. **Если ошибка** — вернуть ответ клиенту, иначе вызвать сервис

Подсказка: В контроллере сначала проверка входных данных, потом вызов бизнес-логики.

Ответ:

- Контейнер создаёт экземпляр контроллера и инъецирует сервисы
- Метод контроллера получает DTO-параметр с декораторами валидации
- Сначала выполнить валидацию DTO
- **Если ошибка** — вернуть ответ клиенту, иначе вызвать сервис

🔧 Шаг 16:

Миссия 16

Расположите этапы расширенного DI с контекстной валидацией (например, в зависимости от роли):

1. Читать метаданные роли и валидаторы на аргументах метода
2. При разрешении зависимости проверять, что роль пользователя позволяет создавать сервис
3. Инъецировать зависимости
4. Внутри метода запускать валидаторы, специфичные для роли

Подсказка: Контекст влияет на разрешение зависимостей и валидацию.

Ответ:

- Читать метаданные роли и валидаторы на аргументах метода
- При разрешении зависимости проверять, что роль пользователя позволяет создавать сервис

- Инъектить зависимости
- Внутри метода запускать валидаторы, специфичные для роли

Шаг 17:

Миссия 17

Расположите шаги построения плагин-системы с DI и валидацией плагинов:

1. Сканировать классы на наличие метаданных @Plugin и @ValidateRules
2. Зарегистрировать плагины и их зависимости в контейнере
3. При активации плагина инъектить зависимости и валидировать конфигурацию
4. Вызывать методы плагинов после успешной инициализации

Подсказка: Подумайте: сначала регистрация, потом создание, проверка, использование.

Ответ:

- Сканировать классы на наличие метаданных @Plugin и @ValidateRules
- Зарегистрировать плагины и их зависимости в контейнере
- При активации плагина инъектить зависимости и валидировать конфигурацию
- Вызывать методы плагинов после успешной инициализации

Урок 4 - Автоматизация с декораторами

Шаг 01:

Миссия 1

Иван хочет автоматически логировать вызовы методов класса с помощью декоратора. Какой подход будет правильным началом?

Подсказка: Нужно перехватывать вызов функции до её выполнения.

Ответ:

- **Использовать декоратор метода, который оборачивает оригинальный метод**
- Добавить код логирования внутри метода вручную
- Запускать логирование через прокси вне класса

Шаг 02:

Миссия 2

Чтобы автоматически сериализовывать объект в JSON, какой момент нужно охватить декоратором?

Подсказка: сериализация обычно вызывается через JSON.stringify.

Ответ:

- **Перехват toJSON метода или добавить toJSON через декоратор**
- Переопределить метод toString
- Сериализовывать через декоратор конструктора

Шаг 03:

Миссия 3

Сопоставьте виды автоматизации с примерами задач, решаемых декораторами:

Ответ:

- Логирование

- Кэширование результата
- Авторизация
- Сериализация / трансформация

1. Запрет или разрешение доступа по роли
2. Запись вызовов метода в лог
3. Сохранение результата метода для повторных вызовов
4. Преобразование объекта при JSON.stringify

Подсказка: Подумайте, что каждый механизм делает автоматически.

- **Логирование** — Запись вызовов метода в лог
- **Кэширование результата** — Сохранение результата метода для повторных вызовов
- **Авторизация** — Запрет или разрешение доступа по роли
- **Сериализация / трансформация** — Преобразование объекта при JSON.stringify

🔮 Шаг 04:

Миссия 4

Сопоставьте аспект автоматизации с технической деталью:

Ответ:

- Обертывание метода
 - Переопределение геттера/сеттера
 - Добавление метода toJSON
 - Проверка прав доступа перед методом
1. Использование descriptor.value и function replacement
 2. Использование descriptor.get / descriptor.set
 3. Добавить ключ в prototype или объект для toJSON
 4. Чтение метаданных ролей и throw error, если нет доступа

Подсказка: Все механизмы связаны с descriptor и метаданными.

- **Обертывание метода** — Использование descriptor.value и function replacement
- **Переопределение геттера/сеттера** — Использование descriptor.get / descriptor.set
- **Добавление метода toJSON** — Добавить ключ в prototype или объект для toJSON
- **Проверка прав доступа перед методом** — Чтение метаданных ролей и throw error, если нет доступа

🔮 Шаг 05:

Миссия 5

Светлана хочет, чтобы при ошибке валидации автоматически логировалась причина и выбрасывалось исключение. Какой шаг декоратора подойдёт?

Подсказка: Нужно перехватить результат проверки и обработать ошибку.

Ответ:

- **Оборачивать оригинальный метод в try/catch внутри декоратора**
- Игнорировать ошибки и возвращать undefined
- Писать логи в консоль после выполнения метода

🔮 Шаг 06:

Миссия 6

Сопоставьте методы автоматизации и необходимые данные, которые нужно хранить в метаданных:

Ответ:

- Логирование
- Кэширование
- Авторизация
- Тайминг

1. Уровень доступа / роли
2. Мар с аргументами и результатами
3. Метка, что метод логируется
4. Флаг, что измерять время

Подсказка: Какие данные нужны, чтобы автоматизировать поведение?

- **Логирование** — Метка, что метод логируется
- **Кэширование** — Мар с аргументами и результатами
- **Авторизация** — Уровень доступа / роли
- **Тайминг** — Флаг, что измерять время

Шаг 07:

Миссия 7

Сопоставьте автоматизацию поведения с местом применения декоратора:

Ответ:

- Метод контроллера HTTP
 - Сервисный метод бизнес-логики
 - Свойство конфигурации
 - Класс плагина
1. Авторизация / проверка ролей
 2. Кэширование вычислений
 3. Десериализация / трансформация данных поля
 4. Регистрация плагинов в контейнере

Подсказка: Где логично автоматизировать каждый аспект?

- **Метод контроллера HTTP** — Авторизация / проверка ролей
- **Сервисный метод бизнес-логики** — Кэширование вычислений
- **Свойство конфигурации** — Десериализация / трансформация данных поля
- **Класс плагина** — Регистрация плагинов в контейнере

Шаг 08:

Миссия 8

Расположите шаги автоматизации метода через декоратор (логика + вспомогательные шаги):

1. Чтение метаданных, описывающих поведение (логирование, тайминг и др.)
2. Оборачивание оригинального метода в функцию-обёртку
3. Внутри обёртки выполнить дополнительные действия (лог, проверка, тайминг)
4. Вызвать оригинальный метод и вернуть результат

Подсказка: Сначала прочитать, затем обернуть, внутри обёртки — вспомогательные шаги, потом вызов метода.

Ответ:

- Чтение метаданных, описывающих поведение (логирование, тайминг и др.)
- Оборачивание оригинального метода в функцию-обёртку
- Внутри обёртки выполнить дополнительные действия (лог, проверка, тайминг)
- Вызвать оригинальный метод и вернуть результат

Шаг 09:

Миссия 9

Расположите шаги при автоматизации свойства через декоратор (валидатор или фильтр):

1. Сохранить исходное значение / приватное поле
2. В `descriptor.set` перехватить присваивание
3. Применить проверку / трансформацию к новому значению
4. Записать проверенное / трансформированное значение в приватное поле

Подсказка: Сначала сохранить базу, потом перехват, проверка, запись.

Ответ:

- Сохранить исходное значение / приватное поле
- В `descriptor.set` перехватить присваивание
- Применить проверку / трансформацию к новому значению
- Записать проверенное / трансформированное значение в приватное поле

Шаг 10:

Миссия 10

Расположите фазы внедрения автоматизации в архитектуре приложения:

1. Декорировать контроллеры и сервисы нужными декораторами поведения
2. **При старте приложения** — сканировать классы и регистрировать метаданные
3. **Когда контейнер создаёт экземпляр** — читать метаданные и применять аспекты
4. **При вызове метода** — выполнять автоматизированные шаги: лог, проверка, кэш и т.д.

Подсказка: Архитектура: регистрация → чтение → применение при вызове.

Ответ:

- Декорировать контроллеры и сервисы нужными декораторами поведения
- **При старте приложения** — сканировать классы и регистрировать метаданные
- **Когда контейнер создаёт экземпляр** — читать метаданные и применять аспекты
- **При вызове метода** — выполнять автоматизированные шаги: лог, проверка, кэш и т.д.

Шаг 11:

Миссия 11

Расположите шаги создания комбинированного декоратора, который объединяет логирование + тайминг + кэширование:

1. Определить структуру метаданных, хранящую поведение в массиве аспектов
2. В декораторе метода прочитать этот массив аспектов из метаданных
3. Обёрнуть метод одной функцией, которая по очереди применяет аспекты в нужном порядке

4. Запустить оригинальный метод внутри обёртки и вернуть итоговый результат

Подсказка: Используйте композицию аспектов внутри одной обёртки.

Ответ:

- Определить структуру метаданных, хранящую поведение в массиве аспектов
- В декораторе метода прочитать этот массив аспектов из метаданных
- Обёрнуть метод одной функцией, которая по очереди применяет аспекты в нужном порядке
- Запустить оригинальный метод внутри обёртки и вернуть итоговый результат

Шаг 12:

Миссия 12

Расположите шаги реализации плагин-системы автоматизированных аспектов:

1. Сканировать директории / модули на наличие классов с аспект-декораторами
2. Регистрация аспектов / плагинов в реестре с их метаданными
3. При инициализации приложения связывать плагины с точками входа (методами)
4. **При вызове метода** — применять зарегистрированные плагины в нужном порядке

Подсказка: От поиска до применения.

Ответ:

- Сканировать директории / модули на наличие классов с аспект-декораторами
- Регистрация аспектов / плагинов в реестре с их метаданными
- При инициализации приложения связывать плагины с точками входа (методами)
- **При вызове метода** — применять зарегистрированные плагины в нужном порядке

Шаг 13:

Миссия 13

Расположите шаги эволюции автоматизации от простого к сложному:

1. Обычный метод без декоратора
2. Декоратор логирования одного метода
3. Декоратор комбинации аспектов (лог + тайминг + кэш)
4. Плагин-архитектура, расширяемая пользователем

Подсказка: От простого к плагинам.

Ответ:

- Обычный метод без декоратора
- Декоратор логирования одного метода
- Декоратор комбинации аспектов (лог + тайминг + кэш)
- Плагин-архитектура, расширяемая пользователем

Урок 5 - Use-case NestJS style decorators

Шаг 01:

Миссия 1

Алекс развивает API на NestJS и хочет использовать декоратор @Controller. Какова его основная роль?

Подсказка: Подумайте, что делает контроллер в MVC-архитектуре.

Ответ:

- **Определяет класс, который обрабатывает маршруты (routes) и HTTP-запросы**
- Служит для описания схемы БД
- Используется только для логирования

🔗 **Шаг 02:**

Миссия 2

Мария использует `@Get('users')` над методом контроллера. Что делает этот декоратор?

Подсказка: Подумайте, что происходит при получении HTTP GET запроса.

Ответ:

- **Этот метод будет обрабатывать HTTP GET запросы по пути '/users'**
- Создает POST-маршрут
- Отправляет статический файл

🔗 **Шаг 03:**

Миссия 3

Игорь создает декоратор `@Injectable()` для сервиса. Зачем это нужно?

Подсказка: Подумайте, как контейнер узнаёт, что можно инжецировать сервис.

Ответ:

- **Чтобы NestJS мог зарегистрировать класс в DI контейнере и инжецировать его**
- Чтобы класс стал маршрутом
- Чтобы выполнить сериализацию объекта автоматически

🔗 **Шаг 04:**

Миссия 4

Сопоставьте NestJS декораторы с объектами, к которым они применяются:

Ответ:

- `@Module`
 - `@Controller`
 - `@Injectable`
 - `@Module-providers`
1. Класс, содержащий controllers и providers
 2. Класс, отвечающий за HTTP маршруты
 3. Класс, который может быть инжецируемым сервисом
 4. Список классов-сервисов, регистрируемых внутри модуля

Подсказка: Если декоратор говорит «внедряемость» — это `Injectable`, если «HTTP запросы» — `Controller`, если структура приложения — `Module`.

- **@Module** — Класс, содержащий controllers и providers
- **@Controller** — Класс, отвечающий за HTTP маршруты
- **@Injectable** — Класс, который может быть инжецируемым сервисом
- **@Module-providers** — Список классов-сервисов, регистрируемых внутри модуля

🔗 **Шаг 05:**

Миссия 5

Сопоставьте HTTP методы и декораторы в NestJS:

Ответ:

- @Post()
- @Delete()
- @Put()
- @Patch()

1. Обновление части ресурса
2. Полное обновление ресурса
3. Создание нового ресурса
4. Удаление ресурса

Подсказка: Подумайте, какой HTTP-глагол соответствует каждому действию CRUD.

- **@Post()** — Создание нового ресурса
- **@Delete()** — Удаление ресурса
- **@Put()** — Полное обновление ресурса
- **@Patch()** — Обновление части ресурса

Шаг 06:

Миссия 6

Сопоставьте параметры декораторов и то, что они настраивают:

Ответ:

- @Get('path')
- @Controller('prefix')
- @Inject(token)
- @Body()

1. Путь маршрута в методе
2. Префикс маршрутов в контроллере
3. Токен/идентификатор для DI зависимости
4. Декорирование тела запроса HTTP POST/PUT

Подсказка: Каждый декоратор может принимать аргументы — они влияют на поведение маршрутов или DI.

- **@Get('path')** — Путь маршрута в методе
- **@Controller('prefix')** — Префикс маршрутов в контроллере
- **@Inject(token)** — Токен/идентификатор для DI зависимости
- **@Body()** — Декорирование тела запроса HTTP POST/PUT

Шаг 07:

Миссия 7

Расположите шаги обработки HTTP запроса в NestJS контроллере:

1. Маршрут контроллера совпадает с запросом URL и HTTP методом
2. NestJS вызывает метод внутри контроллера с параметрами (например, @Body, @Param)
3. Методы-middleware или guards предваряют обработку (если есть декораторы типа @UseGuards)
4. Ответ возвращается клиенту через HTTP

Подсказка: Подумайте, как запрос проходит от сети к контроллеру и обратно.

Ответ:

- Маршрут контроллера совпадает с запросом URL и HTTP методом
- NestJS вызывает метод внутри контроллера с параметрами (например, @Body, @Param)
- Методы-middleware или guards предваряют обработку (если есть декораторы типа @UseGuards)
- Ответ возвращается клиенту через HTTP

Шаг 08:

Миссия 8

Расположите этапы регистрации провайдера в NestJS модуле:

1. Создать класс сервиса с @Injectable()
2. Добавить класс в массив providers в @Module()
3. Импортировать модуль, если потребуется из другого модуля
4. Инъектировать сервис в контроллер или другой сервис через конструктор

Подсказка: Регистрация → импорт → инъекция.

Ответ:

- Создать класс сервиса с @Injectable()
- Добавить класс в массив providers в @Module()
- Импортировать модуль, если потребуется из другого модуля
- Инъектировать сервис в контроллер или другой сервис через конструктор

Шаг 09:

Миссия 9

Расположите шаги создания middleware или guard с декоратором:

1. Определить класс guard/middleware с нужным интерфейсом / контрактом
2. Применить декоратор @UseGuards или @UseMiddleware в контроллере или маршруте
3. Внутри guard проверять условия (например, авторизация, роль)
4. **Если проверка успешна** — передать управление дальше, иначе — отклонить ответ

Подсказка: Проверка доступа идет перед выполнением обработчика метода.

Ответ:

- Определить класс guard/middleware с нужным интерфейсом / контрактом
- Применить декоратор @UseGuards или @UseMiddleware в контроллере или маршруте
- Внутри guard проверять условия (например, авторизация, роль)
- **Если проверка успешна** — передать управление дальше, иначе — отклонить ответ

Шаг 10:

Миссия 10

Расположите этапы обработки параметров запроса в NestJS методе:

1. Клиент отправляет запрос с параметрами в URL, body, query или headers
2. NestJS распознаёт декораторы параметров (например, @Param, @Body, @Query)
3. Проводится валидация или трансформация параметров (если используются Pipes)
4. Метод контроллера вызывается с уже трансформированными / валидированными аргументами

Подсказка: Порядок: получение → распознавание → валидация/трансформация → вызов.

Ответ:

- Клиент отправляет запрос с параметрами в URL, body, query или headers
- NestJS распознаёт декораторы параметров (например, @Param, @Body, @Query)
- Проводится валидация или трансформация параметров (если используются Pipes)
- Метод контроллера вызывается с уже трансформированными / валидированными аргументами

 Шаг 11:

Миссия 11

Расположите шаги выполнения глобального exception filter в NestJS:

1. Создать фильтр с интерфейсом ExceptionFilter
2. Зарегистрировать его глобально или на уровне контроллера или метода через декоратор @UseFilters или через app.useGlobalFilters()
3. Фильтр получает исключение и контекст обработки запроса
4. Фильтр возвращает корректный HTTP ответ с кодом и сообщением

Подсказка: Exception filter вмешивается, когда что-то идёт не так.

Ответ:

- Создать фильтр с интерфейсом ExceptionFilter
- Зарегистрировать его глобально или на уровне контроллера или метода через декоратор @UseFilters или через app.useGlobalFilters()
- Фильтр получает исключение и контекст обработки запроса
- Фильтр возвращает корректный HTTP ответ с кодом и сообщением

 Шаг 12:

Миссия 12

Расположите этапы внедрения Pipe для валидации/трансформации в NestJS:

1. Создать класс Pipe, реализующий интерфейс PipeTransform
2. Определить логику transform и возможно validation внутри него
3. Применить Pipe через декоратор @UsePipes или параметр метода (например, @Body(new ValidationPipe()))
4. При выполнении запроса Nest вызывает Pipe до метода контроллера, преобразуя или валидируя данные

Подсказка: Pipe действует между получением данных и исполнением метода.

Ответ:

- Создать класс Pipe, реализующий интерфейс PipeTransform
- Определить логику transform и возможно validation внутри него
- Применить Pipe через декоратор @UsePipes или параметр метода (например, @Body(new ValidationPipe()))
- При выполнении запроса Nest вызывает Pipe до метода контроллера, преобразуя или валидируя данные

 Шаг 13:

Миссия 13

Расположите этапы использования interceptor в NestJS:

1. Создать класс interceptor, реализующий интерфейс NestInterceptor
2. Определить логику пред и пост обработки (до вызова метода и после него)
3. Применить декоратор @UseInterceptors() к контроллеру или методу
4. Интерцептор оборачивает вызов и может манипулировать входом / выходом

Подсказка: Interceptor — это AOP-стиль: что до и что после метода.

Ответ:

- Создать класс interceptor, реализующий интерфейс NestInterceptor
- Определить логику пред и пост обработки (до вызова метода и после него)
- Применить декоратор @UseInterceptors() к контроллеру или методу
- Интерцептор оборачивает вызов и может манипулировать входом / выходом

Шаг 14:

Миссия 14

Расположите этапы обработки запросов с Guards + Pipes:

1. Guard проверяет авторизацию / роль пользователя
2. Pipe трансформирует или валидирует переданные данные
3. Контроллер получает валидированные параметры
4. Метод контроллера выполняется и отправляет ответ

Подсказка: Правила безопасности прежде, чем данные будут доверены.

Ответ:

- Guard проверяет авторизацию / роль пользователя
- Pipe трансформирует или валидирует переданные данные
- Контроллер получает валидированные параметры
- Метод контроллера выполняется и отправляет ответ

Шаг 15:

Миссия 15

Расположите шаги внедрения Custom Decorator (например, @Roles(...)) в NestJS:

1. Определить декоратор @Roles, который сохраняет роли в метаданных
2. Использовать Reflect.getMetadata внутри Guard, чтобы получить роли из декораторов
3. Применить @Roles(...) над методом или контроллером
4. В Guard сравнить роли пользователя и требуемые роли, разрешить или запретить доступ

Подсказка: Декораторы + Guard работают вместе: один задаёт, другой проверяет.

Ответ:

- Определить декоратор @Roles, который сохраняет роли в метаданных
- Использовать Reflect.getMetadata внутри Guard, чтобы получить роли из декораторов
- Применить @Roles(...) над методом или контроллером
- В Guard сравнить роли пользователя и требуемые роли, разрешить или запретить доступ

Шаг 16:

Миссия 16

Расположите этапы создания globally applied middleware / filter / interceptor:

1. Написать класс с нужным интерфейсом (Middleware, Filter или Interceptor)
2. Определить логику, которая будет выполняться для всех или многих маршрутов / контроллеров
3. Зарегистрировать глобально в приложении (например, app.useGlobalInterceptors, useGlobalFilters или use) в main.ts
4. Убедиться, что все HTTP запросы проходят через эту логику автоматически

Подсказка: Глобальность означает, что не нужно ставить декораторы на каждый метод.

Ответ:

- Написать класс с нужным интерфейсом (Middleware, Filter или Interceptor)
- Определить логику, которая будет выполняться для всех или многих маршрутов / контроллеров
- Зарегистрировать глобально в приложении (например, app.useGlobalInterceptors, useGlobalFilters или use) в main.ts
- Убедиться, что все HTTP запросы проходят через эту логику автоматически

Шаг 17:

Миссия 17

Расположите полное прохождение запроса с NestJS декораторами, от запроса до ответа, включая Guards, Pipes, Interceptors, Filters:

1. Клиент делает HTTP запрос на определённый маршрут
2. Guard проверяет авторизацию / роль (если применён)
3. Pipe проверяет и трансформирует входные данные
4. Interceptor выполняет pre-обработку (например лог, тайминг)
5. Контроллер метод вызывается с валидированными & трансформированными параметрами
6. Interceptor выполняет post-обработку результата
7. **Если возникло исключение** — Filter перехватывает и возвращает ответ ошибки

Подсказка: Схема middleware-стиля: сначала защитные меры, потом данные, затем логика, затем ответы и ошибки.

Ответ:

- Клиент делает HTTP запрос на определённый маршрут
- Guard проверяет авторизацию / роль (если применён)
- Pipe проверяет и трансформирует входные данные
- Interceptor выполняет pre-обработку (например лог, тайминг)
- Контроллер метод вызывается с валидированными & трансформированными параметрами
- Interceptor выполняет post-обработку результата
- **Если возникло исключение** — Filter перехватывает и возвращает ответ ошибки

Урок 6 - Ограничения и конфигурации

Шаг 01:

Миссия 1

Катя использует кастомный декоратор @Readonly для свойства класса. Какую задачу он должен выполнять?

Подсказка: Readonly обычно применяется к полям, которые не должны изменяться после инициализации.

Ответ:

- **Запрещает дальнейшее изменение значения свойства**
- Добавляет логирование при изменении значения
- Автоматически сериализует значение в JSON
- Переименовывает свойство при компиляции

🔗 **Шаг 02:**

Миссия 2

Разработчик Артём добавил кастомный декоратор `@ValidateLength({ min: 5, max: 20 })` к строковому полю. Что это означает?

Подсказка: Часто это используется для ограничения длины ввода, особенно в формах и DTO.

Ответ:

- **Значение поля должно быть строкой длиной от 5 до 20 символов**
- Числовое значение поля должно быть от 5 до 20
- Поле обязательно должно быть числом и больше 20
- Строка должна содержать только латинские символы

🔗 **Шаг 03:**

Миссия 3

Лера добавила декоратор `@MaxValue(1000)` к числовому полю. Что произойдёт, если присвоить 1200?

Подсказка: Такие декораторы часто используются валидационными библиотеками, чтобы ограничивать ввод.

Ответ:

- **Сработает исключение или валидационная ошибка**
- Значение автоматически округлится до 1000
- Никаких ограничений не будет
- Поле станет строкой

🔗 **Шаг 04:**

Миссия 4

Евгений использует декоратор `@Configurable(false)` на методе. Что он пытается сделать?

Подсказка: `config: false` запрещает дальнейшую модификацию свойства через `Object.defineProperty`.

Ответ:

- **Запретить переопределение или удаление метода**
- Сделать метод асинхронным
- Добавить лог при каждом вызове метода
- Автоматически кешировать результат метода

🔗 **Шаг 05:**

Миссия 5

Никита создал декоратор `@Final`. Какое поведение он, скорее всего, ожидает?

Подсказка: Это понятие пришло из Java. Представьте, что вы хотите заблокировать переопределение.

Ответ:

- **Метод или свойство нельзя будет переопределить в наследнике**
- Метод нельзя вызывать напрямую
- Метод становится приватным
- Метод становится read-only

Шаг 06:

Миссия 6

Сопоставьте кастомные декораторы с их предполагаемой задачей:

Ответ:

- @MinValue(0)
 - @Trim()
 - @IsEmail()
 - @UpperCase()
1. Удаляет пробелы в начале и в конце строки
 2. Проверяет корректность формата email
 3. Значение строки приводится к верхнему регистру
 4. Ограничивает значение снизу

Подсказка: Все декораторы изменяют или валидируют значение.

- **@MinValue(0)** — Ограничивает значение снизу
- **@Trim()** — Удаляет пробелы в начале и в конце строки
- **@IsEmail()** — Проверяет корректность формата email
- **@UpperCase()** — Значение строки приводится к верхнему регистру

Шаг 07:

Миссия 7

Сопоставьте декораторы с типами, к которым они применяются:

Ответ:

- @ReadOnly
 - @Injectable
 - @ValidateLength
 - @Configurable(false)
1. Свойства
 2. Классы
 3. Методы
 4. Поля с ограничениями по длине

Подсказка: Некоторые декораторы применимы только к определённым типам (например, класс или свойство).

- **@ReadOnly** — Свойства
- **@Injectable** — Классы
- **@ValidateLength** — Поля с ограничениями по длине
- **@Configurable(false)** — Методы

🔧 Шаг 08:

Миссия 8

Сопоставьте ограничения с примерами поведения:

Ответ:

- Максимальная длина строки
- Только числа от 0 до 100
- Неизменяемое свойство
- Метод нельзя удалить

1. @MaxLength(50)
2. @Range(0, 100)
3. @ReadOnly
4. @Configurable(false)

Подсказка: Вспомните, какие ограничения типичны при валидации и защите структуры объектов.

- **Максимальная длина строки** — @MaxLength(50)
- **Только числа от 0 до 100** — @Range(0, 100)
- **Неизменяемое свойство** — @ReadOnly
- **Метод нельзя удалить** — @Configurable(false)

🔧 Шаг 09:

Миссия 9

Сопоставьте названия декораторов с их действием:

Ответ:

- @Optional()
- @Default('N/A')
- @Positive()
- @Pattern(/^\d{4}-\d{2}-\d{2}\$/)

1. Разрешает значение быть неопределённым
2. Присваивает значение по умолчанию
3. Проверяет, что число положительное
4. Валидирует по шаблону (например, дата)

Подсказка: Названия декораторов подсказывают, какое поведение ожидается от поля или аргумента.

- **@Optional()** — Разрешает значение быть неопределённым
- **@Default('N/A')** — Присваивает значение по умолчанию
- **@Positive()** — Проверяет, что число положительное
- **@Pattern(/^\d{4}-\d{2}-\d{2}\$/)** — Валидирует по шаблону (например, дата)

🔧 Шаг 10:

Миссия 10

Сопоставьте тип декоратора с тем, к чему он применяется:

Ответ:

- Декоратор класса

- Декоратор метода
- Декоратор параметра
- Декоратор свойства

1. @Controller()
2. @Log()
3. @Inject()
4. @Readonly

Подсказка: В зависимости от позиции, TypeScript предоставляет разную сигнатуру декораторам.

- **Декоратор класса** — @Controller()
- **Декоратор метода** — @Log()
- **Декоратор параметра** — @Inject()
- **Декоратор свойства** — @Readonly

Шаг 11:

Миссия 11

Сопоставьте описание с типом ограничивающего декоратора:

Ответ:

- Гарантирует, что значение находится в диапазоне
- Устанавливает значение, если оно не указано
- Разрешает передавать значение null
- Форматирует значение перед сохранением

1. @Range(1, 10)
2. @Default('admin')
3. @Nullable()
4. @Transform()

Подсказка: Валидация и трансформация — часто идут вместе, но выполняют разные роли.

- **Гарантирует, что значение находится в диапазоне** — @Range(1, 10)
- **Устанавливает значение, если оно не указано** — @Default('admin')
- **Разрешает передавать значение null** — @Nullable()
- **Форматирует значение перед сохранением** — @Transform()

Шаг 12:

Миссия 12

Сопоставьте ограничения конфигурации с их эффектом:

Ответ:

- enumerable: false
- writable: false
- configurable: false
- enumerable: true

1. Свойство не будет видно при переборе
2. Значение свойства нельзя изменить
3. Нельзя удалить или переопределить свойство
4. Свойство видно при переборе

Подсказка: Эти флаги — часть `Object.defineProperty`, и декораторы могут их управлять.

- **enumerable: false** — Свойство не будет видно при переборе
- **writable: false** — Значение свойства нельзя изменить
- **configurable: false** — Нельзя удалить или переопределить свойство
- **enumerable: true** — Свойство видно при переборе

Урок 7 - Квест Реализовать мини-фреймворк с декораторами

Шаг 01:

Миссия 1

Алина хочет, чтобы её мини-фреймворк понимал декоратор `@Controller` для классов. Что первым делом нужно реализовать?

Подсказка: Нужно, чтобы фреймворк мог «читать», что этот класс помечен как контроллер.

Ответ:

- **Сохранять метаданные о классе как контроллер**
- Автоматически регистрировать все методы класса как роуты
- Запускать методы контроллера сразу при старте приложения

Шаг 02:

Миссия 2

Сопоставьте части мини-фреймворка и что они должны делать:

Ответ:

- Метаданные маршрута
 - Контроллер
 - Регистратор маршрутов
 - Поддержка параметров (`@Param`, `@Body`)
1. Читает декораторы методов и строит таблицу маршрутов
 2. Класс, отмеченный как набор методов HTTP
 3. Сохраняет путь и метод (GET/POST) для метода контроллера
 4. Обработывает аргументы метода из запроса

Подсказка: Контроллер + декораторы + механизм построения маршрутов + аргументы запроса.

- **Метаданные маршрута** — Сохраняет путь и метод (GET/POST) для метода контроллера
- **Контроллер** — Класс, отмеченный как набор методов HTTP
- **Регистратор маршрутов** — Читает декораторы методов и строит таблицу маршрутов
- **Поддержка параметров (`@Param`, `@Body`)** — Обработывает аргументы метода из запроса

Шаг 03:

Миссия 3

Сопоставьте этапы во время обработки HTTP запроса мини-фреймворком:

Ответ:

- Найти маршрут
- Вызвать метод контроллера
- Распарсить параметры

- Обработать декораторы контроллера

1. Сканирование контроллера и регистрация маршрутов при старте
2. **При получении запроса** — найти контроллер и метод
3. **Перед вызовом** — подготовить аргументы (Param, Body)
4. **Наконец** — вызвать метод с аргументами и вернуть ответ

Подсказка: Запрос движется по маршруту: регистрация → обработка запроса → подготовка → вызов.

- **Найти маршрут** — При получении запроса — найти контроллер и метод
- **Распарсить параметры** — Перед вызовом — подготовить аргументы (Param, Body)
- **Обработать декораторы контроллера** — Сканирование контроллера и регистрация маршрутов при старте
- **Вызвать метод контроллера** — Наконец — вызвать метод с аргументами и вернуть ответ

🔮 Шаг 04:

Миссия 4

Расположите шаги регистрации маршрутов при запуске приложения:

1. Сканировать все контроллеры, помеченные @Controller
2. Для каждого метода считывать метаданные маршрутов (Get, Post и т.д.)
3. Сохранить маршрут + метод + функцию-обработчик в таблице маршрутов
4. Запустить HTTP сервер, используя зарегистрированные маршруты

Подсказка: Регистрация маршрутов — подготовительный этап перед обслуживанием запросов.

Ответ:

- Сканировать все контроллеры, помеченные @Controller
- Для каждого метода считывать метаданные маршрутов (Get, Post и т.д.)
- Сохранить маршрут + метод + функцию-обработчик в таблице маршрутов
- Запустить HTTP сервер, используя зарегистрированные маршруты

🔮 Шаг 05:

Миссия 5

Расположите шаги при обработке входящих параметров метода (Param и Body):

1. Считать метаданные параметров (@Param, @Body)
2. Извлечь соответствующие данные из HTTP запроса (URL, тело, query)
3. Преобразовать / валидировать, если необходимо
4. Передать полученные аргументы в вызов метода контроллера

Подсказка: Порядок: чтение метаданных → извлечение → трансформация → вызов.

Ответ:

- Считать метаданные параметров (@Param, @Body)
- Извлечь соответствующие данные из HTTP запроса (URL, тело, query)
- Преобразовать / валидировать, если необходимо
- Передать полученные аргументы в вызов метода контроллера

🔮 Шаг 06:

Миссия 6

Расположите шаги, когда метод контроллера возвращает промис / асинхронный результат:

1. Метод контроллера вызывается и возвращает Promise
2. Мини-фреймворк ждёт завершения промиса
3. Получает результат / ошибку
4. Отправляет HTTP ответ клиенту (или ошибку)

Подсказка: Асинхронные методы нужно правильно обрабатывать внутри фреймворка.

Ответ:

- Метод контроллера вызывается и возвращает Promise
- Мини-фреймворк ждёт завершения промиса
- Получает результат / ошибку
- Отправляет HTTP ответ клиенту (или ошибку)

Шаг 07:

Миссия 7

Ольга хочет, чтобы её мини-фреймворк поддерживал middleware (обработчики до маршрута). Какой простой способ это реализовать?

Подсказка: Middleware часто вызываются перед маршрутом, могут менять req/res или прерывать цепочку.

Ответ:

- **При запуске маршрута перед вызовом контроллера вызывать функции middleware**
- Применять middleware как декоратор к каждому методу
- Запускать middleware только после метода контроллера

Шаг 08:

Миссия 8

Максим хочет, чтобы можно было конфигурировать префикс URL для контроллеров (например, all routes контроллера под "/api"). Какой подход наиболее подходящий?

Подсказка: Контроллеру нужен параметр конфигурации при декорации.

Ответ:

- **Декоратор @Controller(prefix) сохраняет этот префикс в метаданных**
- Добавлять префикс вручную ко всем путям
- Игнорировать префикс и использовать корневые пути

Шаг 09:

Миссия 9

Расположите шаги запуска мини-фреймворка после конфигурации:

1. Зарегистрировать все контроллеры и маршруты из метаданных
2. Настроить HTTP сервер и привязать маршруты
3. Применить middleware, если есть глобальные или локальные
4. Запустить сервер и слушать порт

Подсказка: Регистрация → настройки → middleware → запуск.

Ответ:

- Зарегистрировать все контроллеры и маршруты из метаданных
- Настроить HTTP сервер и привязать маршруты

- Применить middleware, если есть глобальные или локальные
- Запустить сервер и слушать порт

🔗 Шаг 10:

Миссия 10

Расположите шаги обработки ошибки, выброшенной внутри контроллера:

1. Метод контроллера бросает исключение / возвращает ошибку
2. Мини-фреймворк перехватывает её (try/catch вокруг вызова метода)
3. Применить глобальный фильтр ошибок, если настроен
4. Отправить HTTP ответ с кодом ошибки и сообщением

Подсказка: В оболочке вызова метода всегда нужен блок перехвата ошибок.

Ответ:

- Метод контроллера бросает исключение / возвращает ошибку
- Мини-фреймворк перехватывает её (try/catch вокруг вызова метода)
- Применить глобальный фильтр ошибок, если настроен
- Отправить HTTP ответ с кодом ошибки и сообщением

🔗 Шаг 11:

Миссия 11

Расположите шаги при обработке асинхронного middleware + контроллера + фильтра ошибок:

1. Вызвать middleware (может быть async)
2. Если middleware пропускает, найти подходящий контроллер и метод
3. Вызвать метод контроллера (await, если Promise)
4. **Если метод бросает** — фильтр ошибок возвращает ответ

Подсказка: Middleware → контроллер → ошибка → ответ.

Ответ:

- Вызвать middleware (может быть async)
- Если middleware пропускает, найти подходящий контроллер и метод
- Вызвать метод контроллера (await, если Promise)
- **Если метод бросает** — фильтр ошибок возвращает ответ

🔗 Шаг 12:

Миссия 12

Расположите шаги, если контроллер возвращает объект с заголовками и телом (формат ответа):

1. Проверить, является ли возвращаемое значение специальным ответом (с заголовками)
2. **Если да** — установить заголовки на объект HTTP ответа
3. Сериализовать тело (JSON) и отправить клиенту
4. **Если нет** — просто отправить результат как JSON

Подсказка: Мини-фреймворк должен различать «сырые» объекты и обёрнутые ответы.

Ответ:

- Проверить, является ли возвращаемое значение специальным ответом (с заголовками)
- **Если да** — установить заголовки на объект HTTP ответа
- Сериализовать тело (JSON) и отправить клиенту

- **Если нет** — просто отправить результат как JSON

🔗 Шаг 13:

Миссия 13

Расположите этапы добавления поддержки версии API (например, /v1/users vs /v2/users):

1. При декорации контроллера / метода указывать версию (например, @Version('v1'))
2. Сохранять версию в метаданных маршрута
3. При обработке запроса сравнивать версию из URL с маршрутом
4. Вызывать метод контроллера только если версии совпадают

Подсказка: Версия — это часть маршрута, которую нужно учесть при регистрации и сопоставлении.

Ответ:

- При декорации контроллера / метода указывать версию (например, @Version('v1'))
- Сохранять версию в метаданных маршрута
- При обработке запроса сравнивать версию из URL с маршрутом
- Вызывать метод контроллера только если версии совпадают

🔗 Шаг 14:

Миссия 14

Расположите шаги полного прохода HTTP запроса в мини-фреймворке с декораторами, middleware и фильтрами:

1. Поступает HTTP запрос с методом и путем
2. Запускаются middleware (возможно цепочка) перед маршрутом
3. Найти контроллер и метод по маршруту и HTTP методу
4. Собрать аргументы метода (Param, Body и т.д.)
5. Вызвать метод контроллера (await, если async)
6. **Если возникла ошибка** — применить фильтр ошибок
7. Отправить ответ клиенту (с заголовками, статусом и телом)

Подсказка: Это итоговая миссия — нужно уложить все этапы в логическую последовательность.

Ответ:

- Поступает HTTP запрос с методом и путем
- Запускаются middleware (возможно цепочка) перед маршрутом
- Найти контроллер и метод по маршруту и HTTP методу
- Собрать аргументы метода (Param, Body и т.д.)
- Вызвать метод контроллера (await, если async)
- **Если возникла ошибка** — применить фильтр ошибок
- Отправить ответ клиенту (с заголовками, статусом и телом)